

KAMP 제조AI데이터셋 활용 권한

No. GB20250503-PU1746259150-223244



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 주조 품질보증 AI 데이터셋, 스마트제조 혁신추진단(주)인터엑스, 네스트필드(주), 2022.12.23., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Casting quality assurance AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 주조 품질보증 AI 데이터셋

업종 : 뿌리(주조) | **목적** : 품질보증 | **유형** : csv

알고리즘 : XGBoost를 이용하여 양품/불량 판별에 중요한 공정 데이터를 학습하고 그 결과를 예측한다.

사용조건 : 콘텐츠 변경허용

제공기관 : 스마트제조혁신추진단 (수행기관 : (주)인터엑스/네스트필드(주))

등록일 : 2022-12-23

이용자 정보

회원 ID : jeanboy

다운로드일 : 2025-05-03 18:30:44

활용목적 : 제조AI 교육 및 강의

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

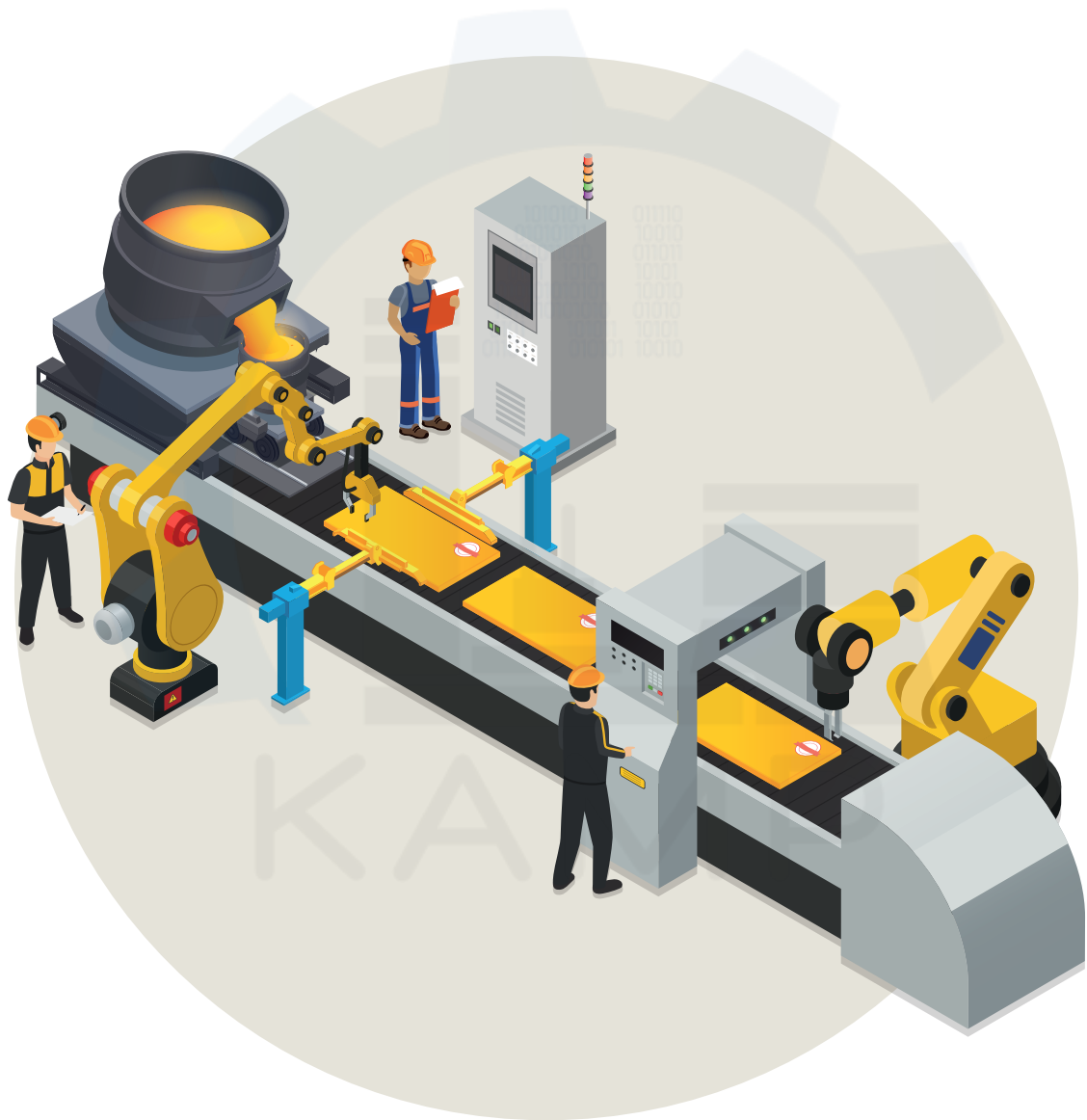
「주조 품질보증 AI 데이터셋」

분석실습 가이드북



「주조 품질보증 AI 데이터셋」

분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	09
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	11
2.1 제조데이터 소개	11
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	22
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	25
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 라이브러리/데이터 불러오기	
[단계 ②] 데이터 종류 및 개수 확인	
[단계 ③] 데이터 특성 파악	
[단계 ④] 데이터 정제(전처리)	
[단계 ⑤] 학습, 평가 데이터 분리	
[단계 ⑥] AI모델 구축	
[단계 ⑦] AI모델 훈련	
[단계 ⑧] 결과 분석 및 해석	
3. 유사 타 현장의 「주조 품질보증 AI 데이터셋」 분석 적용	54
부 록	
분석환경 구축을 위한 설치 가이드	67
데이터 품질 전처리 [실습 코드]	67
AAS S/W 설치 가이드	74



「주조 품질보증 AI 데이터셋」 분석실습 가이드북

- 필요 SW : Python, Anaconda – Jupyter Notebook
- 필요 패키지 : Pandas, Numpy, Sckit-learn, Matplotlib, Seaborn, Imbalanced-learn
- 분석 환경 : [운영체제] Windows 10 Pro 64bit, [CPU] Intel i9-10900 3.70 GHz [RAM] 32GB, [GPU] NVIDIA GeForce RTX 3080
- 필요 데이터 : DieCasting_Quality_Raw_Data.csv

1 분석요약

No	구 분		내 용
1	분석 목적 (현장 이슈, 목적)		다이캐스팅 주조 설비에서 수집되는 공정 데이터 및 센서 데이터를 활용하여 품질 판정을 위한 지도학습(Supervised Learning) 알고리즘을 개발함으로써, 다이캐스팅 주조의 매 shot 마다 수집되는 데이터를 기반으로 별도의 품질검사 없이 실시간 자동으로 품질 결과를 예측하고자 한다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : Shot, Velocity_1 외 55개 2) 데이터 수집 방법 : 설비 shot 단위 공정 조건, 센서 데이터, 양품/불량(불량유형) 이력 데이터 3) 데이터셋 파일 확장자 : csv
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 429,495개(row 7,535개*column 57개) - 데이터셋 총량 : 1.43 MB
4	분석적용 알고리즘	알고리즘	XGBoost를 이용하여 양품/불량 판별에 중요한 공정 데이터를 학습하고 그 결과를 예측한다.
		알고리즘 간략소개	- XGBoost 알고리즘은 Machine Learning(ML) 알고리즘으로, 학습을 통해 데이터의 규칙을 자동으로 찾아내어 분류와 예측을 하는 강력한 알고리즘이다. - 지도학습의 분류, 회귀 분석에 사용되는 다수의 결정 트리들을 학습하는 앙상블(Ensemble) 학습 방법의 일종으로, 일반적인 기계 학습 모델에 비해 성능이 좋은 것으로 알려져 있다.
5	분석결과 및 시사점		- 다이캐스팅 주조품의 양품과 불량을 예측하기 위해 XGBoost를 활용하여 AI 모델을 개발하였다. - 다이캐스팅 주조 설비의 매 shot 마다 수집되는 공정 데이터를 중심으로 관련 센서 데이터까지 활용하여 AI 분류 모델을 생성함으로써 실시간 양품과 불량률의 결과를 일관성 있게 판단할 수 있도록 지원한다. - 학습 모델을 통한 품질검사 결과는 공정 데이터와 함께 시스템에 관리됨으로써 이후 발생하는 품질 문제에 체계적으로 대응할 수 있을 것으로 기대한다.

2 분석 실습

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

• 다이캐스팅(Die-casting) 정의 :

- 다이캐스팅은 금형을 이용하여 용융 금속을 금형 내에 높은 압력으로 빠르게 주입하는 장치이며, 용탕의 냉각 속도가 빠른 주조법이다.

• 다이캐스팅 불량 유형 :

- 미성형(Short Shot): 캐비티 내 충진이 완벽히 되지 않아 제품이 완성되지 않는 것을 말한다.



[그림 1] 미성형

- 박리(Exfoliation): 제품의 표면 등이 벗겨지는 현상. 충전 응고 부족, 부풀음, 틈층 등이 요인이다.



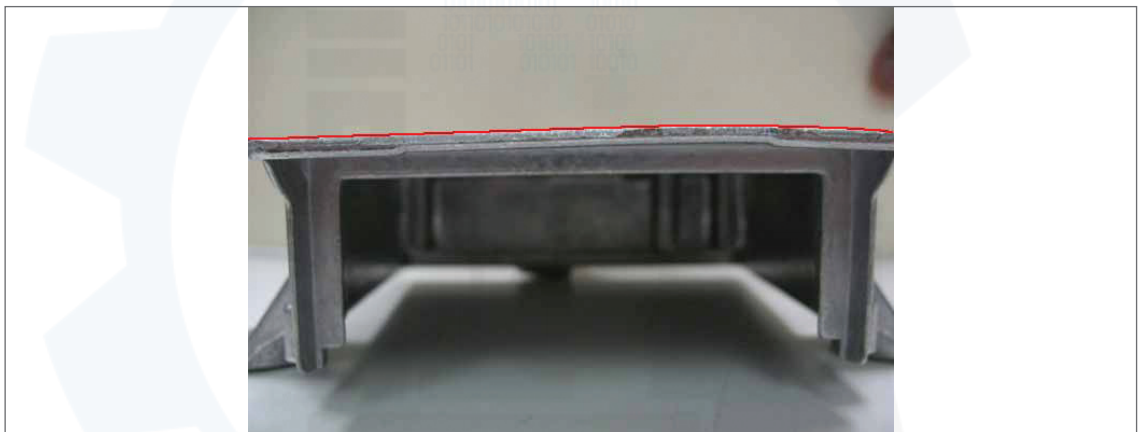
[그림 2] 박리

- 기공(Blow Hole): 고체 재료 속에 기포가 들어가 생긴 중공의 구멍으로써 주물 내부에 생기는 결함의 한 종류이다.



[그림 3] 기공

- 평탄(Deformation): 금속은 응고되면 수축이 생기고 이 수축량이 재료의 각 부분에서 서로 다를 때 내부 잔류 응력이 발생하여 변형이 생기게 되는 결함을 말한다.



[그림 4] 평탄

- 개재물(Inclusions): 용탕 속에 섞이는 각종 불순물로 주형의 청소상태가 불량하거나 용탕, 용해 찌꺼기가 있을 때의 결함을 말한다.



[그림 5] 개재물

• 다이캐스팅 특징 :

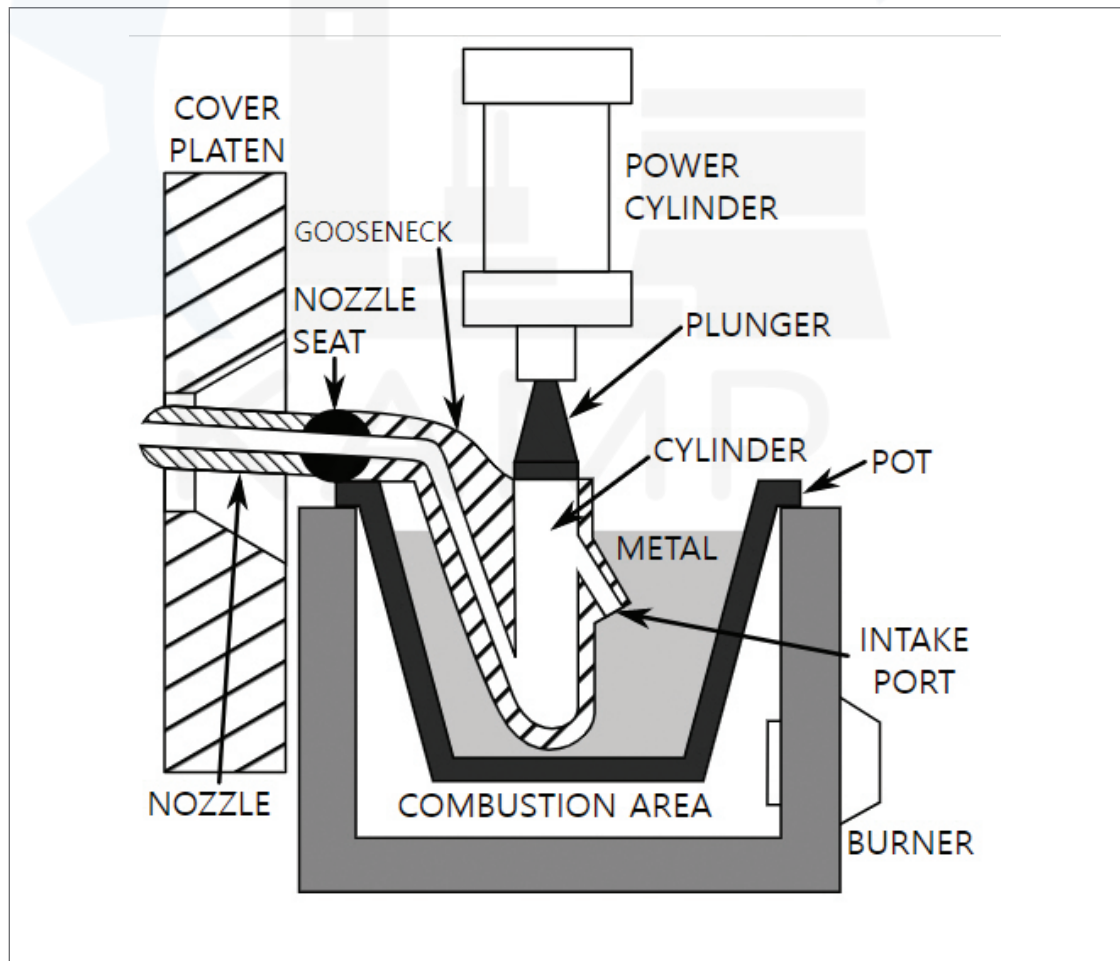
- ① 정밀도가 높고 주물 표면이 깨끗하여 기계 가공을 줄일 수 있다.
- ② 주물의 조직이 치밀하며 강도가 크다.
- ③ 얇은 주물의 주조가 가능하며 제품을 경량화할 수 있다.
- ④ 연속 주조가 가능하며 대량 생산에 적합하다.

• 다이캐스팅 공정에는 크게 열가압실식과 냉가압실식의 두 가지 유형이 있다.

• 다이캐스팅 원리 :

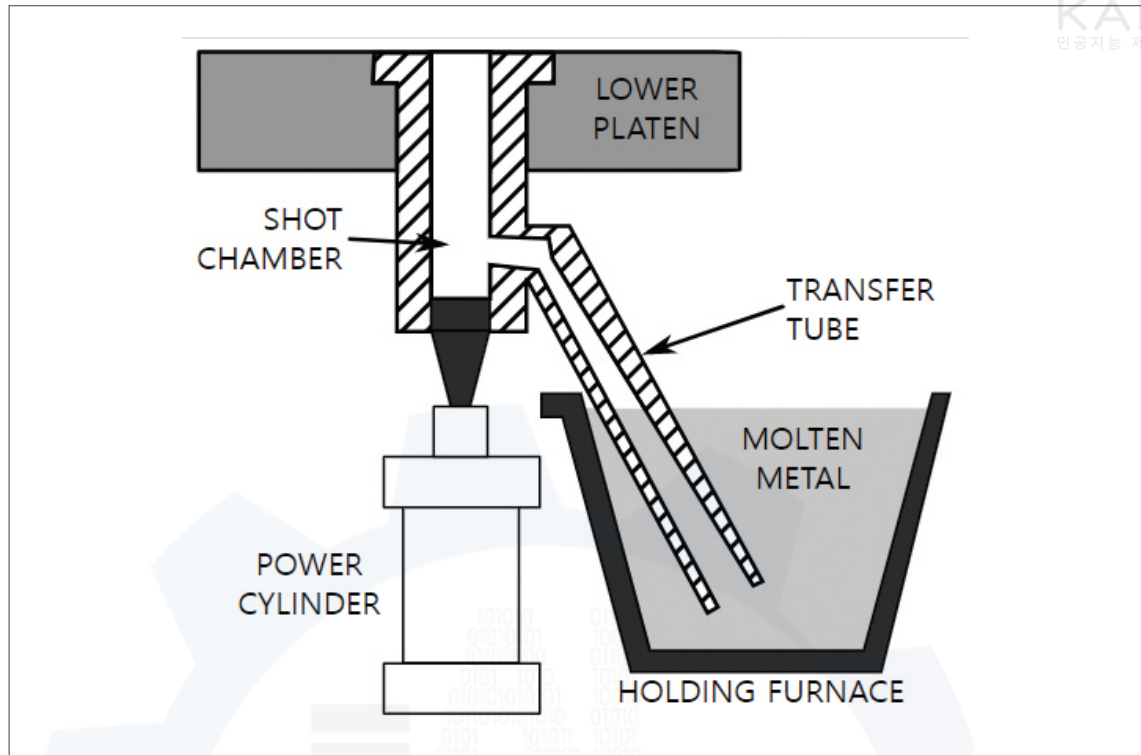
① 열가압실식(Hot Chamber Type)

- 용해로 내에 가압통이 잠겨져 있으며 Plunger를 공압, 수압, 유압으로 가압하여, 용융금속이 노즐을 통해 금형에 압입하는 방식이다.
- 용융금속이 곧바로 금형에 압입되므로 유동성이 양호하다.
- 용융온도가 높은 금속의 주조에 이용한다.
- 가압력 : $30\sim100\text{kg/cm}^2$



[그림 6] 열가압실식 다이캐스팅

② 냉가압실식(Cold Chamber Type)



[그림 7] 냉가압실식 다이캐스팅

③ 열가압실식과 냉가압실식의 차이점

종류	열가압실식	냉가압실식
주조 압력	주조 압력이 작다.	주조 압력이 크다.
주조 속도	주조 속도가 빠르다.	주조 속도가 비교적 느다.
철의 용해량	가압실이 용탕 속에 있으므로, 철의 용해량이 많아진다.	가압실과 노가 분리되어 있으므로, 철의 용해량이 적다.
재료	납, 주석 및 아연 합금과 같은 저용융점 합금을 사용한다.	알루미늄, 마그네슘 및 구리 합금 주조가 가능하다.
주물 크기	주조 압력이 비교적 작으므로 주물은 대체적으로 작다.	주조 압력이 크므로 기공이 작고, 대형도 가능하다.

[표 1] 열가압실식과 냉가압실식의 차이점

2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 다이캐스팅 주조 공정에서는 각종 불량 발생하지만, 현장에서는 생산된 주조품의 불량 판정에 많은 시간과 비용이 요구되기 때문에 실시간 품질검사가 진행되기 어렵고, 추후에 작업자에 의해 육안으로 진행된 품질검사 결과가 시스템 상에서 체계적으로 관리되지 못하고 있다.



- 이로 인해 생산현장에서 발생하는 양품과 불량에 대한 추적 관리가 어렵고, 불량 발생의 원인을 파악하고 조치하는데 어려움을 겪고 있다.

• 문제해결 장애요인

- 제품에 대한 불량검사가 진행되고 있음에도 설비/공정 데이터와 불량검사 결과 간의 정확한 매핑이 어렵다. 또한 현장에서 제품별 이력이 아닌 Lot 단위로 생산 이력이 추적 관리되고 있기 때문에 제품별로 불량 현황을 파악하고 그 원인을 분석하는데 어려움이 존재한다.

• 극복 방안

- 다이캐스팅 주조 설비에서 매 shot 단위로 수집되는 제품별 공정 및 센서 데이터와 그 품질검사 결과를 AI 분석용 데이터셋으로 구성하기 위해 수작업을 통해 데이터 입력과 검증을 진행한다.
- 실시간 수집되는 공정 및 센서 데이터를 학습하여 양품과 불량(유형)을 실시간 자동으로 예측할 수 있는 모델을 개발하여 적용함으로써, 제조현장에서 작업자의 별도 품질검사가 필요 없는 자동화된 품질검사 체계를 구축하고자 한다.

1.2 분석 목표

1) 분석 목표

- 다이캐스팅 주조 설비에서 수집되는 공정 및 센서 데이터를 활용하여 품질 판정을 위한 지도학습(Supervised Learning) 알고리즘을 개발함으로써, 다이캐스팅 주조의 매 shot 마다 수집되는 데이터를 기반으로 별도의 품질검사 없이 실시간 자동으로 품질 결과를 예측하고자 한다.

2) 데이터 정의 및 소개

- 본 데이터셋은 크게 다이캐스팅 주조 설비의 인터페이스를 개발하여 수집한 1) 공정 변수(Process) 데이터, 2) 설비와 공장에 설치한 센서로부터 수집한 센서(Sensor) 데이터, 3) 설비의 매 Shot 마다 생산된 주조품에 대해 품질검사 결과를 진행한 불량(Defects) 데이터로 구성되어 있다. 공정변수는 설비의 매 Shot마다 주기적으로 수집되었으며, 센서는 각 센서의 목적과 사양에 따라 수집주기를 설정하였고, 마지막으로 불량 데이터는 매 Shot에 2개 Cavity에서 취출되는 제품별로 품질검사자에 의해 수기로 진행된 결과를 반영하였다.



3) 데이터 분석 기대효과

- 주조 공정에서 수집되는 다양한 변수들에 따라 양품과 불량률의 실시간으로 예측할 수 있으며, 불량률의 발생 원인을 분석하여 생산성을 높이고 생산 비용을 낮출 수 있을 것으로 기대된다. 이는 공정설비 운영 데이터에 따라 생산품질을 예측할 수 있는 제품의 생산 현장에 폭넓게 활용될 수 있을 것으로 판단된다.

4) 시사점(implication) 요약기술

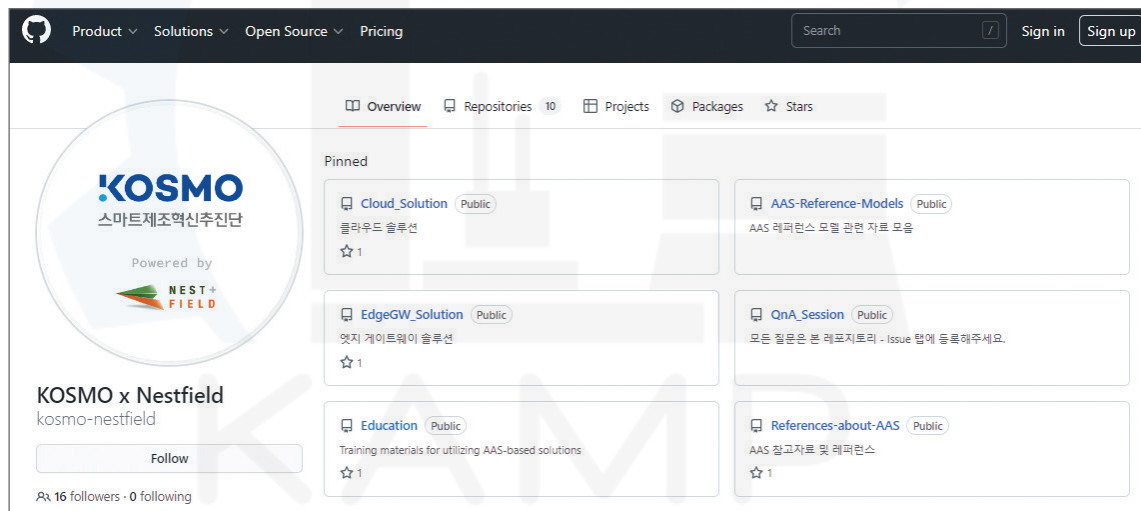
- 현재 다이캐스팅 주조 공정에서 생산되는 제품은 작업자에 의해 육안으로 양품 선별을 진행하지만, 사람의 주관적인 판단에 의해 품질의 균일성을 확보하기 힘든 상황이다.
- 또한 생산공정 이후에 진행되는 품질검사 결과는 공정조건 및 생산 데이터와 연계하여 관리 및 활용되지 못하고 있다. 이로 인해 심각한 불량 문제가 발생하더라도 그 발생 시점과 원인을 정확하게 파악하기 어렵다.
- 본 분석은 다이캐스팅 주조 설비의 매 shot 마다 수집되는 공정 데이터를 중심으로 관련 센서 데이터까지 활용하여 AI 분류 모델을 생성함으로써 실시간 양품과 불량률의 결과를 일관성 있게 판단할 수 있도록 지원한다.
- 또한, 학습 모델을 통한 품질검사 결과는 매 shot 마다 수집되는 공정 데이터와 함께 시스템에 관리됨으로써 이후 발생하는 품질 문제에 체계적으로 대응할 수 있는 기반을 마련하였다.
- 이는 데이터 분석에 열악한 환경을 가진 뿌리산업의 중소기업에 AI를 적용하여 실질적인 품질 및 비용 절감 개선에 기여했다는 점에서 시사점이 크다고 판단된다.

2. 분석실습

2.1 제조데이터 소개

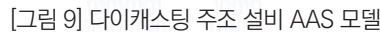
1) 데이터 수집 방법

- **제조 분야** : 자동차 부품
- **제조 공정명** : 다이캐스팅 주조 공정
- **수집장비** : 다이캐스팅 주조 설비 및 센서 데이터 & 품질검사 데이터
- **수집기간** : 2020년 7월 20일 ~ 2020년 7월 30일 (약 10일)
- **수집주기** : 제품 유형에 따라 각 shot 별 데이터와 일정 주기로 수집되는 센서 데이터
- **수집방법** :
 - 본 분석에서는 데이터 수집을 위해 AAS 표준기반 제조데이터 수집/저장 체계가 준비된 상태를 가정한다. 단, 본 데이터 수집/저장 체계 구축을 위한 AAS 모델링 방법 및 솔루션 설치 가이드는 스마트제조혁신추진단과 네스트필드(주)에서 제공하는 오픈소스 제공 사이트인 깃허브 (<https://github.com/kosmo-nestfield>)에서 교육자료를 제공하고 있으므로 본 가이드북에서는 해당 내용을 다루지 않는다.



[그림 8] AAS 표준기반 제조데이터 수집/저장 체계 구축 가이드 배포 사이트

- 제조현장의 설비, 공정에 대한 정보 모델은 IEC 63278-1 AAS (Asset Administration Shell) 기술을 통해 모델링하고 이를 기반으로 AAS 표준기반 제조데이터 수집/저장 체계에 사용될 수 있는 형태로 변환되어 사용된다. 본 분석에 사용된 사출성형 설비의 AAS 파일은 오픈 소스 소프트웨어인 'AASX Package Explorer'를 통해 뷰/편집이 가능하다. AASX Package Explorer 설치는 '부록(AAS S/W 설치 가이드)'를 참고한다.



-



- 현장에서 운용 중인 설비를 제어하기 위해 사용되는 PLC 또는 자체 운용서버(OPC UA 통신 지원)를 통해 수집하는 데이터는 AAS 파일 변환을 통해 생성되는 파일 중 하나인 ‘engineering.csv’에 태그형태로 매핑되어야 한다.

				
DieCasting_Machine.aasx	DieCasting_Machine.xml	engineering.csv	nodeset.xml	syscfg.json

File Name	Description
DieCasting_Machine.aasx	AASX Package Explorer를 통해 생성한 AAS 파일
DieCasting_Machine.xml	AAS 기반 제조데이터 수집/저장 솔루션에서 활용하기 적합한 형태로 변환하기 위한 AAS 파일의 xml 버전
engineering.csv	필드데이터와 AAS 서버 간 태그 매핑을 위한 파일
nodeset.xml	xml 버전의 AAS 파일을 통해 생성한 OPC-UA 서버용 파일
syscfg.json	AAS 서버, 엣지 게이트웨이 그리고 필드기기의 연결을 위한 설정 파일

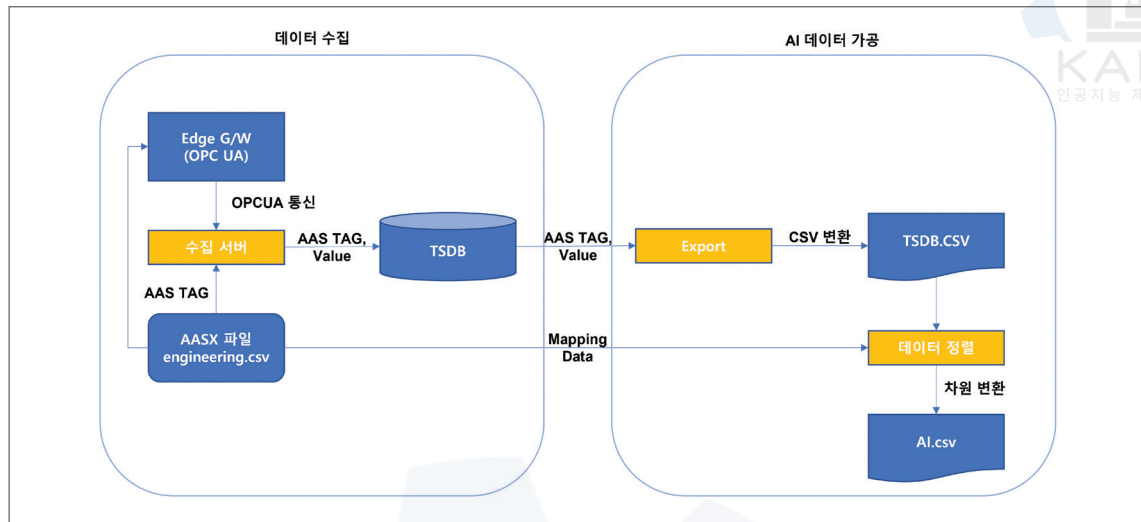
[그림 11] AAS 표준기반 제조데이터 수집/저장 체계 적용을 위한 AAS 파일 및 변환 파일

- engineering.csv 파일에서 A열은 AAS 모델을 통해 생성된 태그명, B열은 엣지 게이트웨이명, C열은 현장 데이터를 수집하는 필드장비명, D열은 현장에서 사용하는 OPC UA 태그명, E열은 ms 단위의 Sampling Interval, F열은 배열 정보, 그리고 G열은 실제 필드에서 사용 중인 변수명이 들어간다. 단, 아래 그림과 같이 외부 공개하기 어려운 부분은 공백으로 표기하였다.

	A	B	C	D	E	F	G	H
52	ns=2s=Die_Casting_Machine.Operational_Data.Defect_Info.Defect_Quantity				100	-1		
53	ns=2s=Die_Casting_Machine.Operational_Data.Defect_Info.Defect_Rate				100	-1		
54	ns=2s=Die_Casting_Machine.Operational_Data.Mold_Info.Enabled_Mold_Number				100	-1		
55	ns=2s=Die_Casting_Machine.Operational_Data.Mold_Info.No_Cavity				100	-1		
56	ns=2s=Die_Casting_Machine.Operational_Data.Mold_Info.Shot				100	-1		
57	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.Velocity_1				100	-1		
58	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.Velocity_2				100	-1		
59	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.Velocity_3				100	-1		
60	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.High_Velocity				100	-1		
61	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.Cylinder_Pressure				100	-1		
62	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.Casting_Pressure				100	-1		
63	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.Rapid_Rise_Time				100	-1		
64	ns=2s=Die_Casting_Machine.Component.Injection_Unit.Component.Cylinder.Operational_Data.Injection_Level_1.Pressure_Rise_Time				100	-1		
65	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Coolant.Operational_Data.Spray_Time				100	-1		
66	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Coolant.Operational_Data.Spray_1_Time				100	-1		
67	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Coolant.Operational_Data.Spray_2_Time				100	-1		
68	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Mold_Open_Close.Mold_Status				100	-1		
69	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Mold_Open_Close.Mold_Open_Time				100	-1		
70	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Mold_Open_Close.Mold_Close_Time				100	-1		
71	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Mold_Open_Close.Mold_Time				100	-1		
72	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Cavity_Number				100	-1		
73	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Temperature.Mold_Pressure_Real_Time				100	-1		
74	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Temperature.Max_Mold_Temp				100	-1		
75	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Temperature.Start_Mold_Temp				100	-1		
76	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Temperature.VP_Mold_Temp				100	-1		
77	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Temperature.Integral_Mold_Temp				100	-1		
78	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Pressure.Mold_Pressure_Real_Time				100	-1		
79	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Pressure.Max_Mold_Pressure				100	-1		
80	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Pressure.Start_Mold_Pressure				100	-1		
81	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Pressure.VP_Mold_Pressure				100	-1		
82	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Pressure.Integral_Mold_Pressure				100	-1		
83	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Fluidity.Shear_Rate				100	-1		
84	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Fluidity.Shear_Stress				100	-1		
85	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Cavity_n.Fluidity.Viscosity				100	-1		
86	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Biscuit_Thickness				100	-1		
87	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Mold.Operational_Data.Clampling_Force				100	-1		
88	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Motor_Drive_n.Identification.Position				100	-1		
89	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Motor_Drive_n.Operational_Data.Start_Status				100	-1		
90	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Motor_Drive_n.Operational_Data.Speed				100	-1		
91	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Motor_Drive_n.Operational_Data.Torque.Average_Torque				100	-1		
92	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Motor_Drive_n.Operational_Data.Torque.Max_Torque				100	-1		
93	ns=2s=Die_Casting_Machine.Component.Clampling_Unit.Component.Motor_Drive_n.Operational_Data.Torque.Min_Torque				100	-1		
94	ns=2s=Sensor.Component.Melting_Furnace_Temp.Sensor.Operational_Data.Melting_Furnace_Temp				100	-1		
95	ns=2s=Sensor.Component.Air_Pressure_Sensor.Operational_Data.Air_Pressure				100	-1		
96	ns=2s=Sensor.Component.Air_Pressure_Sensor.Operational_Data.Air_Pressure_Min				100	-1		
97	ns=2s=Sensor.Component.Air_Pressure_Sensor.Operational_Data.Air_Pressure_Max				100	-1		
98	ns=2s=Sensor.Component.Coolant_Sensor.Operational_Data.Coolant_Temp				100	-1		
99	ns=2s=Sensor.Component.Coolant_Sensor.Operational_Data.Coolant_Temp_Min				100	-1		
100	ns=2s=Sensor.Component.Coolant_Sensor.Operational_Data.Coolant_Temp_Max				100	-1		
101	ns=2s=Sensor.Component.Coolant_Sensor.Operational_Data.Coolant_Pressure				100	-1		
102	ns=2s=Sensor.Component.Factory_Sensor.Operational_Data.Factory_Temp				100	-1		

[그림 12] AAS 변환 파일 중 engineering.csv 파일 구성

- 실제 설비에서 발생하는 데이터들은 의미 없는 값이나 누락이 발생할 수 있어 데이터 품질이 떨어지며, 이는 AAS 표준기반 제조데이터 수집/저장 체계에서도 영향을 받기 때문에 수집된 데이터만으로는 AI 분석에 적합하지 않다. 따라서, 데이터 가공 및 품질 전처리하는 데이터 분석에서 필수적인 단계이다.
- 본 분석에서 AAS 표준기반 제조데이터 수집/저장 체계는 작성한 AASX 파일을 기반으로 Edge Gateway에서 OPC UA 통신을 통해 수집 서버(AAS 서버)로 데이터를 전송한다. 수집 서버는 수신한 데이터에 대하여 AAS TAG에 맞추어 값을 TSDB에 저장하게 된다.



[그림 13] 데이터 수집 및 가공 구성도

- 수집된 데이터는 아래와 같이 명령어를 사용하여 TSDB로부터 CSV 파일을 추출한다.
- 추출한 CSV 파일을 engineering.csv 파일 내 AAS 태그와 매핑된 변수명들을 매칭하여 데이터를 정렬한다. 이렇게 데이터가 모두 정렬되면 분석용 데이터셋 생성을 위한 Raw 데이터를 획득할 수 있다.

```
curl --request -G http://localhost:8086/query?orgID=INFLUX_ORG_ID&database=MyDB&retention_policy=MyRP
--header 'Authorization: Token INFLUX_TOKEN\'
--header 'Accept: application/csv\'
--header 'Content-type: application/json\'
--data-urlencode "q=SELECT used_percent FROM example-db.example-rp.example-measurement WHERE host=host1"
```

[그림 14] 수집 데이터에 대한 CSV 파일 추출 예시

2) 데이터 유형/구조

Process id	Process Product_T	Process Shot	Process Velocity_1	Process Velocity_2	Process Velocity_3	Process High_Velo	Process Cylinder_P	Process Rapid_Rise	Process Biscuit_Thi	Process Clamping	Process Cycle_Tim	Process Pressure	Process Casting_P
1	1	1	0.144	0.17	0.188	2.134	214	0.008	10	258	20.7	0.044	1037
1002	1	2	0.144	0.17	0.182	2.124	217	0.008	11	257	20.7	0.044	1052
2003	1	3	0.144	0.17	0.182	2.116	214	0.008	11	257	20.8	0.041	1037
3004	1	4	0.144	0.17	0.182	2.137	217	0.008	11	257	20.7	0.043	1051
4005	1	5	0.144	0.172	0.176	2.111	217	0.008	12	257	20.7	0.042	1052
5006	1	6	0.144	0.172	0.176	2.111	217	0.008	12	257	20.7	0.042	1052
6008	1	8	0.141	0.168	0.178	2.159	214	0.008	11	257	20.6	0.042	1036
7009	1	9	0.142	0.172	0.176	2.114	217	0.008	12	257	20.7	0.041	1052
8010	1	10	0.142	0.172	0.176	2.114	217	0.008	12	257	20.7	0.041	1052
9011	1	11	0.144	0.168	0.184	2.145	217	0.008	11	257	20.7	0.042	1051
10012	1	12	0.144	0.168	0.184	2.145	217	0.008	11	257	20.7	0.042	1051
11013	1	13	0.144	0.166	0.176	2.12	217	0.008	11	257	20.7	0.044	1052
12014	1	14	0.144	0.17	0.176	2.112	214	0.01	12	257	20.7	0.041	1036
13015	1	15	0.14	0.168	0.184	2.154	217	0.008	11	257	20.8	0.041	1051
14016	1	16	0.142	0.168	0.178	2.13	214	0.007	11	257	20.6	0.043	1036
15017	1	17	0.142	0.168	0.176	2.128	217	0.008	11	257	20.8	0.043	1052
16018	1	18	0.142	0.168	0.176	2.142	214	0.008	11	255	20.7	0.044	1037
17019	1	19	0.142	0.168	0.182	2.114	217	0.009	11	257	20.7	0.044	1051

[그림 13] 데이터 수집 및 가공 구성도



• 데이터셋 구조 :

- 본 데이터셋은 공정 과정에서 수집된 설비 데이터와 센서 데이터에서 비식별화 및 품질 전처리를 수행한 데이터셋이다.
- 1차적인 가공 과정에서 분석실습과 관계없는 항목들을 제외하고, 다이캐스팅 주조 설비에서 각 shot 별로 수집되는 공정 변수 16개, 센서 변수 15개, 불량 변수 26개로 구성되어 있다.

• 데이터 개수 : 57개 변수, 7535개의 관측치

• 제조AI데이터셋 주요 변수 정의 :

Attributes name	Description	Data Type
Shot	Shot 번호	int64
Velocity_1	금형에 용탕을 투입하는 속도	float64
Velocity_2		float64
Velocity_3		float64
High_Velocity		float64
Cylinder_Pressure	Shot을 찍기 위한 실린더 압력	int64
Rapid_Rise_Time	실린더 압력의 상승 시간	float64
Biscuit_Thickness	주조 비스켓의 두께	int64
Clamping_Force	Shot을 찍을 때 금형을 밀어주는 힘	int64
Cycle_Time	제품이 나오기까지 걸리는 총 시간	float64
Pressure_Rise_Time	주조 압력의 상승 시간	float64
Casting_Pressure	Shot을 찍을 때의 압력	int64
Spray_Time	스프레이 작업 시간	float64
Spray_1_Time		float64
Spray_2_Time		float64
Melting_Furnace_Temp	알루미늄을 녹여 일정 온도를 유지시켜주는 장치	float64
Air_Pressure	에어압력 측정값	float64
Air_Pressure_Min	에어압력 최소값	int64
Air_Pressure_Max	에어압력 최대값	int64
Coolant_Temp	냉각수 온도 측정값	float64
Coolant_Temp_Min	냉각수 온도 최소값	int64
Coolant_Temp_Max	냉각수 온도 최대값	int64
Coolant_Pressure	냉각수 압력 측정값	float64

Attributes name	Description	Data Type
Factory_Temp	공장 온도 측정값	float64
Factory_Temp_Min	공장 온도 최소값	float64
Factory_Temp_Max	공장 온도 최대값	float64
Factory_Humidity	공장 습도 측정값	float64
Factory_Humidity_Min	공장 온도 최소값	float64
Factory_Humidity_Max	공장 온도 최대값	float64
Product_Type	제품 유형 (Type1, Type2)	int64
Short_Shot_1, 2	미성형1, 2 (Cavity 1, 2의 불량 유형)	int64
Bubble_1, 2	기포1, 2 (Cavity 1, 2의 불량 유형)	int64
Exfoliation_1, 2	박리1, 2 (Cavity 1, 2의 불량 유형)	int64
Blow_Hole_1, 2	기공1, 2 (Cavity 1, 2의 불량 유형)	int64
Stain_1, 2	얼룩1, 2 (Cavity 1, 2의 불량 유형)	int64
Dent_1, 2	찍힘1, 2 (Cavity 1, 2의 불량 유형)	int64
Deformation_1, 2	평탄1, 2 (Cavity 1, 2의 불량 유형)	int64
Contamination_1, 2	오염1, 2 (Cavity 1, 2의 불량 유형)	int64
Impurity_1, 2	이물1, 2 (Cavity 1, 2의 불량 유형)	int64
Crack_1, 2	뜯김1, 2 (Cavity 1, 2의 불량 유형)	int64
Scratch_1, 2	S/C1, 2 (Cavity 1, 2의 불량 유형)	int64
Buring_Mark_1, 2	소착1, 2 (Cavity 1, 2의 불량 유형)	int64
Inclusions_1, 2	개재물1, 2 (Cavity 1, 2의 불량 유형)	int64
id	설비 id	int64

[표 2] 주요 변수 정의

- 제조AI데이터셋에 포함된 공정 변수는 제품을 생성하는 한 shot 별로 동시에 제공되는 설비 데이터이고, 센서 데이터는 별도로 설치된 센서로부터 수집되는 에어압력, 냉각수, 공장의 온/습도에 대한 데이터이다.
- ‘Product_Type’ 데이터는 제품 종류에 따라 1과 2로 정의하여 제품 유형을 구분한 항목이다.
- 제품 불량의 유형은 다이캐스팅 주조 설비에서 한 shot 별로 2개의 Cavity에서 생산되는 주조품에 대한 것으로서, 1은 Cavity1, 2는 Cavity2를 의미한다. 예를 들어, 미성형1은 Cavity1에서 발생하는 미성형 불량을 의미한다. 또한, 불량 발생 여부는 0과 1로 표현한다. 0이면 정상, 1이면 불량이다. 예를 들어, Short_Shot_1에 1의 값이 존재하면 Cavity1에서 미성형 불량이 발생했다는 의미이다.



• 독립변수 / 종속변수 정의 :

- 독립변수란, 다른 변수에 영향을 받지 않는 변수로, 입력값이나 원인을 나타낸다.
- 종속변수란, 독립변수의 변화에 따라 어떻게 변화하는지를 알고 싶은 변수로, 결과물이나 효과를 나타낸다.

공정 변수 조건		내용
독립변수	속도 관련	Velocity_1, Velocity_2, Velocity_3, High_Velocity
	압력 관련	Cylinder_Pressure, Casting_Pressure, Air_Pressure, Air_Pressure_Min, Air_Pressure_Max, Coolant_Pressure
	시간 관련	Rapid_Rise_Time, Cycle_Time, Pressure_Rise_Time, Spray_Time, Spray_1_Time, Spray_2_Time
	힘 관련	Clamping_Force
	온도 관련	Melting_Furnace_Temp, Coolant_Temp, Coolant_Temp_Min, Coolant_Temp_Max, Factory_Temp, Factory_Temp_Min, Factory_Temp_Max
	습도 관련	Factory_Humidity, Factory_Humidity_Min, Factory_Humidity_Max
	길이 관련	Biscuit_Thickness
종속 변수	불량 관련	Short_Shot, Bubble, Exfoliation, Blow_Hole, Stain, Dent, Deformation, Contamination, Impurity, Crack, Scratch, Buring_Mark, Inclusions

[표 3] 독립/종속 변수 정의

• 주요 변수 기술 통계 :

구분	개수	평균	표준편차	최소값	중앙값	최대값
Velocity_1	7535	0.15	0.01	0.13	0.14	0.18
Velocity_2	7535	0.17	0.00	0.16	0.17	0.21
Velocity_3	7535	0.19	0.01	0.17	0.19	0.23
High_Velocity	7535	2.32	0.22	0.00	2.16	2.74
Cylinder_Pressure	7535	239.66	23.31	107.00	239.00	266.00
Rapid_Rise_Time	7535	0.01	0.00	0.00	0.01	0.02
Biscuit_Thickness	7535	14.31	3.29	0.00	13.00	24.00
Clamping_Force	7535	306.43	57.27	238.00	258.00	388.00
Cycle_Time	7535	27.74	8.72	20.20	22.60	218.60
Pressure_Rise_Time	7535	0.04	0.00	0.00	0.04	0.05
Casting_Pressure	7535	856.94	234.82	516.00	1037.00	1164.00
Spray_Time	7535	9.82	1.84	7.00	9.70	13.10
Spray_1_Time	7535	1.41	0.56	0.70	1.20	2.50
Spray_2_Time	7535	1.40	0.72	0.70	0.80	3.00

구분	개수	평균	표준편차	최소값	중앙값	최대값
Melting_Furnace_Temp	7535	680.65	25.29	635.30	680.30	730.00
Air_Pressure	7535	6.11	0.65	4.60	6.20	7.10
Air_Pressure_Min	7535	3.00	0.00	3.00	3.00	3.00
Air_Pressure_Max	7535	9.00	0.00	9.00	9.00	9.00
Coolant_Temp	7535	26.83	0.53	25.90	26.80	28.10
Coolant_Temp_Min	7535	10.00	0.00	10.00	10.00	10.00
Coolant_Temp_Max	7535	50.00	0.00	50.00	50.00	50.00
Coolant_Pressure	7535	2.70	0.05	2.58	2.72	2.79
Factory_Temp	7535	32.83	1.67	27.40	32.10	37.00
Factory_Temp_Min	7535	18.00	0.00	18.00	18.00	18.00
Factory_Temp_Max	7535	22.00	0.00	22.00	22.00	22.00
Factory_Humidity	7535	61.67	7.03	45.50	63.00	72.30
Factory_Humidity_Min	7535	18.00	0.00	18.00	18.00	18.00
Factory_Humidity_Max	7535	22.00	0.00	22.00	22.00	22.00

[표 4] 주요 변수 기술 통계

- 본 데이터셋에 포함된 연속형 수치 데이터에 대하여 기초적인 기술 통계를 확인한다. ‘Cycle_Time’의 경우, 두 개 유형의 제품 데이터가 통합되었기 때문에 이상치 제거 시에 데이터에 대한 추가적인 확인이 필요하다.

3) 데이터 (품질) 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터는 의미 없는 값이거나, 누락 및 오타가 발생하여 품질이 좋지 못하다. 품질이 낮은 데이터를 분석에 이용하면 좋은 결과를 얻기 힘들다.
- 그러므로 데이터 전처리는 데이터의 분석에 있어서 필수적인 단계이다.
- 데이터 품질 전처리를 위해 아래 6가지의 데이터 품질지수를 파악하고, 데이터의 품질 지수를 향상시킨다.

• 데이터 품질 지수

- 완전성(Completeness) : 필수항목에 누락이 없어야 한다.
- 유일성(Uniqueness) : 데이터 항목은 유일해야 하며 중복되어서는 안된다.

- 유효성(Validity) : 데이터 항목은 정해진 데이터 유효범위 및 도메인을 충족해야 한다.
- 일관성(Consistency) : 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
- 정확성(Accuracy) : 실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
- 무결성(Integrity) : 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터 유일성, 유효성, 일관성이 보호되어야 한다.

• 데이터 품질 지수 : 세부 설명

- 완전성 품질 지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① 결측치 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 결측치 값의 비율을 확인하여 삭제한다.
- ② 데이터의 결측치를 확인하기 위해 'isnull()' 함수를 사용한 뒤 'sum()' 함수를 이용하여 총 결측치 개수를 구한다.
- ③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

- 유일성 품질 지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① 데이터에서 유일한 값을 갖는 열(column)을 찾는다. (EX) 시계열 데이터 : 시간
- ② 유일한 값을 갖는 열(column)에 대해 'groupby()' 함수를 이용하여, 각 열(column)에서의 유일한 데이터 값을 확인하고, 'size()' 함수를 이용하여 그 개수를 구한다.
- ③ 앞서 구한 유일한 데이터의 개수로부터 위 식에 따라 유일성 품질 지수를 구한다.

- 유효성 품질 지수

$$\text{유효성 품질 지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① 'max()', 'min()' 함수를 이용하여 데이터가 유효범위를 벗어나지 않는지를 확인한다.
- ② Pandas의 '*.info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다. 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 유효성을 만족한 데이터의 개수를 구해 위 식에 따라 유효성 품질 지수를 구한다.
- ③ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 유효성을 만족한 데이터의 개수를 구해 위 식에 따라 유효성 품질 지수를 구한다.



- 일관성 품질 지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 테이블 간 종속관계를 확인한다.
- ② Pandas의 '*.info()' 함수로 데이터의 타입이 정해진 형식에 맞는지 확인한다.
- ③ 그 외에 수집된 날짜 안에 들어가 있는지 등을 검증하여 일관성을 만족한 데이터의 개수를 구해 위 식에 따라 일관성 품질 지수를 구한다.

- 정확성 품질 지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터의 '평균값'이 수집된 데이터에 한하여, 다음과 같이 정확성 품질 지수를 확인할 수 있다.
- ② 누락된 데이터가 있는지 확인하기 위해, '*.mean()' 함수를 이용하여 데이터들의 평균을 구한다.
- ③ '*.mean()' 함수로 구한 데이터의 평균에서 데이터의 '평균값'과의 차이를 구한다.
- ④ 이 차이가 0이 아니면 데이터를 위배하는 데이터가 존재하게 된다.

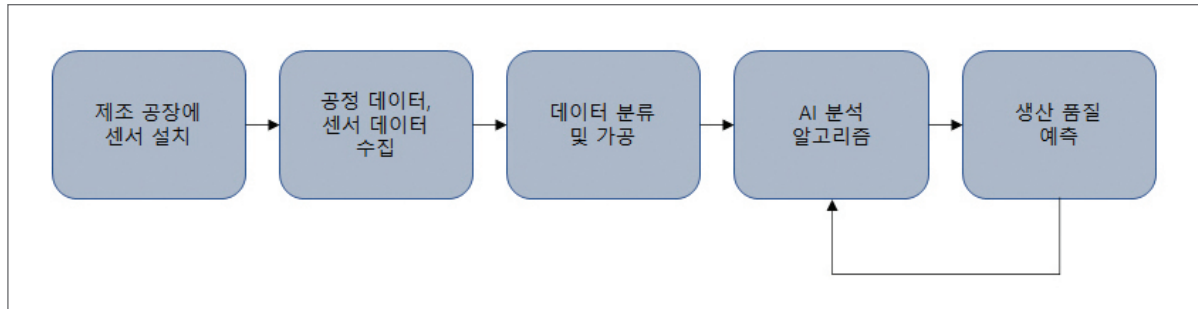
- 무결성 품질 지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100%가 아닌 지수}}{3} \right) \right\} \times 100$$

- ① 무결성 품질지수는 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지표이다. 일반적으로 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질지수로 정의한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 16] 데이터 흐름 및 인공지능 모델 적용 흐름도

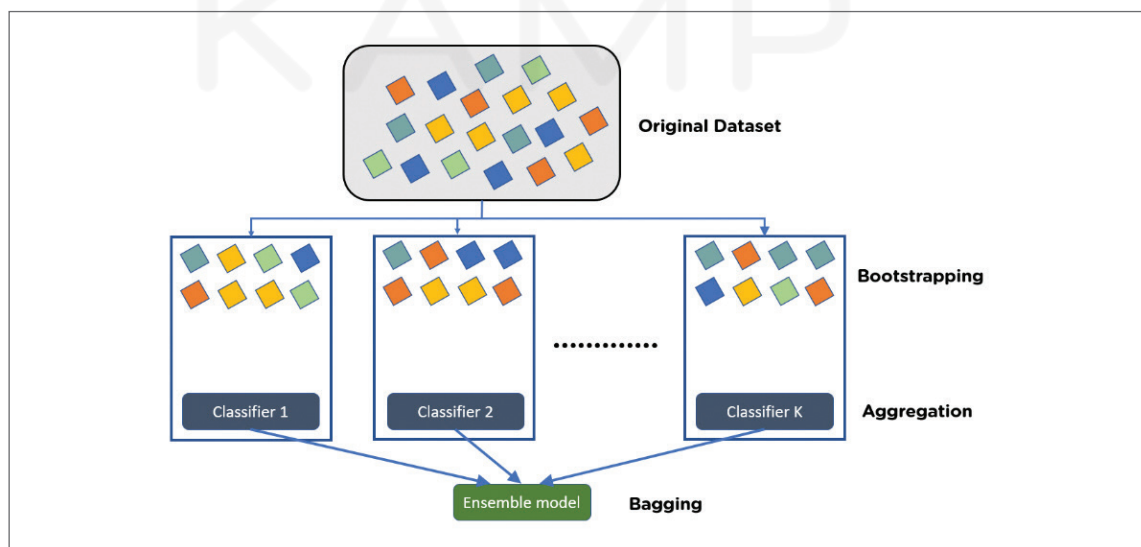
2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유

- 다이캐스팅 주조 품질 데이터 분석 방법 : XGBoost 앙상블 모델
- 앙상블(Ensemble) 기법은 여러 개의 모델을 사용해서 각각의 예측 결과를 만들고 그 예측 결과를 기반으로 최종의 예측 결과를 결정하는 방법으로 일반적으로는 배깅(Bagging)과 부스팅(Boosting)이 있다. 부스팅은 순차적으로 약한 학습자를 추가 결합하여 하나의 강한 모델을 만드는 방법으로, 배깅과 마찬가지로 복원 추출을 사용하는데 샘플에 가중치를 부여하는 차이가 있다. 본 분석에서 적용한 XGBoost는 최근 성능이 검증되어 많이 적용하고 있는 부스팅 기법으로 Gradient를 이용하여 부스팅하는 Gradient Boosting 모델링을 적용하고 있다.

• 적용하고자하는 AI 분석 방법론(알고리즘)의 구체적 소개

- XGBoost 앙상블 모델 :

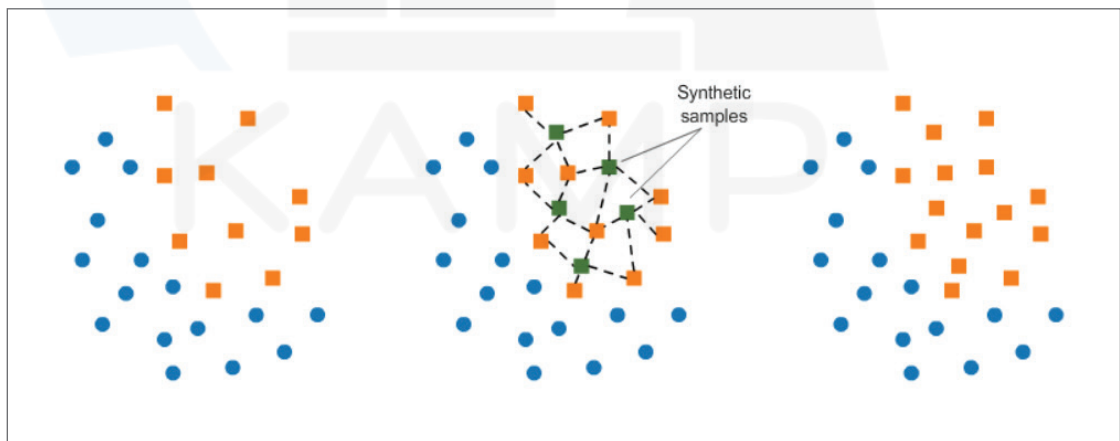


[그림 17] 앙상블 모델 동작 방식

- XGBoost는 트리 기반의 앙상블 모형으로 결정트리를 차례대로 학습하며 개별 트리는 선행하는 트리에 존재하는 오류를 개선해 나가면서 생성된다. XGBoost는 기존의 Gradient Boosting 방법에 Approximation 방법을 적용하여 Parallelize를 가능하게 한 모델로 Chen and Guestrin에 의해 제안되었다.
- XGBoost 이전의 결정트리 모델은 노드를 분할하는 지점을 탐색할 때 모든 경우의 수를 고려함으로써 최적해를 찾을 수 있다. 정확도가 보장되었지만 학습이 느리고 모든 데이터가 메모리에 한번에 할당될 수 없다는 단점이 지적되었다. 이를 보완하기 위해 XGBoost는 Approximation Algorithm으로 최적해의 근사를 통한 병렬처리 방법으로 수행 속도를 높였다.
- Approximation Algorithm은 변수에 대한 데이터의 분포를 고려하여 데이터를 퍼센타일로 나눈 버킷 안에서의 split만 계산하여 효율성을 높이는 방법이다. Column Sampling은 랜덤포레스트에서 차용한 방법으로 샘플링된 변수만을 사용해 학습을 지도하는 방법이다. 이를 통해 XGBoost는 과대적합을 방지하고 학습시간을 단축할 수 있다. 이에 더불어 XGBoost는 하드웨어적인 최적화로 빠른 학습과 높은 정확도를 보여준다. XGBoost 모델은 이벤트 분류, 행동 예측, 동작 감지, 위험 예측 등 다양한 분야에서 활용될 수 있어 확장성을 갖춘 모델로 평가받는다.

- 오버 샘플링 (SMOTE) :

- SMOTE는 Synthetic Minority Over-sampling Technique의 약자로, 대표적인 오버 샘플링 기법 중 하나이다. 이는 낮은 비율로 존재하는 클래스의 데이터를 최근접 이웃 [k-NN 알고리즘] 알고리즘을 활용하여 새롭게 생성하는 방법이다.
- 오버 샘플링 기법 중 단순 무작위 추출을 통해 데이터의 수를 늘리는 방법도 존재하는데, 데이터를 단순히 복사하기 때문에 과적합 문제가 발생하기도 한다. 이에 반해 SMOTE는 알고리즘을 기반으로 데이터를 생성하므로, 과적합 발생 가능성이 단순 무작위 방법보다 적다.



[그림 18] 오버 샘플링(SMOTE) 동작 방식

- SMOTE의 동작 방식을 간단히 설명하면, 임의의 소수 클래스 데이터 사이에 새로운 데이터를 생성하는 방법이라고 할 수 있다. SMOTE는 데이터 생성을 위해서 원본 데이터에 k-NN 알고리즘을 활용하여 같은 클래스의 데이터를 임의로 증식한다. 이 방식을 차례대로 정리하면 다음과 같다.



- ① 소수 클래스의 데이터 중 특정 벡터(샘플)와 가장 가까운 k개의 이웃 벡터를 선정
- ② 기준 벡터와 선정한 벡터 사이를 선분으로 이음
- ③ 선분 위의 임의의 점이 새로운 벡터(혹은 이 중 임의의 하나)

- Feature Importance :

- Tree 기반 머신러닝 모델은 모델 생성 과정에서 각 Node별 Entropy가 발생(계산)한다. Entropy는 '불순도'를 표현하며, 각 Node가 연장되며 가지치기(의사결정)해나가는 과정에서 제대로 가지치기를 하고 있다면 '불순도'가 낮아지고 그 값(Entropy)은 0에 가까워진다. Gradient Boosting 계열의 트리 모델은 이 Entropy 값을 낮추는 방향으로 노드를 이어나간다.
- 만약 제대로 가지치기를 하지 못해 정확히 A와 B로 나누지 못했다면, 즉 A로 분류한 공간에 B가 많이 남아 있을수록 그 값(Entropy)는 점점 올라간다.
- Tree 모델에서 자체 함수로 제공하는 Feature importance는 이렇게 Entropy가 0으로 잘 수렴하고 있는지를 확인하는데, 정보이론 관점에서 보면 정보를 얼마나 잘 획득하고 있는지 확인하는 과정과 같다. 다시 말하면 Entropy값이 상하 노드에서 얼마나 차이가 나는가? 즉 제대로 된 정보를 얼마나 취득해가고 있는가(정보이득)를 체크해서 feature별 유의미성을 판단한다.
- Feature importance는 Tree 기반 모델 자체에서 함수로 제공하고 있으며 plot_importance API를 통해 별도로 호출할 수도 있다. 주로 gradient boosting model 계열에서 활용된다.

• AI 분석 방법론(알고리즘) 구축 절차

- 변수 분할

- 데이터의 정규화를 진행하고 학습 데이터와 테스트 데이터로 분할한다. 여기서 학습 데이터의 정상/비정상 레이블에 따른 불균형을 해결하기 위해 오버 샘플링을 실시한다. 최적의 결과값 내기 위하여 파라미터 최적화를 진행한다. 이렇게 나눈 데이터를 일부는 학습 데이터셋으로, 나머지는 테스트 데이터셋으로 나누어 분석을 진행한다.

- 오버 샘플링 (SMOTE)

- 소수의 클래스를 보완하기 위해 오버 샘플링을 실시한다. 소수의 클래스 주변의 값들을 임의의 값으로 채워준다. 모든 클래스의 개수를 동일하게 맞춰 불균형을 해소한다.

- XGBoost

- Gradient를 이용하여 부스팅하는 경사 하강법(Gradient Boosting) 모델링을 적용하여 진행한다. 트리의 손실을 최소화하기 위해 새로운 약한 학습자를 한 번에 한 개씩 추가하며 기존 트리는 변경하지 않는다. 손실이 허용 가능한 수준에 도달하거나 검증 데이터셋에서 더이상 개선하지 않으면 중단한다.

2.3 분석 체험

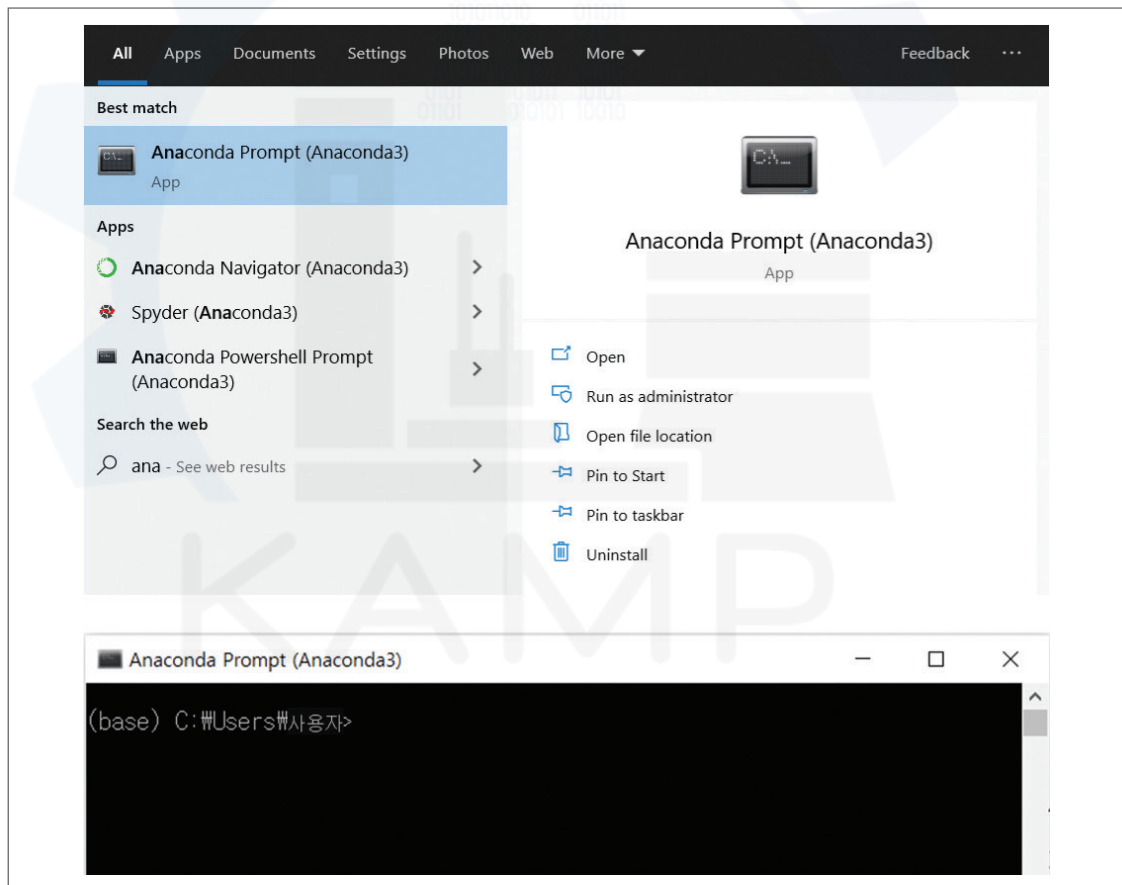
1) 필요 SW, 패키지 설치 방법 및 절차 가이드

▶ SW 설치는 ‘부록_분석환경 구축을 위한 설치 가이드’ 참고

▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주 피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

① 아나콘다 프롬프트 실행





② 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
conda create -n KAMP python=3.9.7
```

```
Anaconda Prompt (anaconda3) - conda create -n KAMP python=3.9.7

(base) C:\Users\USER>conda create -n KAMP python=3.9.7
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
current version: 4.11.0
latest version: 22.9.0

Please update conda by running

$ conda update -n base -c defaults conda
```

...

```
Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.
```

...

```
done
#
# To activate this environment, use
#
#     $ conda activate KAMP
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#

(base) C:\Users\사용자>
```

③ 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
conda activate KAMP
```

```
(base) C:\Users\사용자>conda activate KAMP
(KAMP) C:\Users\사용자>
```

④ 가상 환경 종료

분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\사용자>conda deactivate
(base) C:\Users\사용자>
```



⑤ 주피터 노트북에서 가상 환경 실행

주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 비활성화(종료) 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

```
pip install ipykernel
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
python -m ipykernel install --user --name KAMP --display-name "KAMP"
# jupyter notebook 실행
jupyter notebook
```



▶ 패키지 설치 가이드

- 패키지 설치 전 파이썬 관련 용어 정리

모듈 (module)	- 특정 함수, 변수, 클래스 등이 구현되어 있는 파이썬 파일(.py)을 칭한다. 대표적으로 아래 모듈들이 존재한다. - 예) numpy: 수치해석 모듈, pandas: 데이터 분석 모듈
패키지 (package)	- 여러 모듈들의 모음 - 패키지 안에 여러 가지 폴더가 존재할 수 있다.
라이브러리 (library)	- 여러 패키지의 모음 - 파이썬을 설치할 때, 기본적으로 설치되는 라이브러리를 표준라이브러리라고 한다. 파이썬 공식이 아닌 외부에서 개발한 모듈과 패키지를 묶어 외부 라이브러리라고 한다.
프레임워크 (framework)	- 라이브러리 모음 - 프레임워크가 곧 프로그램의 구성요소이다.

- 패키지 설치 가이드

① 주피터 노트북 내에서 패키지 및 모듈 설치하기

```
!pip install pandas
!pip install numpy
!pip install scikit-learn
!pip install matplotlib
!pip install seaborn
!pip install -U imbalanced-learn
```

* 이번 분석을 위한 필요 패키지 및 모듈은 pandas, numpy, scikit-learn, matplotlib, seaborn, imbalanced-learn이기 때문에 위와 같이 시작 전에 설치를 하면 된다.



- 모듈 설치 확인하기

```
!pip list
```

- 패키지 및 모듈 임포트 하기

① import

import를 사용하여 해당 모듈 전체를 임포트한다.

```
import pandas
pandas.read_csv('DieCasting_Quality_Raw_Data.csv')
```

pandas를 import 하고 '.'을 사용하여 pandas의 'read_csv' 함수를 이용하여 'DieCasting_Quality_Raw_Data.csv' 파일을 불러온다.

② from, import

해당 모듈에서 특정한 타입만 임포트한다.

예) pandas에서 'read_csv'만 임포트

```
from pandas import read_csv
read_csv('DieCasting_Quality_Raw_Data.csv')
```

from, import를 사용해서 필요한 함수만 import로 사용할 수 있다.

③ * import

해당 모듈 내에 정의된 모든 것을 임포트하나 다른 모듈들과 섞일 수 있으므로 일반적으로 사용이 권장되지 않는다.

```
from pandas import *
```

④ as

모듈 임포트할 때, 별명(alias)를 지정할 때 사용한다.

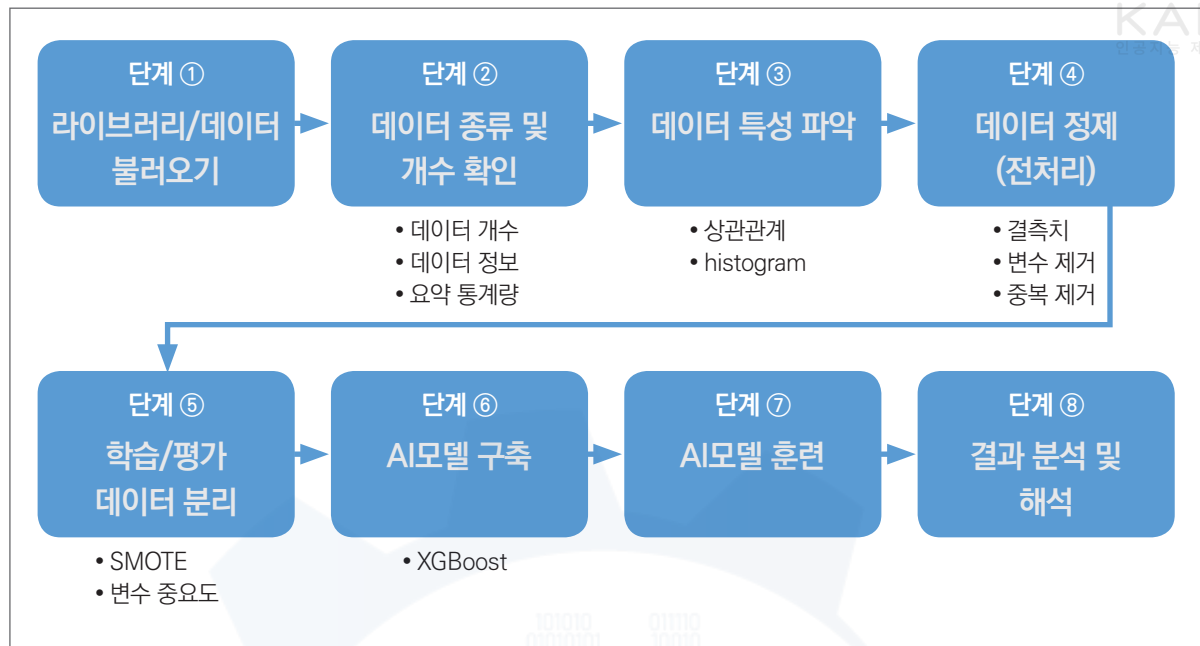
```
import pandas as pd
pd.read_csv('DieCasting_Quality_Raw_Data.csv')
```

'pandas'를 'pd'로 별명 지정 후에는 'pd'로 불러올 수 있다.

* 한번 설치한 모듈 및 패키지는 영구적이며 필요시 버전 업그레이드가 필요하다. 또한, 분석, 시각화 등 상황에 따라 필요한 패키지를 import 한다.



2) 분석 단계별 프로세스 - Flow Chart



[그림 19] 분석 Flow Chart

[단계 ①] 라이브러리/데이터 불러오기

①-1. 라이브러리 불러오기

- 분석에 앞서 우선 필요한 Package를 불러와야 한다(import). Package는 한마디로, 사용하고자 하는 계산식 혹은 알고리즘 관련 함수를 모아놓은 모듈들의 모음이라고 할 수 있다. 다양한 머신러닝/딥러닝 알고리즘을 단 몇 줄만으로 실행가능 하도록 한다. 다음과 같은 방법으로 Package를 불러온다.

```

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, roc_auc_score, recall_score, accuracy_score
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from imblearn.over_sampling import *
import matplotlib.pyplot as plt
import seaborn as sns
  
```

[코드 1] 라이브러리 불러오기

- Package와 모듈을 불러오는 코드는 'from 패키지명 import 모듈명'을 기본으로 한다. 모듈을 import할 때 'as'를 사용하여 별명(alias)을 지정하면 더욱 간편하게 불러올 수 있다.



①-2. 데이터 불러오기

```
df=pd.read_csv('DieCasting_Quality_Raw_Data.csv',header=[0,1])
```

[코드 2] 데이터 불러오기

- `pd.read_csv()` (“데이터파일_경로명”)을 통해 csv 형태의 데이터를 판다스 (pandas)의 DataFrame으로 읽어 들인다. 이 예제에서는 csv 형태이기 때문에 `read_csv()`를 사용하였지만 excel, hdf5 형식인 데이터도 불러올 수 있다. 이런 형식일 경우에는 `read_excel()`, `read_hdf5()`를 사용하면 된다.
- 해당 데이터셋은 데이터의 종류를 확인하기 위해 멀티칼럼으로 이루어진 데이터프레임이다. 57개의 변수를 3개의 범주(Process, Sensor, Defects)로 나눠 어떤 종류의 변수에 해당하는지 이해하기 쉽도록 하였다. 파일을 불러올 때 매개변수 `header`를 사용하여 0행과 1행을 머릿글 행으로 하는 멀티칼럼을 만든다.

[단계 ②] 데이터 종류 및 개수 확인

②-1. 데이터 확인

```
df.head()
```

Process										
	id	Product_Type	Shot	Velocity_1	Velocity_2	Velocity_3	High_Velocity	Cylinder_Pressure	Rapid_Rise_Time	Biscuit_Thickness
0	1	1	1	0.144	0.170	0.188	2.134	214	0.008	10
1	1002	1	2	0.144	0.170	0.182	2.124	217	0.008	11
2	2003	1	3	0.144	0.170	0.182	2.116	214	0.008	11
3	3004	1	4	0.144	0.170	0.182	2.137	217	0.008	11
4	4005	1	5	0.144	0.172	0.176	2.111	217	0.008	12

5 rows × 57 columns

[코드 3] 데이터 확인

- 불러온 데이터프레임 `df`에 `head()` 함수를 입력하면 데이터셋 일부를 확인할 수 있다. 데이터의 양이 많기 때문에 상위 5개 행(Index 0~4)을 출력하여 확인한다. 만약 데이터셋의 마지막 행의 일부를 확인하고 싶다면, `tail(default=5)` 함수를 사용할 수 있다.



②-2. 칼럼명 확인

df.columns

```
MultiIndex([(Process', 'id'),
            (Process', 'Product_Type'),
            (Process', 'Shot'),
            (Process', 'Velocity_1'),
            (Process', 'Velocity_2'),
            (Process', 'Velocity_3'),
            (Process', 'High_Velocity'),
            (Process', 'Cylinder_Pressure'),
            (Process', 'Rapid_Rise_Time'),
            (Process', 'Biscuit_Thickness'),
            (Process', 'Clamping_Force'),
            (Process', 'Cycle_Time'),
            (Process', 'Pressure_Rise_Time'),
            (Process', 'Casting_Pressure'),
            (Process', 'Spray_Time'),
            (Process', 'Spray_1_Time'),
            (Process', 'Spray_2_Time'),
            (Sensor', 'Melting_Furnace_Temp'),
            (Sensor', 'Air_Pressure'),
            (Sensor', 'Air_Pressure_Min'),
            (Sensor', 'Air_Pressure_Max'),
            (Sensor', 'Coolant_Temp'),
            (Sensor', 'Coolant_Temp_Min'),
            (Sensor', 'Coolant_Temp_Max'),
            (Sensor', 'Coolant_Pressure'),
            (Sensor', 'Factory_Temp'),
            (Sensor', 'Factory_Temp_Min'),
            (Sensor', 'Factory_Temp_Max'),
            (Sensor', 'Factory_Humidity'),
            (Sensor', 'Factory_Humidity_Min'),
            (Sensor', 'Factory_Humidity_Max'),
            (Defects', 'Short_Shot_1'),
            (Defects', 'Bubble_1'),
            (Defects', 'Exfoliation_1'),
            (Defects', 'Blow_Hole_1'),
            (Defects', 'Stain_1'),
            (Defects', 'Dent_1'),
            (Defects', 'Deformation_1'),
            (Defects', 'Contamination_1'),
            (Defects', 'Impurity_1'),
            (Defects', 'Crack_1'),
            (Defects', 'Scratch_1'),
            (Defects', 'Buring_Mark_1'),
            (Defects', 'Inclusions_1'),
            (Defects', 'Short_Shot_2'),
            (Defects', 'Bubble_2'),
            (Defects', 'Exfoliation_2'),
            (Defects', 'Blow_Hole_2'),
            (Defects', 'Stain_2'),
            (Defects', 'Dent_2'),
            (Defects', 'Deformation_2'),
            (Defects', 'Contamination_2'),
            (Defects', 'Impurity_2'),
            (Defects', 'Crack_2'),
            (Defects', 'Scratch_2'),
            (Defects', 'Buring_Mark_2'),
            (Defects', 'Inclusions_2')])
```

[코드 4] 칼럼명 확인 코드

- 데이터프레임에 들어 있는 칼럼명을 확인하는 코드로 총 57개의 멀티 칼럼을 확인할 수 있다.

②-3. 요약 통계량 확인

df.describe()

Process									
	id	Product_Type	Shot	Velocity_1	Velocity_2	Velocity_3	High_Velocity	Cylinder_Pressure	Rapid_Rise_Time
count	7.535000e+03	7535.000000	7535.000000	7535.000000	7535.000000	7535.000000	7535.000000	7535.000000	7535.000000
mean	3.767454e+06	1.441672	453.798938	0.148219	0.168801	0.191193	2.319210	239.655607	0.009596
std	2.175264e+06	0.496619	319.451698	0.007134	0.004720	0.011563	0.222041	23.305451	0.002148
min	1.000000e+00	1.000000	0.000000	0.134000	0.158000	0.172000	0.000000	107.000000	0.000000
25%	1.883893e+06	1.000000	195.000000	0.142000	0.166000	0.181000	2.134000	217.000000	0.008000
50%	3.767193e+06	1.000000	401.000000	0.144000	0.168000	0.188000	2.161000	239.000000	0.009000
75%	5.650924e+06	2.000000	645.000000	0.156000	0.170000	0.202000	2.523000	265.000000	0.012000
max	7.534661e+06	2.000000	1296.000000	0.180000	0.212000	0.234000	2.744000	266.000000	0.021000

8 rows × 57 columns

[코드 5] 요약 통계량 확인 코드

- describe() 명령을 통해 최대, 최소, 평균 등과 같은 다양한 기술 통계를 확인할 수 있다.
 - count : null이 아닌 행의 개수 (유효한 값)
 - mean : 평균
 - std : 표준편차
 - min : 최소값
 - 25% : 1/4 분위
 - 50% : 중앙값 (4/2 분위)
 - 75% : 3/4 분위
 - max : 최대값

②-4. 데이터 정보 확인

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7535 entries, 0 to 7534
Data columns (total 57 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   (Process, id)                             7535 non-null   int64
1   (Process, Product_Type)                   7535 non-null   int64
2   (Process, Shot)                           7535 non-null   int64
3   (Process, Velocity_1)                     7535 non-null   float64
4   (Process, Velocity_2)                     7535 non-null   float64
5   (Process, Velocity_3)                     7535 non-null   float64
6   (Process, High_Velocity)                   7535 non-null   float64
7   (Process, Cylinder_Pressure)               7535 non-null   int64
8   (Process, Rapid_Rise_Time)                 7535 non-null   float64
9   (Process, Biscuit_Thickness )              7535 non-null   int64
10  (Process, Clamping_Force )                 7535 non-null   int64
11  (Process, Cycle_Time)                     7535 non-null   float64
12  (Process, Pressure_Rise_Time)              7535 non-null   float64
13  (Process, Casting_Pressure)                7535 non-null   int64
14  (Process, Spray_Time)                     7535 non-null   float64
15  (Process, Spray_1_Time)                    7535 non-null   float64
16  (Process, Spray_2_Time)                    7535 non-null   float64
17  (Sensor, Melting_Furnace_Temp)             7535 non-null   float64
18  (Sensor, Air_Pressure)                     7535 non-null   float64
19  (Sensor, Air_Pressure_Min)                 7535 non-null   int64
20  (Sensor, Air_Pressure_Max)                 7535 non-null   int64
21  (Sensor, Coolant_Temp)                     7535 non-null   float64
22  (Sensor, Coolant_Temp_Min)                 7535 non-null   int64
23  (Sensor, Coolant_Temp_Max)                 7535 non-null   int64
24  (Sensor, Coolant_Pressure)                 7535 non-null   float64
25  (Sensor, Factory_Temp)                     7445 non-null   float64
26  (Sensor, Factory_Temp_Min)                 7445 non-null   float64
27  (Sensor, Factory_Temp_Max)                 7445 non-null   float64
28  (Sensor, Factory_Humidity)                 7445 non-null   float64
29  (Sensor, Factory_Humidity_Min)             7445 non-null   float64
30  (Sensor, Factory_Humidity_Max)             7445 non-null   float64
31  (Defects, Short_Shot_1)                   7535 non-null   int64
32  (Defects, Bubble_1)                       7535 non-null   int64
33  (Defects, Exfoliation_1)                  7535 non-null   int64
34  (Defects, Blow_Hole_1)                    7535 non-null   int64
35  (Defects, Stain_1)                        7535 non-null   int64
36  (Defects, Dent_1)                         7535 non-null   int64
37  (Defects, Deformation_1)                  7535 non-null   int64
38  (Defects, Contamination_1)                7535 non-null   int64
39  (Defects, Impurity_1)                     7535 non-null   int64
40  (Defects, Crack_1)                        7535 non-null   int64
41  (Defects, Scratch_1)                      7535 non-null   int64
42  (Defects, Buring_Mark_1)                   7535 non-null   int64
43  (Defects, Inclusions_1)                    7535 non-null   int64
44  (Defects, Short_Shot_2)                   7535 non-null   int64
45  (Defects, Bubble_2)                       7535 non-null   int64
46  (Defects, Exfoliation_2)                  7535 non-null   int64
47  (Defects, Blow_Hole_2)                    7535 non-null   int64
48  (Defects, Stain_2)                        7535 non-null   int64
49  (Defects, Dent_2)                         7535 non-null   int64
50  (Defects, Deformation_2)                  7535 non-null   int64
51  (Defects, Contamination_2)                7535 non-null   int64
52  (Defects, Impurity_2)                     7535 non-null   int64
53  (Defects, Crack_2)                        7535 non-null   int64
54  (Defects, Scratch_2)                      7535 non-null   int64
55  (Defects, Buring_Mark_2)                   7535 non-null   int64
56  (Defects, Inclusions_2)                    7535 non-null   int64
dtypes: float64(20), int64(37)
memory usage: 3.3 MB
```

[코드 6] 데이터 정보 확인 코드

- 불러들인 데이터를 구성하고 있는 요소들의 칼럼명, 칼럼별 null이 아닌 데이터 개수, 데이터 타입을 확인할 수 있다. 이 데이터는 float64형이 20개, int64형이 37개다.



②-5. 데이터 크기 확인

```
df.shape
```

```
(7535, 57)
```

[코드 7] 데이터 크기 확인 코드

- 데이터프레임의 크기는 shape 명령을 통해 (행, 열)의 개수를 확인할 수 있다. 이 데이터는 행이 7535개이고, 열이 57개인 데이터임을 보여준다.

②-6. 결측치 확인

```
df.isna().sum()
```

Process	id	0	Defects	Short_Shot_1	0
	Product_Type	0		Bubble_1	0
	Shot	0		Exfoliation_1	0
	Velocity_1	0		Blow_Hole_1	0
	Velocity_2	0		Stain_1	0
	Velocity_3	0		Dent_1	0
	High_Velocity	0		Deformation_1	0
	Cylinder_Pressure	0		Contamination_1	0
	Rapid_Rise_Time	0		Impurity_1	0
	Biscuit_Thickness	0		Crack_1	0
	Clamping_Force	0		Scratch_1	0
	Cycle_Time	0		Buring_Mark_1	0
	Pressure_Rise_Time	0		Inclusions_1	0
	Casting_Pressure	0		Short_Shot_2	0
	Spray_Time	0		Bubble_2	0
	Spray_1_Time	0		Exfoliation_2	0
	Spray_2_Time	0		Blow_Hole_2	0
Sensor	Melting_Furnace_Temp	0		Stain_2	0
	Air_Pressure	0		Dent_2	0
	Air_Pressure_Min	0		Deformation_2	0
	Air_Pressure_Max	0		Contamination_2	0
	Coolant_Temp	0		Impurity_2	0
	Coolant_Temp_Min	0		Crack_2	0
	Coolant_Temp_Max	0		Scratch_2	0
	Coolant_Pressure	0		Buring_Mark_2	0
	Factory_Temp	90		Inclusions_2	0
	Factory_Temp_Min	90			
	Factory_Temp_Max	90			
	Factory_Humidity	90			
	Factory_Humidity_Min	90			
	Factory_Humidity_Max	90			

dtype: int64

[코드 8] 결측치 확인 코드

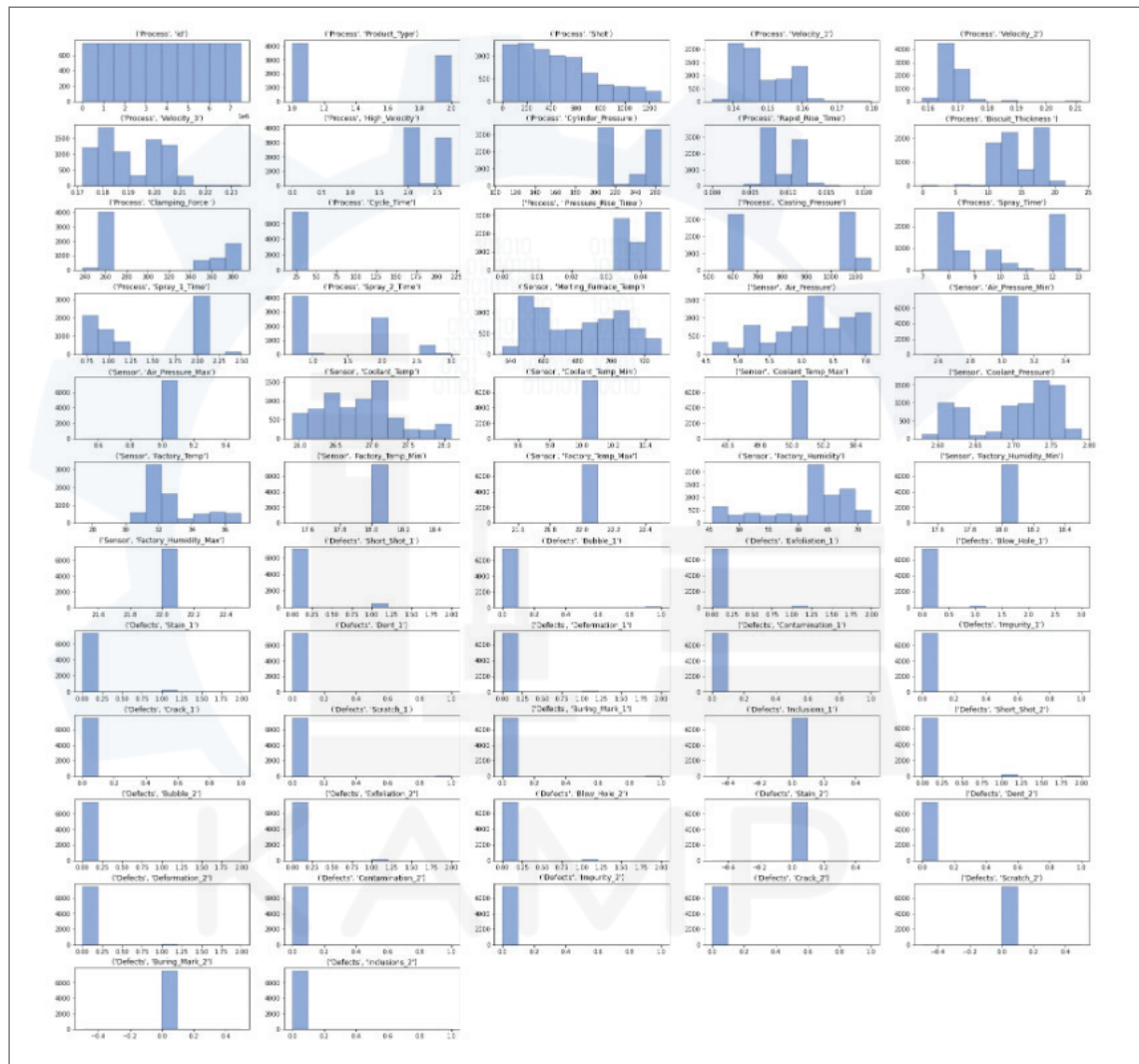
- 각 칼럼에 포함되어 있는 null의 개수를 확인하기 위해 사용되는 코드이다. 57개의 칼럼 중에 6개의 칼럼에서 null값이 존재하는 것을 확인할 수 있다. 전처리 단계에서 이러한 값들을 제거할 예정이다.

[단계 ③] 데이터 특성 파악

③-1. Histogram을 통한 변수별 데이터 파악

```
plt.figure(figsize=(30,30))
plt.subplots_adjust(hspace=0.38) #그림 간격 조절

# 각 변수의 막대그래프 개수
for index,value in enumerate(df):
    sub=plt.subplot(12,5,index+1)
    sub.hist(df[value],facecolor=(144/255,171/255,221/255),
            linewidth=.3,edgecolor='black')
    plt.title(value)
```



[코드 9] 데이터 분포 파악 코드

- 히스토그램은 도수분포표를 그래프로 나타낸 것으로서, X축은 특정 변수의 구간이며 Y축은 구간에 해당하는 데이터의 개수이다. 히스토그램을 통한 시각화는 하나의 변수에 대한 데이터의 분포를 어렵잡아 볼 수 있다는 장점이 있다. 히스토그램의 패턴이 단봉, 쌍봉인지 등의 모달리티(modality)와 왼쪽 혹은 오른쪽으로 쏠려있는 정도인 비대칭도 등을 확인할 수 있다.

- 히스토그램을 시각화하기 위하여, 각 변수별로 막대그래프의 개수를 설정해주고, 각 변수마다 해당 막대그래프의 개수가 할당될 수 있도록 index와 value로 설정해준다. matplotlib 패키지의 pyplot(plt로 표현)을 이용하여 히스토그램을 그려준다. subplot() 함수를 이용하여 한번에 모든 변수의 히스토그램 결과를 볼 수 있도록 한다.

③-2. 변수 간의 상관관계 파악

```
plt.subplots(figsize=(30,30))
sns.heatmap(data=df.corr(),linewidth=0.1,annot=True,fmt='.2f',cmap='Blues');
```



[코드 10] 상관관계 파악 코드

- 위 그림은 변수 간의 상관계수를 히트맵을 활용하여 시각화한 결과 화면으로, 각 칸의 값은 해당되는 X축, Y축의 변수 간의 상관계수이며, 우측의 범례처럼 상관계수 값마다 다른 색을 가진다.

- 상관관계 분석을 통해 두 변수 간의 선형적인 관계의 존재 여부를 확인할 수 있다. 분석에 사용된 상관계수는 보편적으로 사용되는 피어슨 상관계수로서 1과 1사이의 값을 가진다. 상관계수의 절대값이 1에 가까울수록 변수 간의 선형관계가 강하다고 볼 수 있으며, 값이 양수일 때는 양적인 선형관계, 값이 음수일 때는 음적인 선형 관계를 가진다. 상관관계 분석은 비선형적인 관계를 확인하기 어렵지만, 변수 간의 선형관계를 정량적으로 확인할 수 있다는 장점이 있다. 상관계수의 절대값이 0.1과 0.3 사이이면 약한 선형관계, 0.3과 0.7 사이이면 뚜렷한 선형관계, 0.7 이상인 경우 강한 선형관계를 나타낸다고 이해할 수 있다.

[단계 ④] 데이터 정제(전처리)

④-1. 결측치 제거 및 확인

```
# 불량 유형
cavity1_out_list=[('Defects','Short_Shot_1'),('Defects','Bubble_1'),('Defects','Exfoliation_1'),('Defects','Blow_Hole_1'),
('Defects','Stain_1'),('Defects','Dent_1'),('Defects','Deformation_1'),('Defects','Contamination_1'),('Defects','Impurity_1'),
('Defects','Crack_1'),('Defects','Scratch_1'),('Defects','Buring_Mark_1'),('Defects','Inclusions_1')]

cavity2_out_list=[('Defects','Short_Shot_2'),('Defects','Bubble_2'),('Defects','Exfoliation_2'),('Defects','Blow_Hole_2'),
('Defects','Stain_2'),('Defects','Dent_2'),('Defects','Deformation_2'),('Defects','Contamination_2'),('Defects','Impurity_2'),
('Defects','Crack_2'),('Defects','Scratch_2'),('Defects','Buring_Mark_2'),('Defects','Inclusions_2')]

# 캐비티 1번, 2번 불량 유형 열의 NAN값 0으로 대체
df[cavity1_out_list]=df[cavity1_out_list].fillna(0)
df[cavity2_out_list]=df[cavity2_out_list].fillna(0)

cavity_all=cavity1_out_list+cavity2_out_list
df[cavity_all].isnull().sum()
```

```
Defects Short_Shot_1 0
Bubble_1 0
Exfoliation_1 0
Blow_Hole_1 0
Stain_1 0
Dent_1 0
Deformation_1 0
Contamination_1 0
Impurity_1 0
Crack_1 0
Scratch_1 0
Buring_Mark_1 0
Inclusions_1 0
Short_Shot_2 0
Bubble_2 0
Exfoliation_2 0
Blow_Hole_2 0
Stain_2 0
Dent_2 0
Deformation_2 0
Contamination_2 0
Impurity_2 0
Crack_2 0
Scratch_2 0
Buring_Mark_2 0
Inclusions_2 0
dtype: int64
```

[코드 11] 결측치 제거 및 확인 코드



- Cavity 1번에서 발생하는 불량 유형을 리스트(cavity1_out_list)로 만들고 Cavity 2번에서 발생하는 불량 유형을 리스트(cavity2_out_list)로 만든다.
- 위에서 만든 불량 유형 리스트에 결측치가 존재하는 경우 0으로 대체한다. 여기서 발생하는 결측치는 불량이 아닌 양품(혹은 품질검사 결과가 존재하지 않음)으로 간주하여 0으로 입력하는 것이다. 이는 불량 예측에서 결과에 중요한 영향을 미치는 소수 클래스인 불량 데이터의 정확성을 유지하며 전체 데이터 양의 충분한 확보를 위한 처리 과정이다.
- cavity1_out_list와 cavity2_out_list를 합쳐 새로운 리스트 cavity_all을 생성한다. 새로운 리스트 cavity_all은 Cavity 1번과 Cavity 2번에서 발생하는 모든 불량 유형이다. cavity_all에 해당하는 값의 결측치 여부를 isnull().sum() 명령을 활용하여 확인한다. 결측치가 존재하지 않는 것을 확인할 수 있다.

④-2. 양품 레이블 추가 생성

```
# 양품이면 1 아니면 0

# 캐비티1번: 양품이면 1 불량이면 0
df.loc[:,("Defects", "Good1")] = np.where(np.sum(df[cavity1_out_list], axis=1) == 0, 1, 0)
# 캐비티2번: 양품이면 1 불량이면 0
df.loc[:,("Defects", "Good2")] = np.where(np.sum(df[cavity2_out_list], axis=1) == 0, 1, 0)

# 불량 유형 담은 리스트에 양품 여부열 추가
cavity1_out_list.append(("Defects", "Good1"))
cavity2_out_list.append(("Defects", "Good2"))

df
```

Defects									
Dent_2	Deformation_2	Contamination_2	Impurity_2	Crack_2	Scratch_2	Buring_Mark_2	Inclusions_2	Good1	Good2
0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	1	1
...	...	...	...	...	...	...	...	...	...
0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	1

[코드 12] 양품 레이블 추가 코드

- 불량 레이블만이 아닌 양품도 불량 유형 중 하나의 레이블로 활용하기 위하여 새로운 칼럼 “Good1”과 “Good2”를 생성한다. 불량인 경우 0을 양품인 경우 1을 삽입한다. 불량을 판단하는 기준은 다음과 같다. shot 별로 불량 유형(cavity1_out_list, cavity2_out_list)에 해당하는 칼럼의 값이 모두 0인 경우만 양품에 해당한다. 한 개라도 1이 존재하면 불량이다. np.sum() 명령을 활용하여 양품 여부를 구한다. 합산한 값이 0이면 양품, 1이상 이면 불량이다.
- 양품 여부를 확인한 결과 위의 데이터프레임과 같이 나타난 것을 확인할 수 있다.

④-3. 불량 유형 레이블 인코딩

불량 발생하는 유형을 표기하기 위한 열 생성

```
df[("Defects", "cavity1out")] = np.argmax(np.array(df[cavity1_out_list]), axis=1)
df[("Defects", "cavity2out")] = np.argmax(np.array(df[cavity2_out_list]), axis=1)
df
```

Defects									
Contamination_2	Impurity_2	Crack_2	Scratch_2	Buring_Mark_2	Inclusions_2	Good1	Good2	cavity1out	cavity2out
0	0	0	0	0	0	0	1	1	13
0	0	0	0	0	0	0	1	1	13
0	0	0	0	0	0	0	1	1	13
0	0	0	0	0	0	0	0	1	2
0	0	0	0	0	0	0	1	1	13
...	...	...	...	...	...	...	...	...	...
0	0	0	0	0	0	0	1	1	13
0	0	0	0	0	0	0	0	1	3
0	0	0	0	0	0	0	0	1	3
0	0	0	0	0	0	0	0	1	3
0	0	0	0	0	0	0	0	1	3

[코드 13] 불량 유형 레이블 추가 코드

- 원-핫 인코딩(One-Hot Encoding) 형식으로 표현된 불량 유형을 레이블 인코딩(Label Encoding) 형식으로 바꾼다.
- 원-핫 인코딩 형식으로 표현된 26개 칼럼을 레이블 인코딩 형식으로 2개의 칼럼에 나타낸다. Cavity 1번에 해당하는 불량 유형 13개(cavity1_out_list)는 칼럼 cavity1out에 표현하고, Cavity 2번에 해당하는 불량 유형 13개(cavity2_out_list)는 칼럼 cavity2out에 표현한다.
- 불량 유형의 번호를 불량 리스트에 해당하는 인덱스로 설정한다.

- numpy의 argmax 명령어를 통해 불량 리스트에 해당하는 변수의 인덱스 값을 반환한다. 다만, 두 개 이상의 불량이 존재하는 경우 가장 큰 값의 인덱스를 가지는 불량을 선택한다. 예를 들어, Bubble변수와 Stain변수가 1을 가지는 경우 더 큰 인덱스인 Stain의 4를 반환한다. 불량 변수의 인덱스는 다음과 같다.

Short_Shot	Bubble	Exfoliation	Blow_Hole	Stain	Dent	Deformation
0	1	2	3	4	5	6

Contamination	Impurity	Crack	Scratch	Burning_Mark	Inclusion	Good
7	8	9	10	11	12	13

[표 5] 불량 변수 인덱스

④-4. 양품, 주요 불량 그리고 기타 불량으로 분류

```
# 새롭게 불량 유형 표기
cavity_list=["Good","Short_Shot","Bubble","Exfoliation","Blow_Hole","Etc"]

# 캐비티1
df[("Defects","cavity1out2")]=np.where(df[("Defects","cavity1out")]==13,0,
np.where(df[("Defects","cavity1out")]==0,1,
np.where(df[("Defects","cavity1out")]==1,2,
np.where(df[("Defects","cavity1out")]==2,3,
np.where(df[("Defects","cavity1out")]==3,4,5))))))

# 캐비티2
df[("Defects","cavity2out2")]=np.where(df[("Defects","cavity2out")]==13,0,
np.where(df[("Defects","cavity2out")]==0,1,
np.where(df[("Defects","cavity2out")]==1,2,
np.where(df[("Defects","cavity2out")]==2,3,
np.where(df[("Defects","cavity2out")]==3,4,5))))))

df
```

Defects									
Crack_2	Scratch_2	Buring_Mark_2	Inclusions_2	Good1	Good2	cavity1out	cavity2out	cavity1out2	cavity2out2
0	0	0	0	1	1	13	13	0	0
0	0	0	0	1	1	13	13	0	0
0	0	0	0	1	1	13	13	0	0
0	0	0	0	0	1	2	13	3	0
0	0	0	0	1	1	13	13	0	0
...	...	...	...	...	...	...	...	...	...
0	0	0	0	1	1	13	13	0	0
0	0	0	0	0	1	3	13	4	0
0	0	0	0	0	1	3	13	4	0
0	0	0	0	0	1	3	13	4	0
0	0	0	0	0	1	3	13	4	0

[코드 14] 양품, 주요 불량, 기타 불량으로 분류하는 코드

- 현재 불량 유형이 너무 많이 존재하고 일부 불량은 거의 발생하지 않기 때문에 불량의 유형을 크게 양품, 주요 불량들, 그리고 기타 불량으로 정리하여 재분류하고자 한다. 주요 불량은 Short_Shot, Bubble, Exfoliation, Blow_Hole에 해당한다. 나머지 불량은 기타로 처리한다.
- 위에서 표로 정리한 인덱스를 새로운 번호로 정리한다. 정리된 번호는 새로운 칼럼 (cavity1out2, cavity2out2)에 삽입하여 정리한다. 새로운 번호는 다음과 같다.

불량 유형	Good	Short_Shot	Bubble	Exfoliation	Blow_Hole	Etc
기존 번호	13	0	1	2	3	4~12
새로운 번호	0	1	2	3	4	5

[표 6] 새로운 불량 변수 인덱스

④-5. 불필요한 열 제거

```
# 불필요한 열 줄이기
df=df.drop(columns=cavity1_out_list)
df=df.drop(columns=cavity2_out_list)

# 불필요한 열 줄이기
df=df.drop(columns=[("Defects","cavity1out"),("Defects","cavity2out")])

# 빈 행 제거 # 7445
df=df.dropna()

# 멀티 칼럼 제거
df.columns=df.columns.droplevel(0)

# 불필요한 열 제거
df=df.drop(["id","Shot"],axis=1)

df.shape
```

```
(7445, 31)
```

[코드 15] 불필요한 열과 행 제거 코드

- 본 분석에 활용하지 않는 열을 제거하고 결측치가 포함된 행을 제거한다.
- 제거된 열은 불량 유형을 나타내는 cavity1_out_list(Short_Shot_1, Bubble_1 등), cavity2_out_list(Short_Shot_2, Bubble_2 등)에 해당하는 칼럼들과 cavity1out, cavity2out이다.
- 멀티칼럼을 제거하고 양품의 특성을 나타내지 못하는 불필요한 칼럼 id와 Shot을 제거한다.
- 그 결과, 최종 31개의 칼럼으로 데이터가 정리된 것을 확인할 수 있다.

④-6. 데이터프레임 재구조화

```
# cavity1, cavity2 행으로 연결
df_1=pd.melt(df,id_vars=df.columns.values[:-2],
              value_vars=df.columns.values[-2:])
df_1
```

Factory_Temp_Max	Factory_Humidity	Factory_Humidity_Min	Factory_Humidity_Max	variable	value
22.0	58.4	18.0	22.0	cavity1out2	0
22.0	58.2	18.0	22.0	cavity1out2	0
22.0	58.2	18.0	22.0	cavity1out2	0
22.0	58.2	18.0	22.0	cavity1out2	3
22.0	57.8	18.0	22.0	cavity1out2	0
...	...	...	...	...	...
22.0	69.4	18.0	22.0	cavity2out2	0
22.0	69.5	18.0	22.0	cavity2out2	0
22.0	69.5	18.0	22.0	cavity2out2	0
22.0	69.6	18.0	22.0	cavity2out2	0
22.0	69.6	18.0	22.0	cavity2out2	0

[코드 16] 데이터프레임 재구조화 코드

cavity1out2	cavity2out2	variable	value
0	0	cavity1out2	0
0	0	cavity1out2	0
0	0	cavity1out2	0
3	0	cavity1out2	3
0	0	cavity1out2	0
...	...	...	...
0	0	cavity2out2	0
4	0	cavity2out2	0
4	0	cavity2out2	0
4	0	cavity2out2	0
4	0	cavity2out2	0

[그림 20] 변경 전

[그림 21] 변경 후

- 비효율적인 열의 구조를 행으로 변환하여 재구조화하여 학습시키기 위한 pd.melt를 사용한다. 이 모듈은 가로로 긴 데이터프레임을 세로로 길게 만들어 재구조화한다. 칼럼 cavity1out2, cavity2out2를 기준으로 재구조화한다. id_vars는 기준열을 설정하는 파라미터이고, value_vars는 선택한 칼럼을 variable에 나열하고 해당 칼럼의 값을 value에 나열한다. 그 결과, 위와 같이 variable에 Cavity 번호가, value에는 불량 유형이 나타난 것을 확인할 수 있다. 그림 20은 재구조화 전, 그림 21은 재구조화 후 데이터프레임의 일부이다.

④-7. 칼럼과 인덱스 최종 정리

```
# 캐비티 1번이면 0, 2번이면 1
df_1["cavity"] = np.where(df_1["variable"] == "cavity1out2", 0, 1)
df_1
```

Factory_Humidity	Factory_Humidity_Min	Factory_Humidity_Max	variable	value	cavity
58.4	18.0	22.0	cavity1out2	0	0
58.2	18.0	22.0	cavity1out2	0	0
58.2	18.0	22.0	cavity1out2	0	0
58.2	18.0	22.0	cavity1out2	3	0
57.8	18.0	22.0	cavity1out2	0	0
...	...	...	...	...	...
69.4	18.0	22.0	cavity2out2	0	1
69.5	18.0	22.0	cavity2out2	0	1
69.5	18.0	22.0	cavity2out2	0	1
69.6	18.0	22.0	cavity2out2	0	1
69.6	18.0	22.0	cavity2out2	0	1

[코드 17] 정수형 데이터로 정리하는 코드

- variable 컬럼의 값으로 문자열 “cavity1out2”와 “cavity2out2” 대신에 정수형인 0과 1로 작성하고 이 값들을 variable 열 대신에 “cavity” 열을 생성하여 저장한다.

```
# 중복항 제거
df_1.drop_duplicates(inplace=True)

# 불필요한 열 제거
df_1 = df_1.drop(columns=["variable"])

# 인덱스 정리
df_1.reset_index(drop=True, inplace=True)
df_1
```

Factory_Humidity	Factory_Humidity_Min	Factory_Humidity_Max	value	cavity
58.4	18.0	22.0	0	0
58.2	18.0	22.0	0	0
58.2	18.0	22.0	0	0
58.2	18.0	22.0	3	0
57.8	18.0	22.0	0	0
...	...	...	...	...
69.3	18.0	22.0	0	1
69.4	18.0	22.0	0	1
69.4	18.0	22.0	0	1
69.5	18.0	22.0	0	1
69.6	18.0	22.0	0	1

[코드 18] 데이터 정리하는 코드

- 주조 공정은 반복 작업으로 데이터의 수집 과정에서 동일한 데이터를 반복적으로 저장하는 경우가 생기고 데이터프레임을 재구조화하는 과정에서 중복된 행이 들어난다. 중복된 행을 제거하고 variable열의 전처리를 통해 cavity열을 생성하였기 때문에, 분석에 사용하지 않는 variable열을 제거한다. 여러 행들을 제거하는 과정에서 인덱스가 정돈되지 않았다. reset_index를 통해 재정렬해준다.

[단계 ⑤] 학습/평가 데이터 분리

⑤-1. 학습/평가 데이터 분리 사전 작업

```
# input_data, target_data 분리
x_list=df_1.columns.drop(["value"])

xx=df_1.loc[:,x_list]
yy=df_1.loc[:, "value"]

yy.value_counts()
```

```
0    7615
1     380
5     288
3     207
4     182
2       66
Name: value, dtype: int64
```

[코드 19] 학습/평가 데이터 분리 사전 작업

- 학습/평가 데이터 분할에 앞서 xx와 yy 변수로 분리한다. xx를 input data (독립변수), yy를 target data(종속변수)라 한다.
- 위 코드에서는 yy의 종류별(불량 유형) 개수를 확인할 수 있다. 각 번호의 의미는 ④-4를 참조한다.

```
# 캐비티 타입 변환
xx["cavity"]=xx["cavity"].astype("category")
xx.dtypes
```

```
Product_Type          int64
Velocity_1            float64
Velocity_2            float64
Velocity_3            float64
High_Velocity         float64
Cylinder_Pressure     int64
Rapid_Rise_Time       float64
Biscuit_Thickness     int64
Clamping_Force        int64
Cycle_Time            float64
Pressure_Rise_Time    float64
Casting_Pressure      int64
Spray_Time            float64
Spray_1_Time          float64
Spray_2_Time          float64
Melting_Furnace_Temp  float64
Air_Pressure          float64
Air_Pressure_Min      int64
Air_Pressure_Max      int64
Coolant_Temp          float64
Coolant_Temp_Min      int64
Coolant_Temp_Max      int64
Coolant_Pressure      float64
Factory_Temp          float64
Factory_Temp_Min      float64
Factory_Temp_Max      float64
Factory_Humidity      float64
Factory_Humidity_Min  float64
Factory_Humidity_Max  float64
cavity                category
dtype: object
```

[코드 20] 캐비티 타입 변환

- 추후 [단계 ⑤-B-1]에서 오버 샘플링을 적용할 때 category 타입을 가지는 변수를 설정하기 위해 cavity 칼럼의 데이터 타입을 int64형에서 category로 변환한다.

[단계 ⑤-1] 이후부터 [단계 ⑧]까지는 총 4가지 방법으로 모델을 생성하고 테스트하는 과정이 반복적으로 수행된다. 4가지 방법에 대해서는 A, B, C, D 순서로 설명한다.

A: XGBoost

- 일반적인 지도학습 방법으로 학습/평가 데이터 분할 후 모델을 생성하고 훈련한다.

B: XGBoost + SMOTE

- 학습/평가 데이터 분리에 앞서 불균형한 데이터([코드 19]참고)를 보완하기 위해 SMOTE기법으로 데이터가 적은 클래스의 데이터를 늘려 모든 클래스의 데이터가 동일하도록 만든다. 그 후 모델을 생성하고 학습한다.



C: XGBoost + SMOTE + Feature_Importance(변별력 없는 변수 제거)

- SMOTE 기법 적용 후 지도학습 모델링에 내장된 함수 Feature_Importance를 활용해 변수 중 요도를 확인한다. 소수점 셋째 자리에서 반올림했을 때 값이 0이면 변별력이 없는 변수로 판단하여 제거하고 남은 변수로만 학습한다.

D: XGBoost + SMOTE + Feature_Importance(상위 10개 변수)

- C에서 변별력 없는 변수를 제거한 것이 아닌 상위 10개 변수만을 이용해 학습한다.

A. XGBoost

[단계 ⑤-A] 학습/평가 데이터 분리

```
# 학습/평가 데이터 분리
x_train,x_test,y_train,y_test=train_test_split(xx,yy,test_size=0.3,random_state=1234)
```

[코드 21] A-학습/평가 데이터 분리

- 머신러닝의 과정은 데이터 학습과 데이터의 검정 단계로 나뉜다.
- 전체 데이터를 7 : 3의 비율로 train_test_split() 함수를 활용하여 학습용 데이터와 검정용 데이터로 분할한다.

[단계 ⑥-A] AI모델 구축

```
# AI모델 구축
xg1=GradientBoostingClassifier(n_estimators=700)
```

[코드 22] A-AI모델 구축

- Gradient Boosting은 깊이가 얇은 결정 트리를 사용하여 이전 트리의 오차를 보완하는 방식으로 앙상블 하는 기법이다. 결정 트리 개수는 700개이고 학습률 learning_rate의 기본값은 0.1이다.

[단계 ⑦-A] AI모델 훈련

```
# AI모델 훈련
xg1.fit(x_train,y_train)
xg1_pred=xg1.predict(x_test)
```

[코드 23] A-AI모델 훈련

- 학습용 데이터인 x_train과 y_train으로 [단계 ⑥-A]에서 생성한 모델을 훈련시킨다. 검정용 데이터 x_test를 학습된 모델을 이용해 테스트한다. xg1_pred는 테스트 예측 결과이다.



[단계 ⑧-A] 결과 분석 및 해석

```
# 결과 분석 및 해석
accuracy_score(y_test, xg1_pred)
```

```
0.8405797101449275
```

[코드 24] A-결과 분석 및 해석

- 예측 결과를 올바르게 예측한 데이터 비율을 accuracy_score를 통해 구한다.
- 모델링 방법 A의 예측 결과 정확도는 84%이고, 비교적 낮은 성능이라고 판단할 수 있다.

B. XGBoost + SMOTE

[단계 ⑤-B-1] 학습/평가 데이터 분리 사전 작업

```
x_st=StandardScaler().fit(x_train)
x_train_st=x_st.transform(x_train)
x_test_st=x_st.transform(x_test)

## SMOTE 적용
X_smote,y_smote=SMOTENC(random_state=1234,categorical_features=[29]).fit_resample(xx,yy)
X_smote=pd.DataFrame(X_smote)
X_smote.columns=xx.columns
X_smote.cavity.unique()
pd.Series(y_smote).value_counts()
```

```
0    7615
3    7615
5    7615
1    7615
2    7615
4    7615
Name: value, dtype: int64
```

[코드 25] B-학습/평가 데이터 분리 사전 작업

- 불균형한 데이터를 보완하기 위해 오버 샘플링을 사용한다. 오버 샘플링에 앞서 데이터의 특성마다 범위가 달라 정규분포(평균 0, 분산 1)를 만들어 주는 스케일링을 거친다.
- 모든 클래스의 개수가 동일한 것을 확인할 수 있다.

[단계 ⑤-B-2] 학습/평가 데이터 분리

```
# 학습/평가 데이터 분리
xs_train,xs_test,ys_train,ys_test=train_test_split(X_smote,y_smote,test_size=0.3,random_state=1234)
```

[코드 26] B-학습/평가 데이터 분리

- 오버 샘플링하여 동일한 클래스의 개수를 갖춘 전체 데이터를 7 : 3의 비율로 학습용 데이터와 검정용 데이터로 분할한다.



[단계 ⑥-B] AI모델 구축

```
# AI모델 구축
xg_smote=GradientBoostingClassifier(n_estimators=900,learning_rate=0.1)
```

[코드 27] B-AI모델 구축

- Gradient Boosting은 깊이가 얇은 결정 트리를 사용하여 이전 트리의 오차를 보완하는 방식으로 앙상블 하는 기법이다. 결정 트리 개수는 900개이고 학습률 learning_rate는 기본값 0.1이다.

[단계 ⑦-B] AI모델 훈련

```
#Ai모델 훈련
xg_smote.fit(xs_train,ys_train)
xg_smote_pred=xg_smote.predict(xs_test)
```

[코드 28] B-AI모델 훈련

- 학습용 데이터인 xs_train과 ys_train으로 [단계 ⑥-B]에서 생성한 모델을 훈련시킨다. 훈련된 모델은 검정용 데이터 xs_test를 활용해 테스트한다. xg_smote_pred는 테스트한 예측 결과이다.

[단계 ⑧-B] 결과 분석 및 해석

```
# 결과 분석 및 해석
accuracy_score(ys_test,xg_smote_pred)
```

```
0.9479827825198803
```

[코드 29] B-결과 분석 및 해석

- 예측 결과를 올바르게 예측한 데이터 비율을 accuracy_score를 통해 구한다.
- B의 예측 결과가 A보다 약 10% 정도 높은 95%의 정확도를 보여준다.

C. XGBoost + SMOTE + Feature_Importance(변별력 없는 변수 제거)

[단계 ⑤-C-1] 학습/평가 데이터 분리 사전 작업

```

asmo_xg_smote=xg_smote.feature_importances_
asmo_xg_smote=pd.DataFrame(asmo_xg_smote)

asmo_xg_smote["var"]=xs_train.columns
asmo_xg_smote.columns.values[0]="importance"
asmo_xg_smote_imp=asmo_xg_smote.sort_values(by="importance",ascending=False)
asmo_xg_smote_imp
  
```

importance	var		
19	0.102377	Coolant_Temp	
29	0.085061	cavity	
8	0.084950	Clamping_Force	
15	0.081650	Melting_Furnace_Temp	
26	0.073904	Factory_Humidity	
23	0.069579	Factory_Temp	
9	0.061139	Cycle_Time	
12	0.061034	Spray_Time	
2	0.059953	Velocity_2	
22	0.042343	Coolant_Pressure	
4	0.035458	High_Velocity	
1	0.034626	Velocity_1	
3	0.034521	Velocity_3	
11	0.032240	Casting_Pressure	
10	0.030887	Pressure_Rise_Time	
6	0.030160	Rapid_Rise_Time	
16	0.027004	Air_Pressure	
5	0.015285	Cylinder_Pressure	
7	0.014518	Biscuit_Thickness	
14	0.013324	Spray_2_Time	
13	0.009986	Spray_1_Time	
0	0.000002	Product_Type	
21	0.000000	Coolant_Temp_Max	
20	0.000000	Coolant_Temp_Min	
24	0.000000	Factory_Temp_Min	
25	0.000000	Factory_Temp_Max	
18	0.000000	Air_Pressure_Max	
27	0.000000	Factory_Humidity_Min	
28	0.000000	Factory_Humidity_Max	
17	0.000000	Air_Pressure_Min	

[코드 30] C-학습/평가 데이터 분리 사전 작업

```

remove_list=["Product_Type","Coolant_Temp_Max","Coolant_Temp_Min","Factory_Temp_Min","Factory_Temp_Max",
"Air_Pressure_Max","Factory_Humidity_Min","Factory_Humidity_Max","Air_Pressure_Min"] # 소수점 셋째 자리 반올림
  
```

[코드 31] C-변수 제거 목록

- B 모델에서 구축한 학습 모델 xg_smote의 내장함수 feature_importances를 활용하여 변수 중요도를 구한 후 크기 순으로 나열한다. 정리된 수를 소수점 셋째 자리에 반올림했을 때 값이 0이면 해당 변수를 제거할 변수로 정한다.
- 제거할 변수 목록은 Product_Type, Coolant_Temp_Max, Coolant_Temp_Min, Factory_Temp_Min, Factory_Temp_Max, Air_Pressure_Max, Factory_Humidity_Min, Factory_Humidity_Max, Air_Pressure_Min이다. 해당 변수 목록을 remove_list에 저장하여 나중에 제거하기 쉽도록 한다.



[단계 ⑤-C-2] 학습/평가 데이터 분리

학습/평가 데이터 분리

```
xs_train_r=pd.DataFrame(xs_train,columns=x_list).drop(columns=remove_list)
xs_test_r=pd.DataFrame(xs_test,columns=x_list).drop(columns=remove_list)
xs_train_r.columns
```

```
Index(['Velocity_1', 'Velocity_2', 'Velocity_3', 'High_Velocity',
       'Cylinder_Pressure', 'Rapid_Rise_Time', 'Biscuit_Thickness ',
       'Clamping_Force ', 'Cycle_Time', ' Pressure_Rise_Time',
       'Casting_Pressure', 'Spray_Time', 'Spray_1_Time', 'Spray_2_Time',
       'Melting_Furnace_Temp', 'Air_Pressure', 'Coolant_Temp',
       'Coolant_Pressure', 'Factory_Temp', 'Factory_Humidity', 'cavity'],
      dtype='object')
```

[코드 32] C-학습/평가 데이터 분리

- [단계 ⑤-C-1]에서 선정한 제거할 칼럼을 xs_train과 xs_test에서 제거한다. xs_train과 xs_test은 B에서 나눈 평가/학습 데이터이다.

[단계 ⑥-C] AI모델 구축

AI모델 구축

```
asmo_xg_smote_r=GradientBoostingClassifier(n_estimators=900,learning_rate=0.1)
```

[코드 33] C-AI모델 구축

- Gradient Boosting은 깊이가 얇은 결정 트리를 사용하여 이전 트리의 오차를 보완하는 방식으로 앙상블 하는 기법이다. 결정 트리 개수는 900개이고 학습률 learning_rate는 기본값 0.1을 유지한다.

[단계 ⑦-C] AI모델 훈련

AI모델 훈련

```
asmo_xg_smote_r.fit(xs_train_r,ys_train)
asmo_xg_smote_pred_r=asmo_xg_smote_r.predict(xs_test_r)
```

[코드 34] C-AI모델 훈련

- 중요도가 낮은 칼럼을 제거한 학습용 데이터 xs_train_r과 ys_train_r로 [단계 ⑥-C]에서 생성한 모델을 훈련시킨다. 검정용 데이터 xs_test_r로 학습된 모델을 테스트한다. asmo_xg_smote_pred_r는 예측 결과이다.

[단계 ⑧-C] 결과 분석 및 해석

결과 분석 및 해석

```
accuracy_score(ys_test,asmo_xg_smote_pred_r)
```

```
0.9479098270956445
```

[코드 35] C-결과 분석 및 해석

- 예측 결과를 올바르게 예측한 데이터 비율을 accuracy_score를 통해 구한다.
- C의 모델링 예측 결과는 B의 결과와 유사하게 95%의 정확도를 보여준다.



D. XGBoost + SMOTE + Feature_Importance(상위 10개 변수)

[단계 ⑤-D-1] 학습/평가 데이터 분리 사전 작업

```
xx_r=pd.DataFrame(asmo_xg_smote_r.feature_importances_)
xx_r["var"]=xs_train_r.columns
xx_r.columns.values[0]="importance"
xx_r_imp=xx_r.sort_values(by="importance",ascending=False)

## 영향력 있는 변수 10개 선정
imp_variable=xx_r_imp[0:10]["var"].tolist()
imp_variable
```

```
['Coolant_Temp',
 'cavity',
 'Clamping_Force ',
 'Melting_Furnace_Temp',
 'Factory_Humidity',
 'Factory_Temp',
 'Cycle_Time',
 'Spray_Time',
 'Velocity_2',
 'Coolant_Pressure']
```

[코드 36] D-학습/평가 데이터 분리 사전 작업

- 앞서 정리한 C 방법에서 구축한 학습 모델 asmo_xg_smote_r의 내장함수 feature_importances를 활용하여 변수 중요도가 가장 높은 10개 변수를 구한다. 선정한 10개 변수만을 가지고 학습한다.

[단계 ⑤-D-2] 학습/평가 데이터 분리

```
# 학습/평가 데이터 분리
xs_train_imp=xs_train_r[imp_variable]
xs_test_imp=xs_test_r[imp_variable]
```

[코드 37] D-학습/평가 데이터 분리

- [단계 ⑤-D-1]에서 선정한 10개 변수만을 남기고 나머지는 제거한다.

[단계 ⑥-D] AI모델 구축

```
# AI모델 구축
asmo_xg_smote_imp=GradientBoostingClassifier(n_estimators=900,learning_rate=0.1)
```

[코드 38] D-AI모델 구축

- Gradient Boosting은 깊이가 얇은 결정 트리를 사용하여 이전 트리의 오차를 보완하는 방식으로 앙상블 하는 기법이다. 결정 트리 개수는 900개이고 학습률 learning_rate는 기본값 0.1을 유지한다.



[단계 ⑦-D] AI모델 훈련

```
# AI모델 훈련
asmo_xg_smote_imp.fit(xs_train_imp,ys_train)
asmo_xg_smote_pred_imp=asmo_xg_smote_imp.predict(xs_test_imp)
```

[코드 39] D-AI모델 훈련

- [단계 ⑤-D-2]에서 생성한 학습용 데이터를 학습시킨다. 검정용 데이터 `xs_test_imp`를 사용하여 학습된 모델을 테스트한다. 예측한 결과는 `asmo_xg_smote_pred`이다.

[단계 ⑧-D] 결과 분석 및 해석

```
# 결과 분석 및 해석
accuracy_score(ys_test,asmo_xg_smote_pred_imp)
```

```
0.9034070183118115
```

[코드 40] D-결과 분석 및 해석

- 예측 결과를 올바르게 예측한 데이터 비율을 `accuracy_score`를 통해 구한다.
- D의 모델링 예측 결과는 90%의 정확도를 보여준다. 오히려 B와 C보다 낮은 결과를 보여준다. 상위 10개 변수보다 비교적 중요도가 낮게 나온 변수들도 정확도에 영향을 미치는 것을 확인할 수 있다.

분석 방법	A	B	C	D
정확도	84%	95%	95%	90%

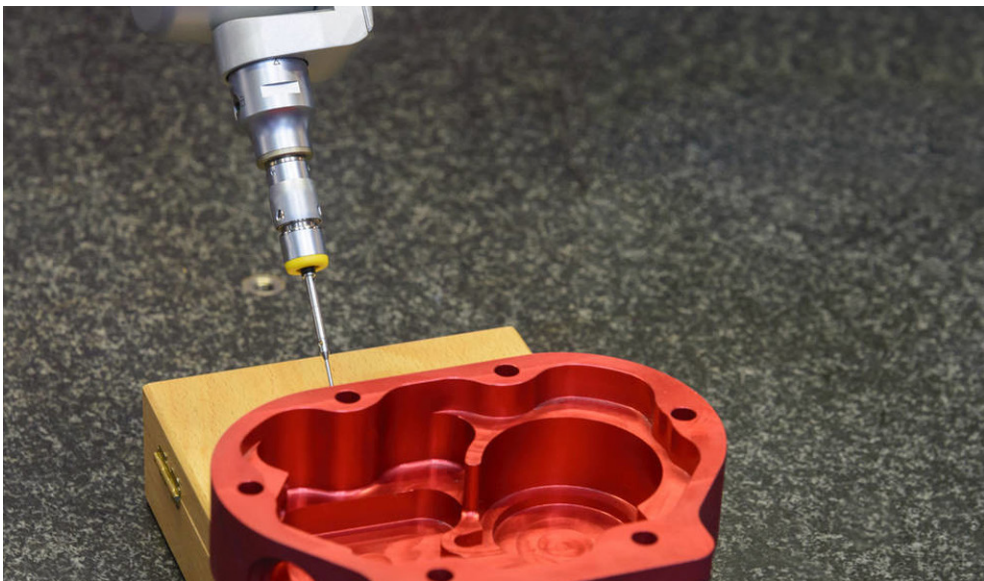
[표 7] 4개 분석 방법의 결과 비교

⇒ 최종적으로, 모델 B와 C가 정확도 95%로 가장 높게 나왔다. 모델 D는 정확도 90%로 세 번째, 모델 A는 정확도 84%로 가장 낮게 나왔다. A와 나머지 모델을 통해 오버샘플링이 정확도를 높여준다는 것을 알 수 있다. 또한, C와 D를 통해 변수 중요도가 낮더라도 모델 학습에 유의미한 영향을 끼친다는 것을 알 수 있다.

3. 유사 타 현장의 「주조 품질보증 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 제조현장에서 생산되는 제품에 대한 양품과 불량 여부 판단과 불량 유형에 대한 확인은 다이캐스팅 주조 공정 뿐 아니라 대부분의 공정에서 품질 관리의 매우 중요한 부분이다.
- 따라서 본 분석의 결과는 다양한 제조공정의 유형에서 유사하게 적용될 수 있다고 판단된다. 즉, 제조설비에서 양산 중 발생하는 공정 데이터를 수집할 수 있고, 이에 대한 품질검사 데이터를 확보하여 지도학습 데이터셋을 구성할 수 있는 경우는 해당 모델을 적용하여 유의미한 결과를 도출할 수 있다.
- 특히 본 분석에서는 총 4가지의 모델링 접근(A. XGBoost, B. XGBoost + SMOTE, C. XGBoost + SMOTE + Feature_Importance(변별력 없는 변수 제거), D. XGBoost + SMOTE + Feature_Importance(상위 10개 변수))을 순차적으로 진행하며 그 결과를 비교하여 제시하였다. 일반적으로 제조현장에서 발생하는 품질 검사 데이터는 양품과 불량의 클래스 간 불균형 문제를 모두 포함하고 있기 때문에 제시된 방법론들은 모두 SMOTE 기법을 통해 모델링 전에 해당 문제를 해결할 수 있도록 지원하였다. 그러나 데이터셋에 따라 SMOTE 외에 불균형 문제를 해결할 수 있는 다른 기법들을 적용할 수 있다. 예를 들어 Over Sampling이 아닌 Under Sampling 혹은 Combination Sampling 기법들을 적용할 수 있고, 혹은 Over Sampling 기법 중에서도 수치 범주형 데이터에 적합한 SMOTENC, SMOTE의 변형인 Borderline-SMOTE 등을 적용할 수 있다.
- 또한 XGBoost 앙상블 모델은 다른 머신러닝 모델에 비해 일반적으로 좋은 성능을 나타낸다고 평가받고 있고, 제조공정의 품질 예측에 많이 활용되어 검증된 방법론이기 때문에 다이캐스팅 주조 공정 뿐 아니라 다양한 유형의 공정에 일반적으로 적용될 수 있다고 기대한다.



[그림 22] 다이캐스팅 주조 품질 검사 자동화/고도화 참고자료



3.2 본 「주조 품질보증 AI 데이터셋」 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 대상 설비에서 생산이 진행되면서 수집되는 공정 변수에 대한 실시간 접근 및 활용이 가능한 인프라 구축이 필요하다.
- 타 제조현장 적용 시에 충분한 예산과 시간이 확보된다면 센서 데이터를 유사하게 활용할 수 있지만 그렇지 못한 상황이라면 기본적으로 설비에서 수집되는 데이터와 품질 정보에 우선순위를 두고 분석을 진행하는 것이 필요하다.
- 본 분석에서는 다이캐스팅 주조 설비의 매 shot과 cavity 별 제품 단위로 공정 변수, 센서 데이터와 품질검사 결과를 확보하여 하나의 데이터셋으로 구성하였다. 그러나 대부분의 제조현장은 Lot 단위로 품질검사를 진행하고 정보를 관리하고 있기 때문에 본 분석을 적용하기 어려울 수 있다. 이같은 경우에는, Lot 단위로 수집되는 품질검사 데이터와 공정/센서 데이터 간 불균형 문제를 대표값을 feature로 활용하는 등의 방안으로 해결할 수 있다.
- 분석 결과는 해당 공정 및 품질 도메인 전문가와 함께 해석되어야 하며, 현장 적용여부는 분석 알고리즘의 성능지표 뿐 아니라 생산지표 및 현장상황을 고려하여 진행되어야 한다.



3 부록_분석환경 구축을 위한 설치 가이드

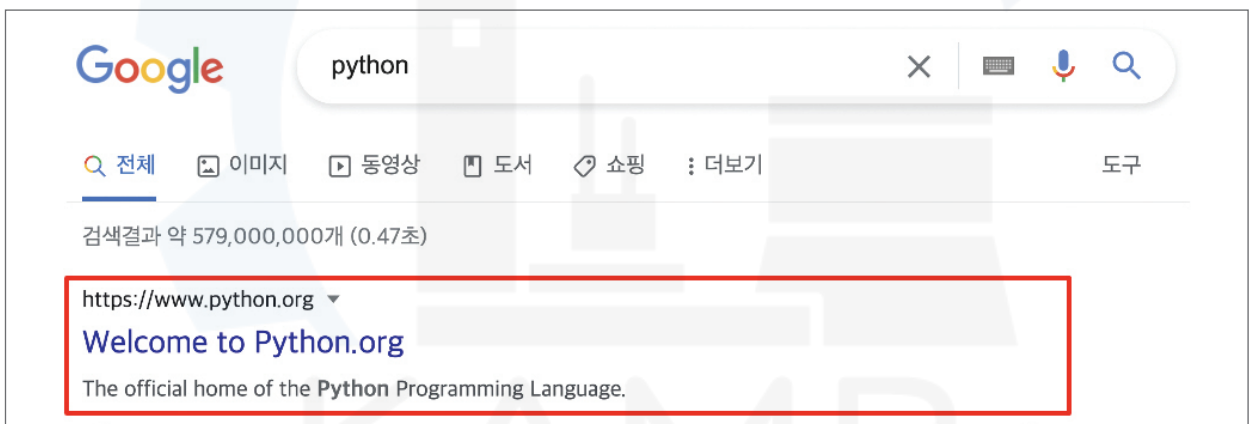
1. 파이썬(Python) 설치

파이썬은 컴퓨터 프로그래밍 언어로서, 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하는 방법을 안내한다.

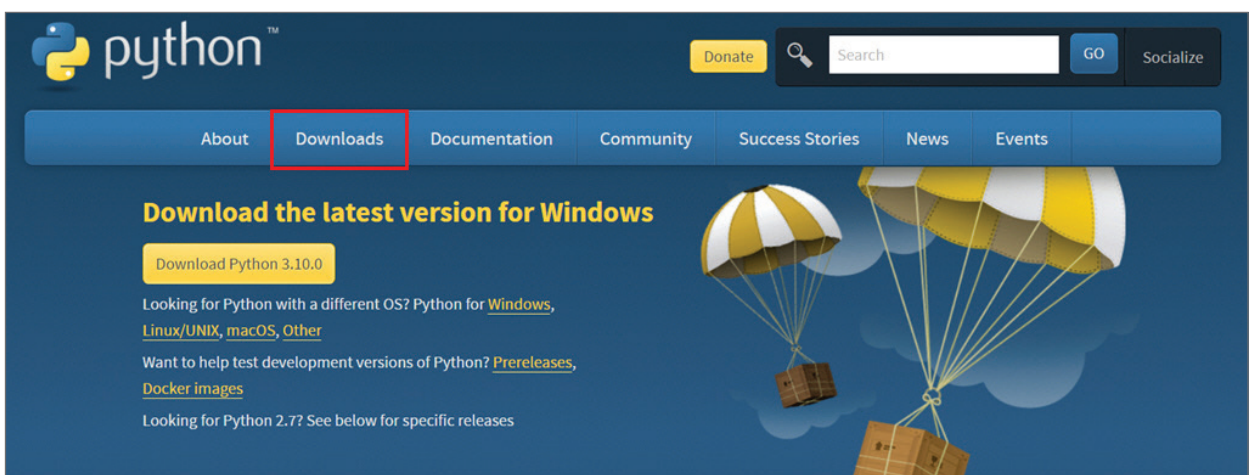
① google.com 등의 검색 엔진에 'python'을 검색한다.



② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



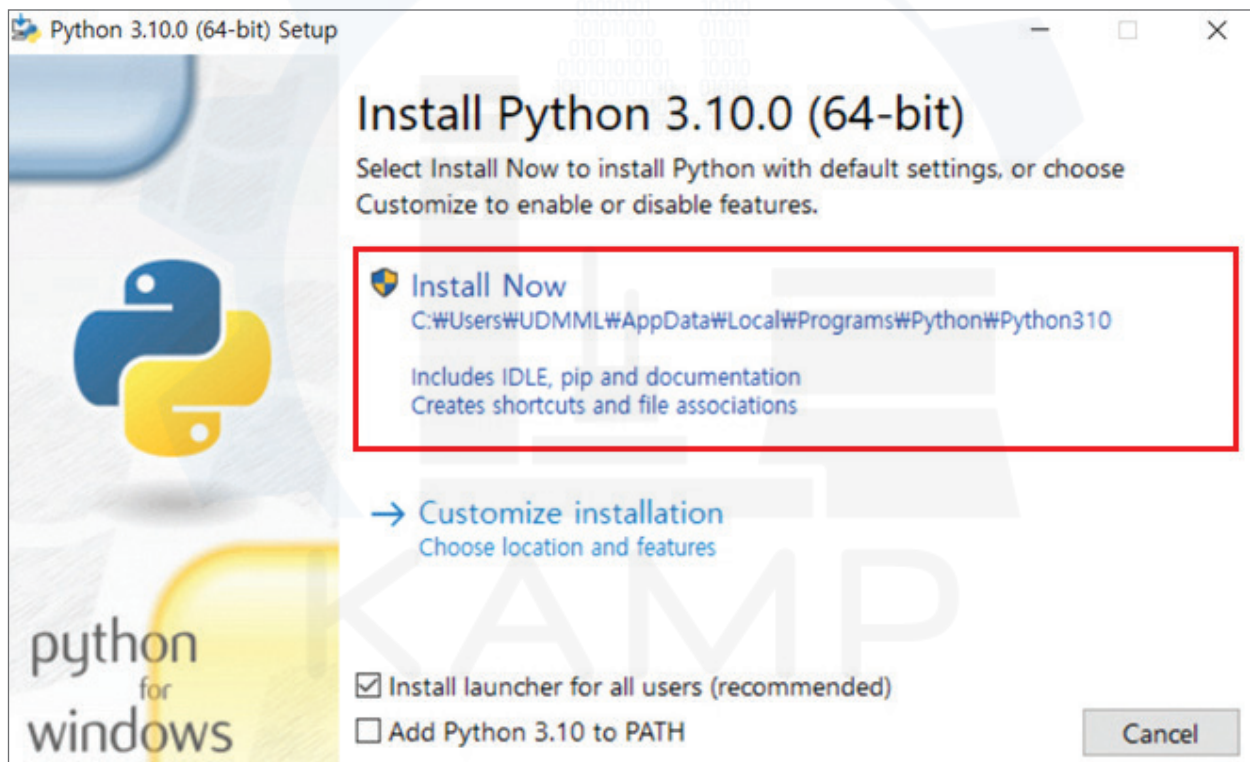
③ 클릭 후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



- ④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 버전은 업데이트 될 수 있다.)

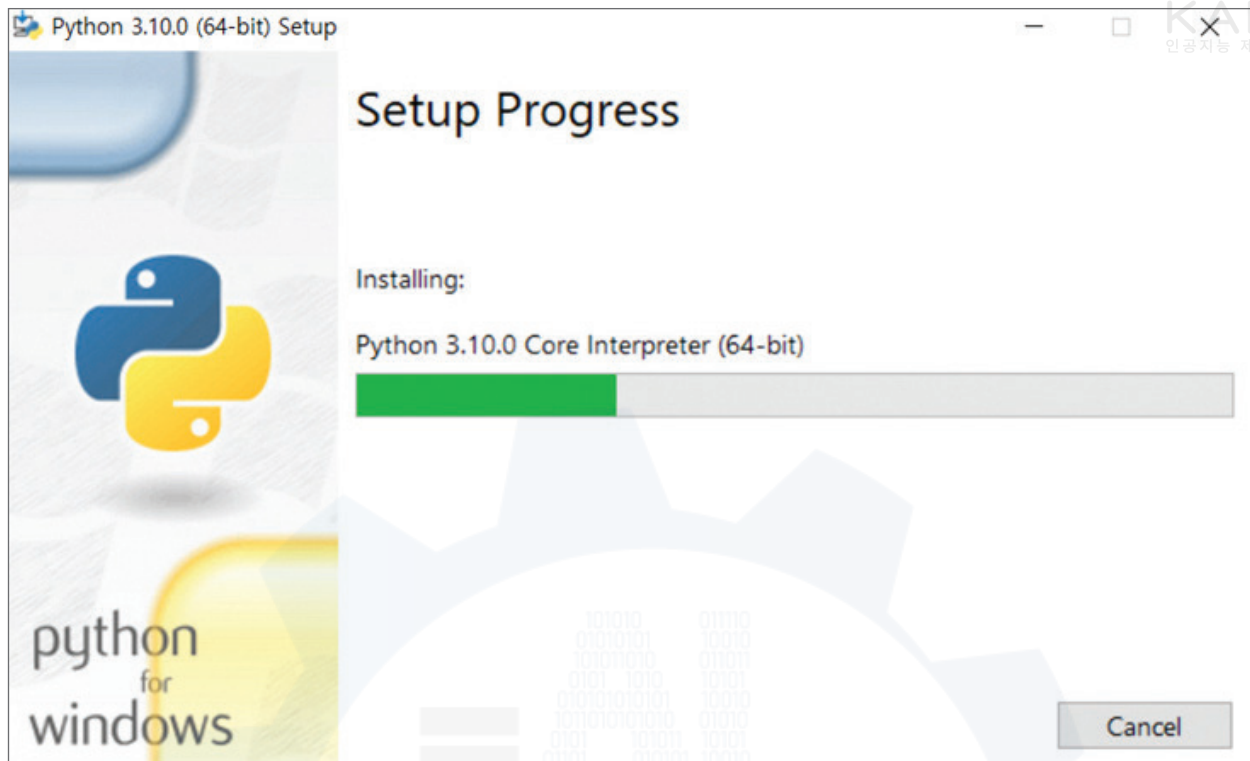


- ⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.

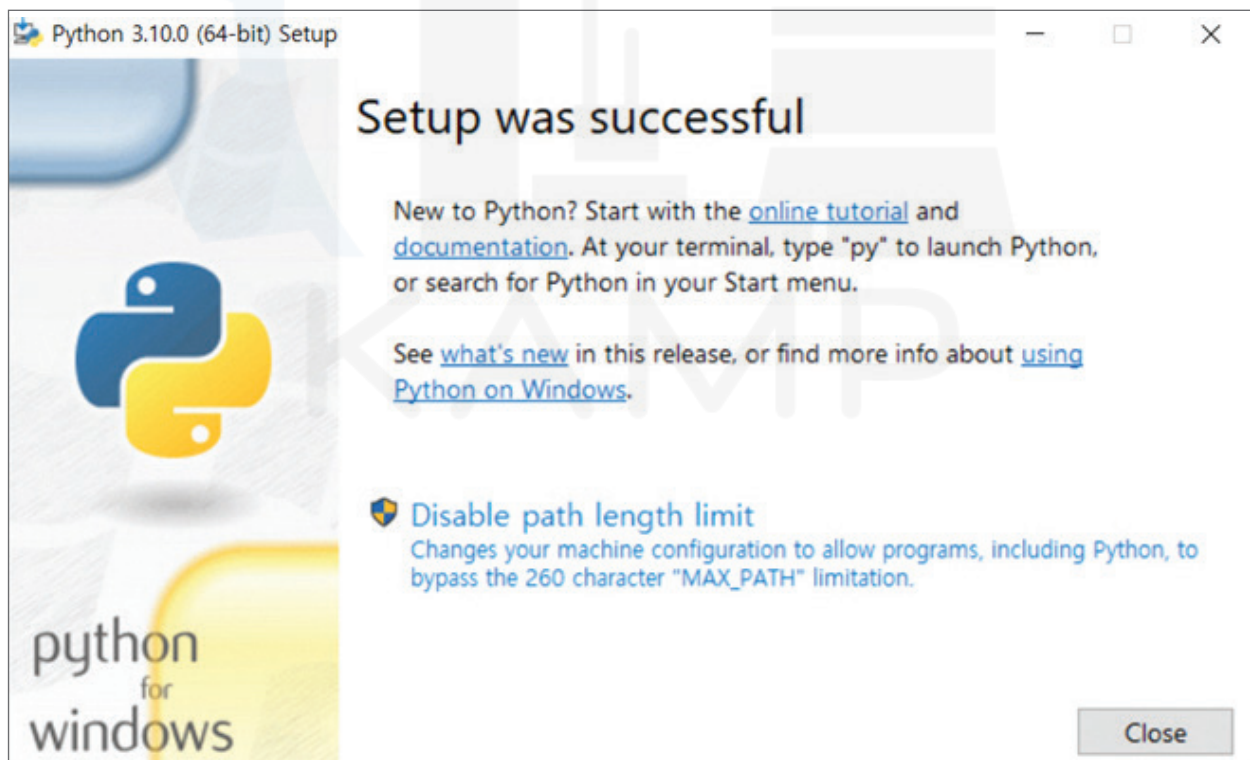




⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



⑦ 아래와 같은 창이 나타나면 설치 완료된 것으로, 확인 후 종료한다. [설치완료]

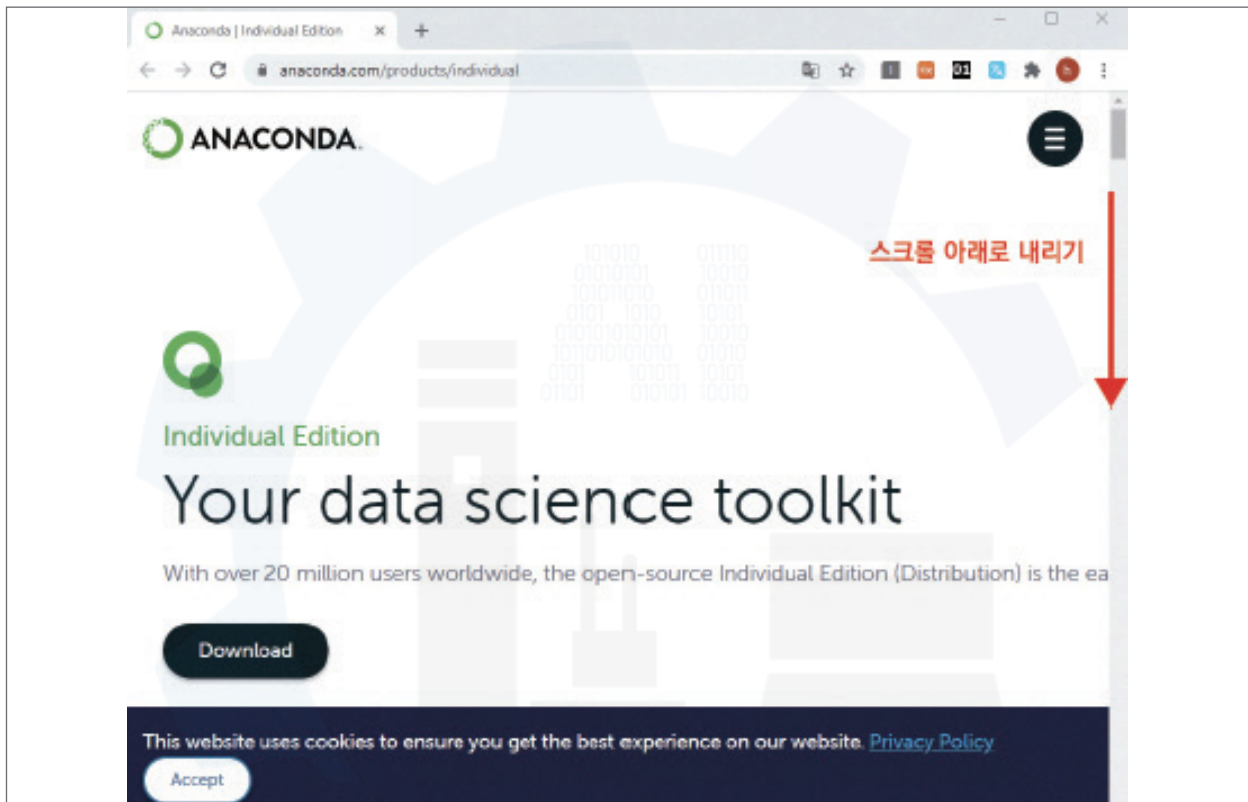




2. 아나콘다(Anaconda) 설치

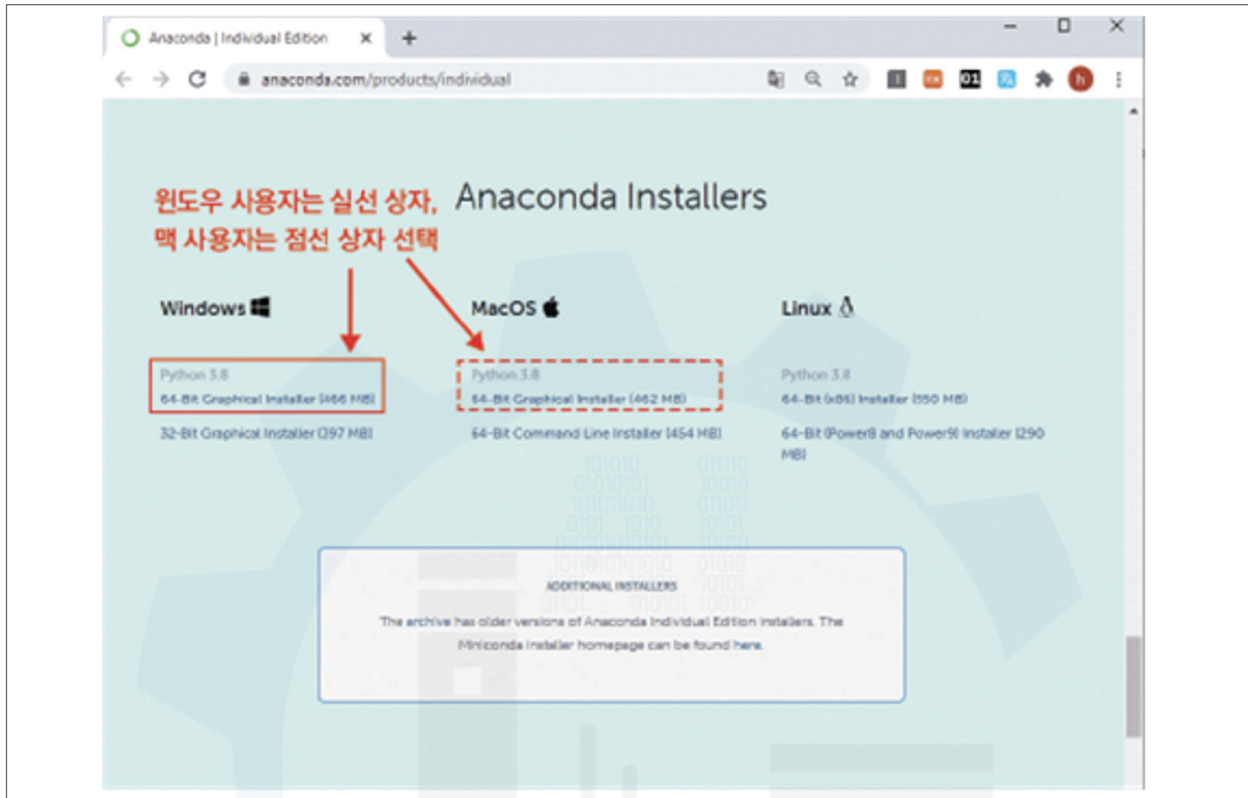
- 아나콘다란 파이썬 등의 데이터 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 효율적으로 사용할 수 있으며, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고, 데이터를 분석할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



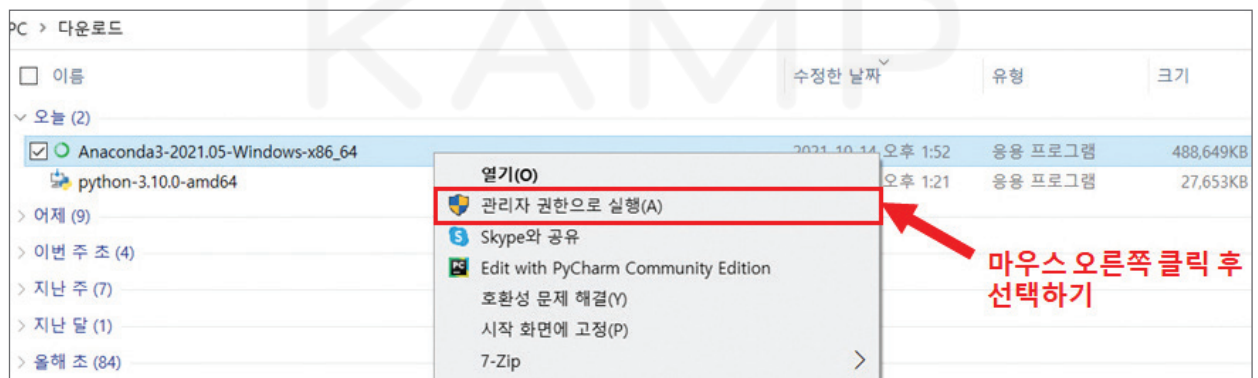
② 스크롤을 내려 다음의 화면을 확인한 후, 로컬 컴퓨터 사용 환경에 맞는 아나콘다 설치 파일을 다운로드한다.

- ▶ Windows : 64-Bit Graphical Installer
- ▶ MacOS : 64-Bit Graphical Installer



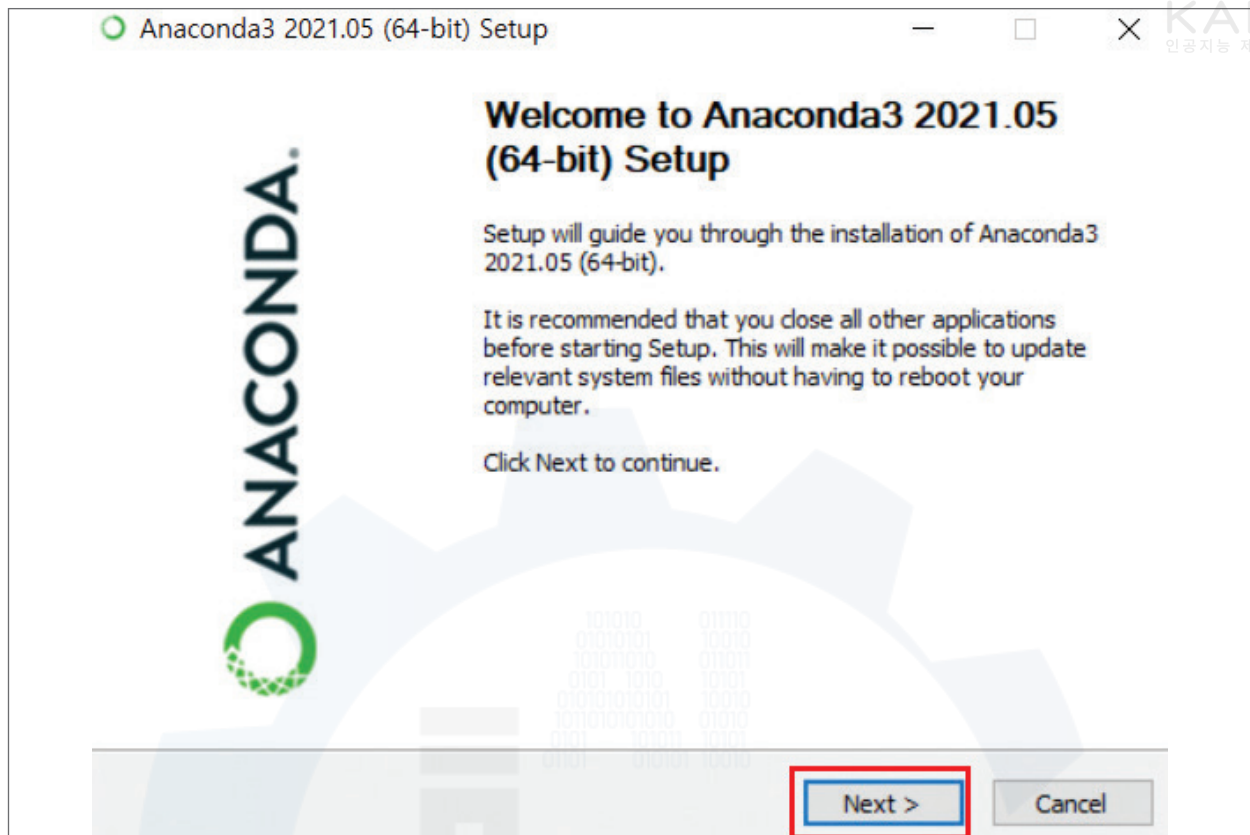
③ 다운로드한 아나콘다 설치 파일에서 마우스 오른쪽을 클릭한 후, 방패모양의 ‘관리자 권한으로 실행’을 클릭한다.

- ▶ (예) ‘다운로드’ 폴더로 아나콘다를 다운로드한 경우

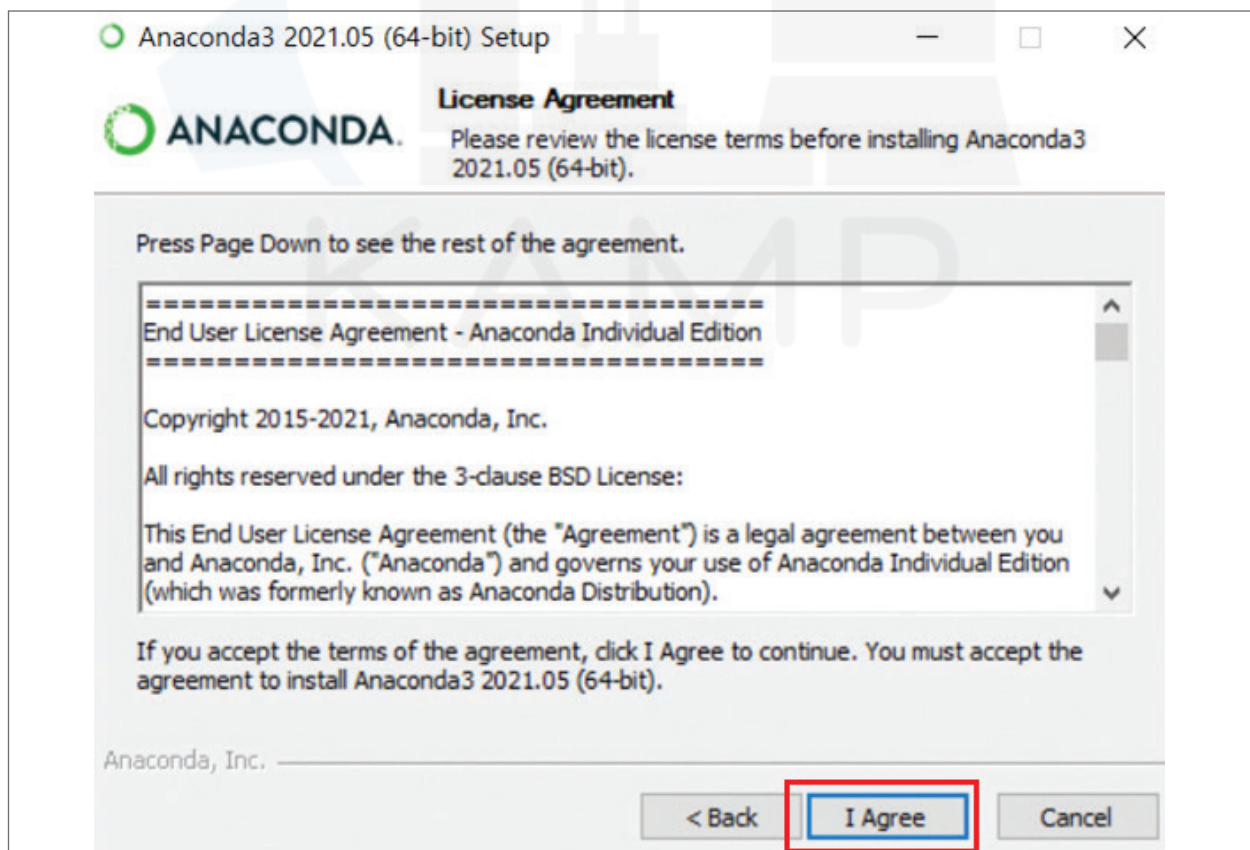




④ 설치 파일을 실행한 다음, 'Next' 버튼을 클릭한다.

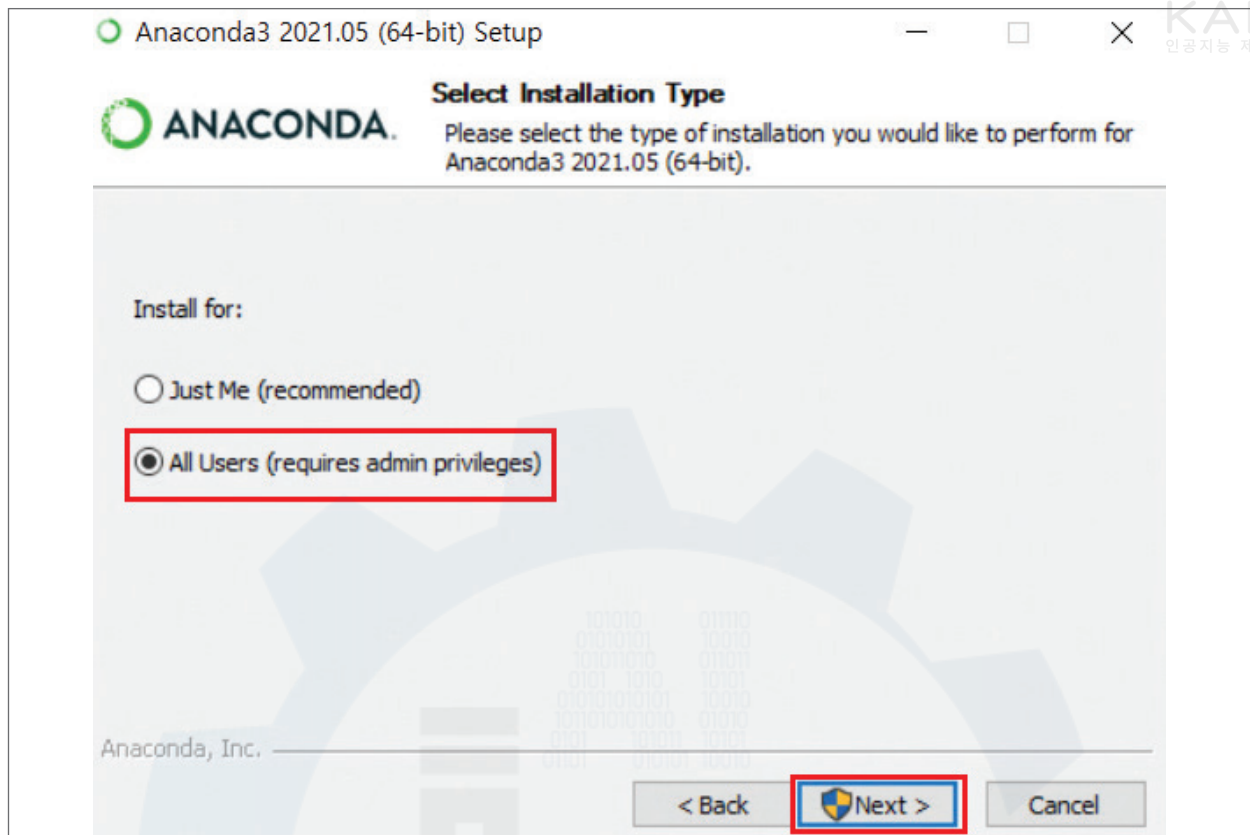


⑤ 다음 단계에서 아래와 같은 창이 나타나면, 'I Agree'를 선택한다.

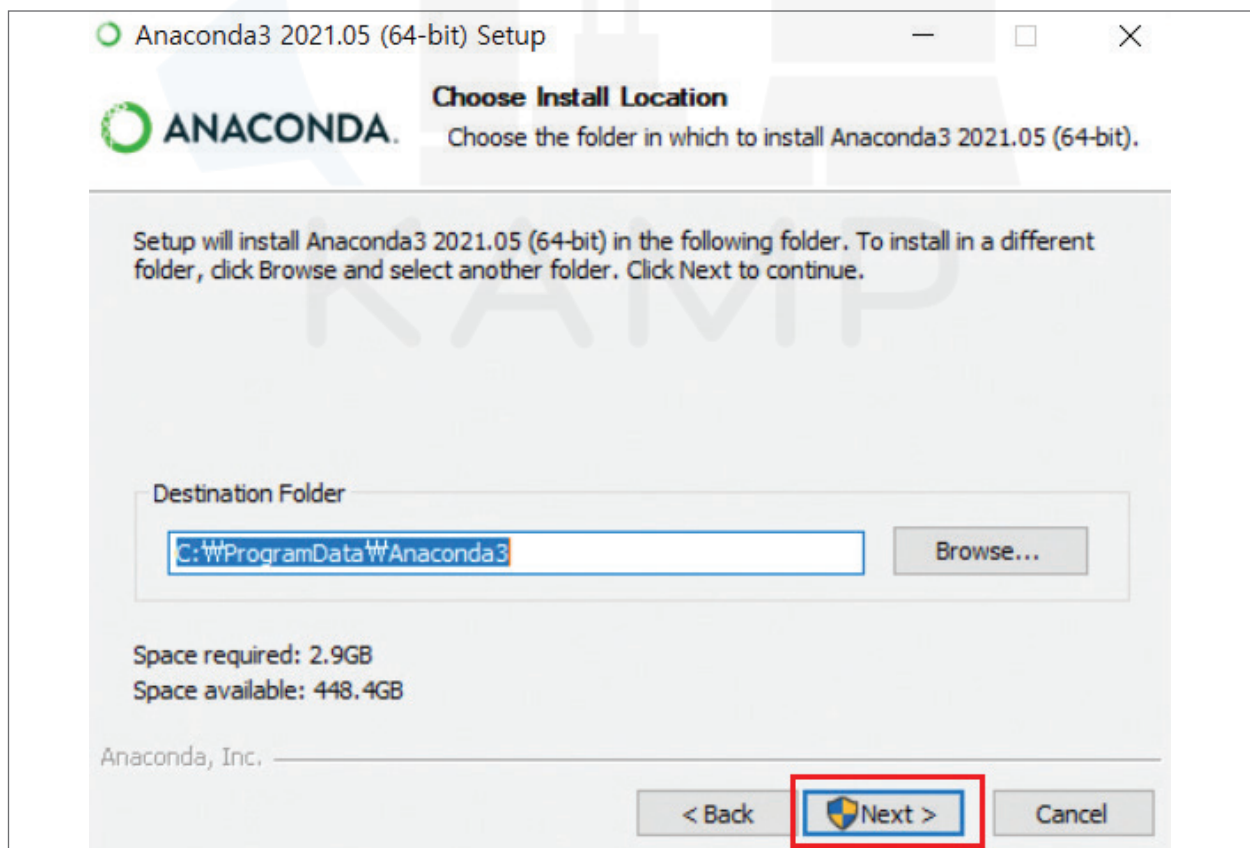




⑥ 아래와 같은 창이 나타나면, 'All Users'를 선택하여 다음 단계로 진행한다.

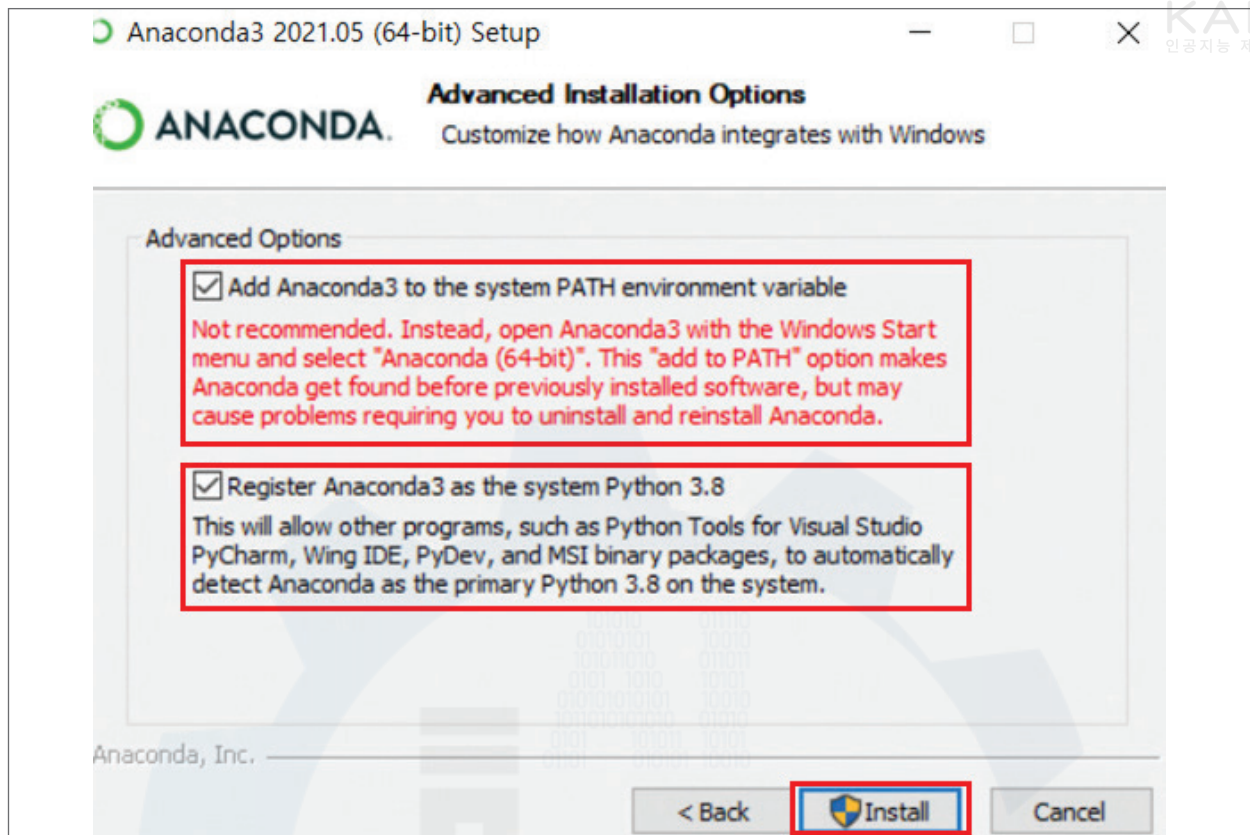


⑦ 다운로드할 경로를 설정한 뒤, 아래의 'Next'를 클릭한다.

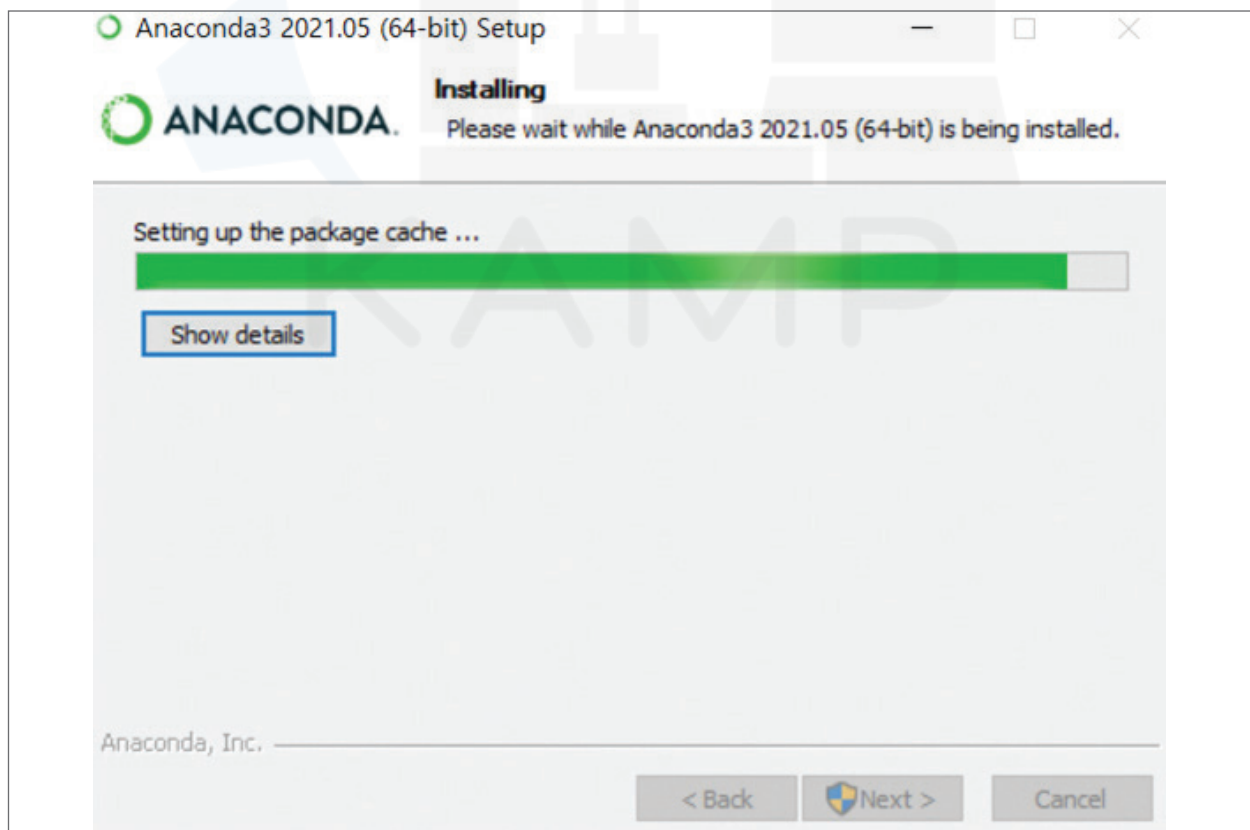




⑧ 고급 옵션 선택창에서 아래와 같이 모두 선택 후, 'Install'을 클릭한다.

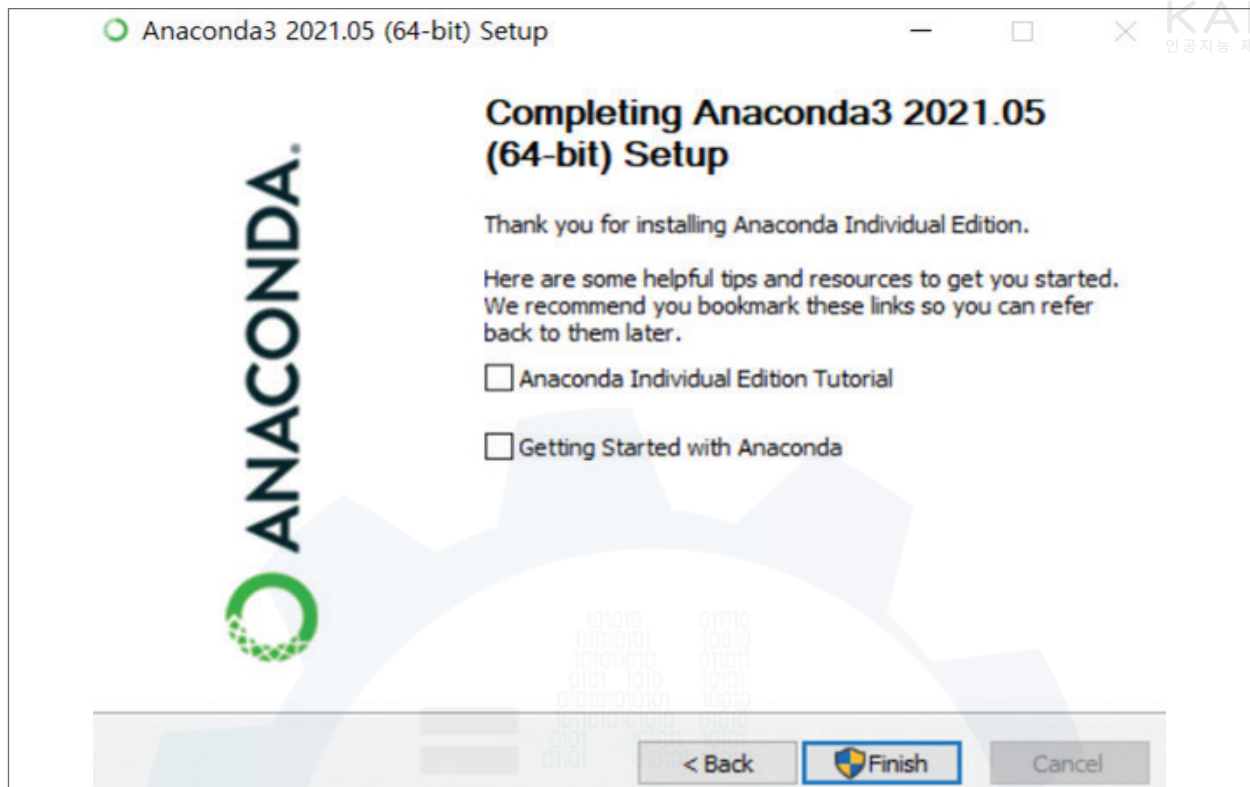


⑨ 다음과 같은 설치창으로 넘어가면, 완료가 될 때까지 대기한다. (5분이상 소요)

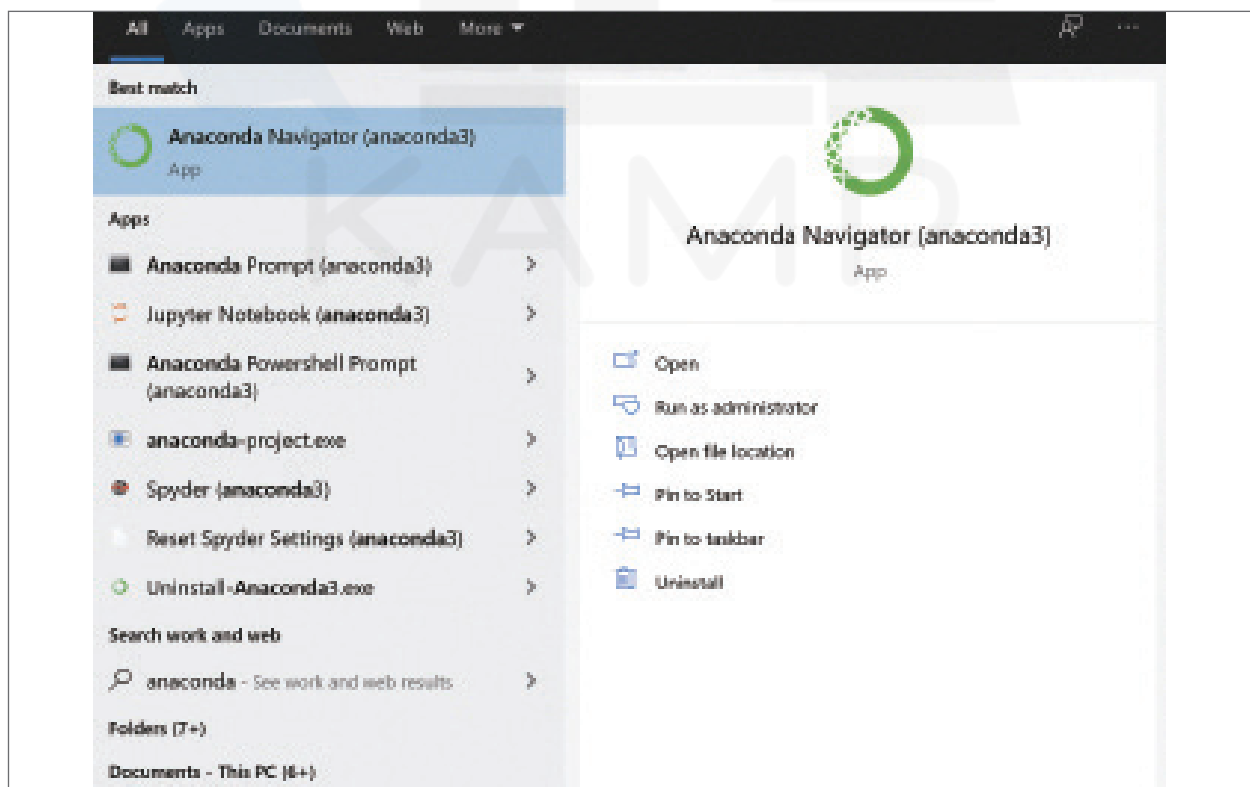




⑩ 마지막 화면에서 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



⑪ PC의 '시작 버튼(Windows icon)'을 눌러서 화면과 같이 anaconda prompt가 설치되었는지 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.

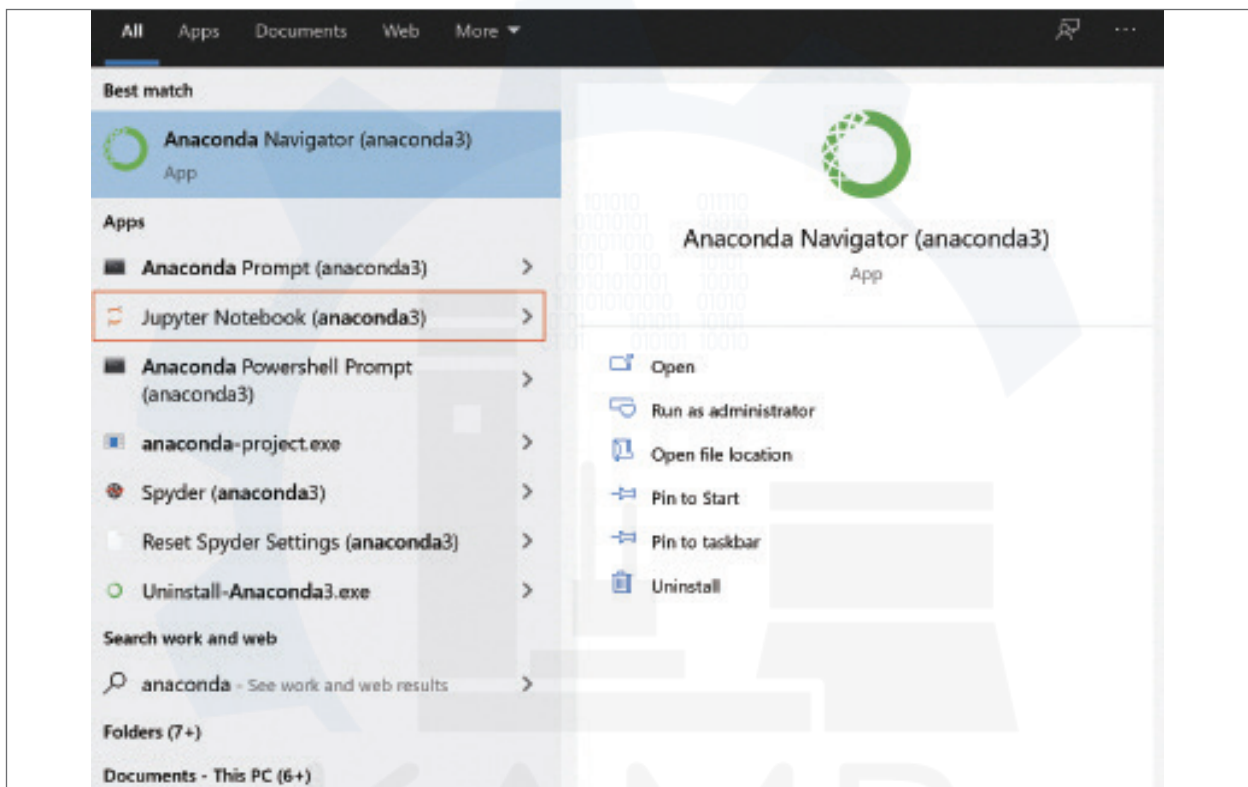




3. 주피터 노트북 (Jupyter Notebook) 실행

주피터 노트북은 사용자가 코딩(분석 코드 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 ‘Word’ 프로그램이나, 한컴소프트 사의 ‘한글’ 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 주피터 노트북을 활용하는 방법을 안내한다.

- ① PC의 ‘시작 버튼(Windows icon)’을 눌러서 화면과 같이 ‘anaconda’를 검색한 후 폴더 리스트 중 ‘Jupyter Notebook (anaconda3)’을 실행한다.





② Jupyter Notebook을 실행하면, 아래와 같이 2개의 윈도우가 실행된다.

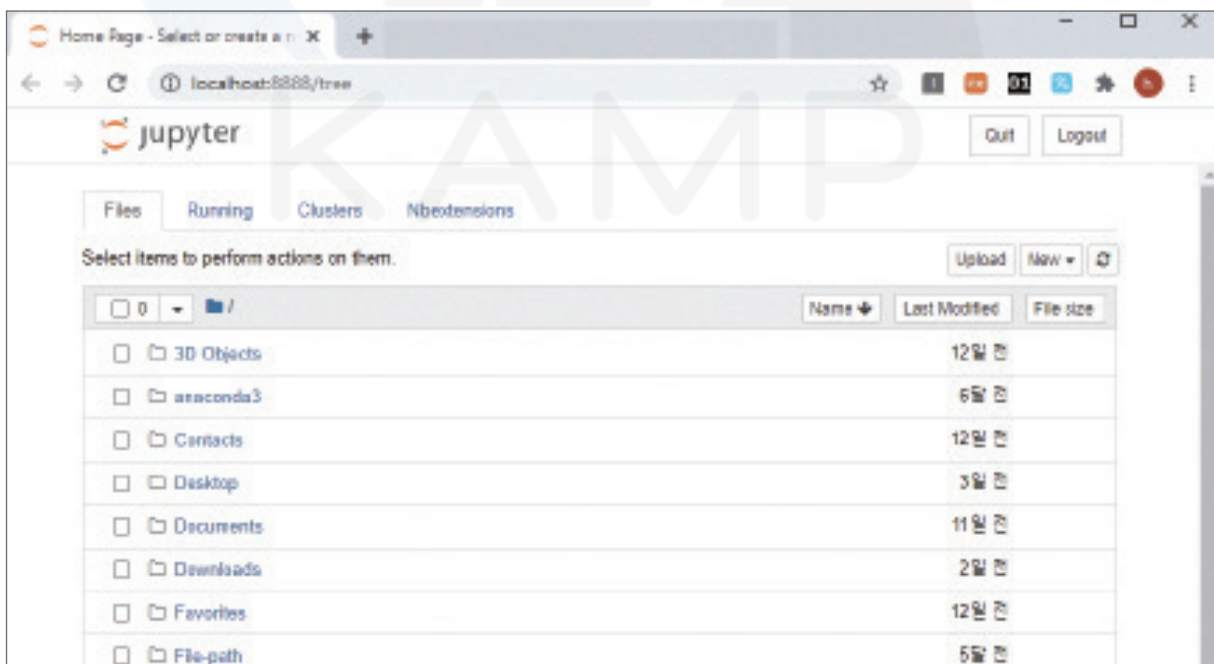
- (1) 아래 검은색 배경의 화면은 주피터 노트북이 실행되는 환경 상태를 나타내주는 상태 표시 창이다. 주피터 노트북을 사용하는 동안에는 종료하면 안된다.

```

Jupyter Notebook (anaconda3)
[I 12:39:38.566 NotebookApp] JupyterLab extension loaded from C:\Users\leedaeyoung\Anaconda3\lib\site-packages\jupyterlab
[I 12:39:38.566 NotebookApp] JupyterLab application directory is C:\Users\leedaeyoung\Anaconda3\share\jupyter\lab
[I 12:39:38.627 NotebookApp] Serving notebooks from local directory: C:\Users\leedaeyoung
[I 12:39:38.628 NotebookApp] The Jupyter Notebook is running at:
[I 12:39:38.628 NotebookApp] http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 12:39:39.853 NotebookApp]

To access the notebook, open this file in a browser:
file:///C:/Users/leedaeyoung/AppData/Roaming/jupyter/runtime/nbserver-6568-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fdb8b7b86d823ba8bc31a88c2
  
```

- (2) 기본 브라우저(크롬, 엣지 등)에서 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야 한다.)

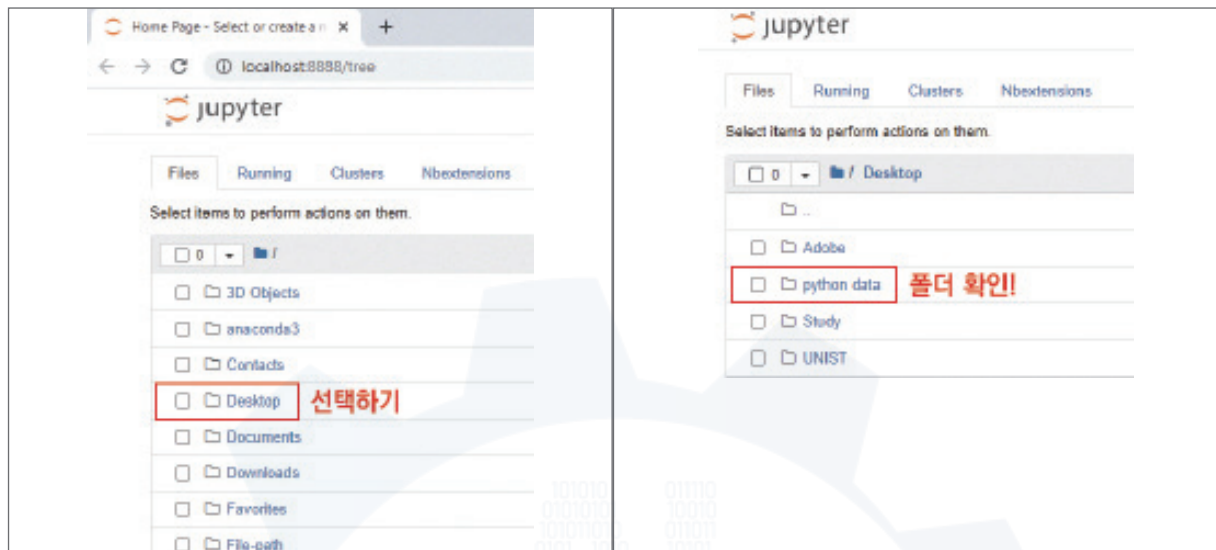




③ 데이터 분석을 진행할 폴더를 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성한 다음, 파이썬 코드 실행하고 저장한다.

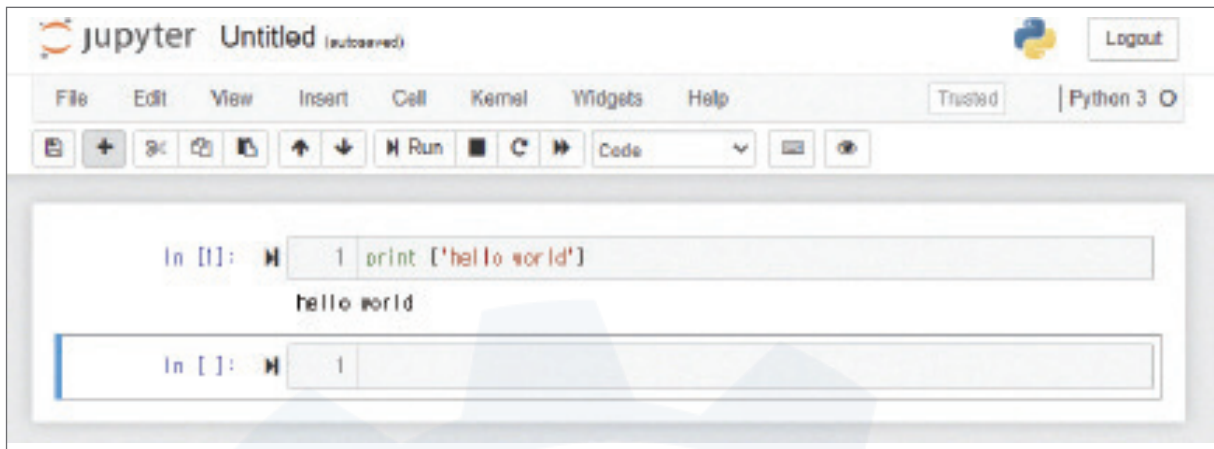
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



(2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world')를 입력한 다음, 상단의  Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '*.ipynb'로 저장된다.



4 부록_데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한, Gartner에서 제시한 6종의 데이터 품질 평가 지표(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)에 대하여 논의하였다.

결측치가 처리된 데이터셋을 기준으로 품질지수를 계산하고자 한다. 데이터를 불러온 후 3가지 과정을 거친다. 첫째, 멀티칼럼의 상위레벨을 제거한다. 둘째, 결측치를 제거한다. 셋째, 인덱스를 재정렬한다. 위 과정을 마친 후 품질지수를 구한다. 단, 정확성 품질지수를 구하기 위해 결측치가 처리되기 전 상태에서 원본 데이터를 복사해둔다.

```
import pandas as pd
import numpy as np

df=pd.read_csv('DieCasting_Quality_Raw_Data.csv', header=[0,1])
df.columns=df.columns.droplevel(0)
df_origin=df.copy()
df=df.dropna()
df=df.reset_index(drop=True)
df
```

	id	Product_Type	Shot	Velocity_1	Velocity_2	Velocity_3	High_Velocity	Cylinder_Pressure	Rapid_Rise_Time
0	1	1	1	0.144	0.170	0.188	2.134	214	0.008
1	1002	1	2	0.144	0.170	0.182	2.124	217	0.008
2	2003	1	3	0.144	0.170	0.182	2.116	214	0.008
3	3004	1	4	0.144	0.170	0.182	2.137	217	0.008
4	4005	1	5	0.144	0.172	0.176	2.111	217	0.008
...	...	...	...	...	...	...	...	...	...
7440	7530659	2	659	0.150	0.166	0.210	2.492	265	0.011
7441	7531660	2	660	0.144	0.174	0.206	2.514	264	0.011
7442	7532660	2	660	0.144	0.174	0.206	2.514	264	0.011
7443	7533661	2	661	0.147	0.174	0.204	2.532	265	0.012
7444	7534661	2	661	0.147	0.174	0.204	2.532	265	0.012

[코드 1] 데이터 불러오기 및 전처리 코드

▶ 완전성 품질 지수 = ((1-결측치)/전체 데이터 수) * 100

- ① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 Null 값의 비율을 확인하여 삭제한다. 이 데이터는 30%를 넘는 열이 존재하지 않으므로 열을 제거하지 않는다.

: 기본적인 Raw Data는 결측치를 포함하고 있으나, 데이터 전처리 과정에서 처리(삭제)하기 때문에 결측치가 처리된 데이터셋을 기준으로 완전성 품질 지수를 계산한다.


```
perc =30
dc_data.isnull().sum()/len(dc_data)*100
```

id	0.0		
Product_Type	0.0	Factory_Humidity_Min	0.0
Shot	0.0	Factory_Humidity_Max	0.0
Velocity_1	0.0	Short_Shot_1	0.0
Velocity_2	0.0	Bubble_1	0.0
Velocity_3	0.0	Exfoliation_1	0.0
High_Velocity	0.0	Blow_Hole_1	0.0
Cylinder_Pressure	0.0	Stain_1	0.0
Rapid_Rise_Time	0.0	Dent_1	0.0
Biscuit_Thickness	0.0	Deformation_1	0.0
Clamping_Force	0.0	Contamination_1	0.0
Cycle_Time	0.0	Impurity_1	0.0
Pressure_Rise_Time	0.0	Crack_1	0.0
Casting_Pressure	0.0	Scratch_1	0.0
Spray_Time	0.0	Buring_Mark_1	0.0
Spray_1_Time	0.0	Inclusions_1	0.0
Spray_2_Time	0.0	Short_Shot_2	0.0
Melting_Furnace_Temp	0.0	Bubble_2	0.0
Air_Pressure	0.0	Exfoliation_2	0.0
Air_Pressure_Min	0.0	Blow_Hole_2	0.0
Air_Pressure_Max	0.0	Stain_2	0.0
Coolant_Temp	0.0	Dent_2	0.0
Coolant_Temp_Min	0.0	Deformation_2	0.0
Coolant_Temp_Max	0.0	Contamination_2	0.0
Coolant_Pressure	0.0	Impurity_2	0.0
Factory_Temp	0.0	Crack_2	0.0
Factory_Temp_Min	0.0	Scratch_2	0.0
Factory_Temp_Max	0.0	Buring_Mark_2	0.0
Factory_Humidity	0.0	Inclusions_2	0.0

[코드 2] 완전성 품질지수 코드 (1)

```
perc=30
dc_data.isnull().sum()/len(dc_data)*100>perc
```

id	False		
Product_Type	False	Factory_Humidity_Min	False
Shot	False	Factory_Humidity_Max	False
Velocity_1	False	Short_Shot_1	False
Velocity_2	False	Bubble_1	False
Velocity_3	False	Exfoliation_1	False
High_Velocity	False	Blow_Hole_1	False
Cylinder_Pressure	False	Stain_1	False
Rapid_Rise_Time	False	Dent_1	False
Biscuit_Thickness	False	Deformation_1	False
Clamping_Force	False	Contamination_1	False
Cycle_Time	False	Impurity_1	False
Pressure_Rise_Time	False	Crack_1	False
Casting_Pressure	False	Scratch_1	False
Spray_Time	False	Buring_Mark_1	False
Spray_1_Time	False	Inclusions_1	False
Spray_2_Time	False	Short_Shot_2	False
Melting_Furnace_Temp	False	Bubble_2	False
Air_Pressure	False	Exfoliation_2	False
Air_Pressure_Min	False	Blow_Hole_2	False
Air_Pressure_Max	False	Stain_2	False
Coolant_Temp	False	Dent_2	False
Coolant_Temp_Min	False	Deformation_2	False
Coolant_Temp_Max	False	Contamination_2	False
Coolant_Pressure	False	Impurity_2	False
Factory_Temp	False	Crack_2	False
Factory_Temp_Min	False	Scratch_2	False
Factory_Temp_Max	False	Buring_Mark_2	False
Factory_Humidity	False	Inclusions_2	False

[코드 3] 완전성 품질지수 코드 (2)

- ② 데이터의 결측치를 확인하기 위하여 isnull() 함수를 사용한 뒤, sum() 함수를 이용하여 총 결측치 개수를 구한다.

```
df.isnull().sum()
```

id	0	Factory_Humidity_Min	0
Product_Type	0	Factory_Humidity_Max	0
Shot	0	Short_Shot_1	0
Velocity_1	0	Bubble_1	0
Velocity_2	0	Exfoliation_1	0
Velocity_3	0	Blow_Hole_1	0
High_Velocity	0	Stain_1	0
Cylinder_Pressure	0	Dent_1	0
Rapid_Rise_Time	0	Deformation_1	0
Biscuit_Thickness	0	Contamination_1	0
Clamping_Force	0	Impurity_1	0
Cycle_Time	0	Crack_1	0
Pressure_Rise_Time	0	Scratch_1	0
Casting_Pressure	0	Buring_Mark_1	0
Spray_Time	0	Inclusions_1	0
Spray_1_Time	0	Short_Shot_2	0
Spray_2_Time	0	Bubble_2	0
Melting_Furnace_Temp	0	Exfoliation_2	0
Air_Pressure	0	Blow_Hole_2	0
Air_Pressure_Min	0	Stain_2	0
Air_Pressure_Max	0	Dent_2	0
Coolant_Temp	0	Deformation_2	0
Coolant_Temp_Min	0	Contamination_2	0
Coolant_Temp_Max	0	Impurity_2	0
Coolant_Pressure	0	Crack_2	0
Factory_Temp	0	Scratch_2	0
Factory_Temp_Min	0	Buring_Mark_2	0
Factory_Temp_Max	0	Inclusions_2	0
Factory_Humidity	0		

[코드 4] 완전성 품질지수 코드 (3)



```
cmpt_len=df.isnull().sum().sum()
cmpt_len
```

```
0
```

[코드 5] 완전성 품질지수 코드 (4)

③ 구한 결측치의 개수를 이용하여 완전성 품질지수를 구한다.

```
print("결측치 = %d개 \n완전성 지수 : %i%%"%(cmpt_len,(1-cmpt_len/len(df))*100))
```

```
결측치 = 0개
완전성 지수 : 100.00%
```

[코드 6] 분석 도구를 활용한 완전성 품질지수 구하기

▶ **유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) * 100**

: 이 데이터는 '_id' 컬럼이 유일한 값을 가지는 기본키이다.

① unique() 함수를 이용하여 '_id' 컬럼이 가지는 유일한 값들을 확인하고, 유일한 데이터 개수를 구한다.

```
df['_id'].unique()
array([      1,    1002,    2003, ..., 7532660,
       7533661, 7534661],
      dtype=int64)
```

[코드 7] 유일성 품질지수 코드 (1)

```
len(df['_id'].unique())
```

```
7445
```

[코드 8] 유일성 품질지수 코드 (2)

② 구한 유일한 데이터 개수를 이용하여 유일성 품질지수를 구한다.

```
uniq_len=len(df['_id'].unique())
print("유일성 지수 : %i%%"%((uniq_len/len(df))*100))
```

```
유일성 지수 : 100%
```

[코드 9] 분석 도구를 활용한 유일성 품질지수 구하기



▶ 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수)*100

① 데이터가 유효범위 내에 들어가 있는가?

: 데이터셋에 정의된 수집 범위를 이용하여 조건(유효 범위)을 모두 충족하는지 확인한다. 모든 공정 데이터는 음수일 수 없기 때문에, 음수인 데이터가 존재할 시에 경고문을 출력하고 값을 확인한다.

```
nonval_count = 0
for col in df.columns:
    tmp_chk = False
    for dc in df[col]:
        tmp_chk = (dc >= 0)
    if not tmp_chk:
        print("non validation data detected",dc)
        nonval_count += 1
        break
val_rate_1 = (1 - nonval_count / len(df.columns))
print("유효성 지수 : %i%%" % (val_rate_1 * 100))
```

유효성 지수 : 100%

[코드 10] 분석 도구를 활용한 유효성 품질지수 구하기

- 최종적으로 음수 데이터가 포함되지 않은 것을 확인하였다.



▶ 일관성 품질 지수 = (일관성 만족 데이터수/전체 데이터 수)*100

- ① 본 데이터의 구조, 값, 형태가 일관성이 있는지 확인한다. 'dtypes()' 함수를 이용하여 전체 데이터의 형태를 확인한다.

```
print(df.dtypes)
nonval_count=0
for col in df.columns:
    tmp_chk=False
    col_c=""
    for i,dc in enumerate(df[col]):
        if i==0:
            if 'int' in str(type(dc)):
                col_c='int'
            elif 'float' in str(type(dc)):
                col_c='float'
    tmp_chk=(col_c in str(type(df[col])))
    if not tmp_chk:
        nonval_count+=1
val_rate_2=(1-nonval_count/len(df))
print("일관성 지수: %i%%"%(val_rate_1+val_rate_2)/2*100))
```

id	int64	Factory_Humidity	float64
Product_Type	int64	Factory_Humidity_Min	float64
Shot	int64	Factory_Humidity_Max	float64
Velocity_1	float64	Short_Shot_1	int64
Velocity_2	float64	Bubble_1	int64
Velocity_3	float64	Exfoliation_1	int64
High_Velocity	float64	Blow_Hole_1	int64
Cylinder_Pressure	int64	Stain_1	int64
Rapid_Rise_Time	float64	Dent_1	int64
Biscuit_Thickness	int64	Deformation_1	int64
Clamping_Force	int64	Contamination_1	int64
Cycle_Time	float64	Impurity_1	int64
Pressure_Rise_Time	float64	Crack_1	int64
Casting_Pressure	int64	Scratch_1	int64
Spray_Time	float64	Buring_Mark_1	int64
Spray_1_Time	float64	Inclusions_1	int64
Spray_2_Time	float64	Short_Shot_2	int64
Melting_Furnace_Temp	float64	Bubble_2	int64
Air_Pressure	float64	Exfoliation_2	int64
Air_Pressure_Min	int64	Blow_Hole_2	int64
Air_Pressure_Max	int64	Stain_2	int64
Coolant_Temp	float64	Dent_2	int64
Coolant_Temp_Min	int64	Deformation_2	int64
Coolant_Temp_Max	int64	Contamination_2	int64
Coolant_Pressure	float64	Impurity_2	int64
Factory_Temp	float64	Crack_2	int64
Factory_Temp_Min	float64	Scratch_2	int64
Factory Temp Max	float64	Buring_Mark_2	int64
		Inclusions_2	int64
		dtype: object	
		일관성 지수: 100%	

[코드 11] 분석 도구를 활용한 일관성 품질지수 구하기

▶ 정확성 품질 지수 = (1-(정확성 위배 데이터 수/전체 데이터 수))*100

① 원본 데이터와 분석에 사용하는 데이터의 차이가 있는가?

: 원본 데이터는 결측치(null) 값이 포함되어 있고, 분석에 사용하는 데이터셋은 결측치가 처리된 데이터이다.

```
dc_df.mean()
id 3.767454e+06
Product_Type 1.441672e+00
Shot 4.537989e+02
Velocity_1 1.482190e-01
Velocity_2 1.688005e-01
Velocity_3 1.911928e-01
High_Velocity 2.319210e+00
Cylinder_Pressure 2.396556e+02
Rapid_Rise_Time 9.596284e-03
Biscuit_Thickness 1.430962e+01
Clamping_Force 3.064333e+02
Cycle_Time 2.773598e+01
Pressure_Rise_Time 3.934811e-02
Casting_Pressure 8.569441e+02
Spray_Time 9.815979e+00
Spray_1_Time 1.409104e+00
Spray_2_Time 1.396045e+00
Melting_Furnace_Temp 6.806527e+02
Air_Pressure 6.109595e+00
Air_Pressure_Min 3.000000e+00
Air_Pressure_Max 9.000000e+00
Coolant_Temp 2.683013e+01
Coolant_Temp_Min 1.000000e+01
Coolant_Temp_Max 5.000000e+01
Coolant_Pressure 2.701155e+00
Factory_Temp 3.282968e+01
Factory_Temp_Min 1.800000e+01
Factory_Temp_Max 2.200000e+01
Factory_Humidity 6.167300e+01
Factory_Humidity_Min 1.800000e+01
Factory_Humidity_Max 2.200000e+01
Short_Shot_1 6.794957e-02
Bubble_1 9.555408e-03
Exfoliation_1 2.322495e-02
Blow_Hole_1 3.251493e-02
Stain_1 2.773723e-02
Dent_1 9.289980e-04
Deformation_1 1.446583e-02
Contamination_1 5.308560e-04
Impurity_1 2.654280e-04
Crack_1 1.327140e-04
Scratch_1 2.654280e-04
Buring_Mark_1 6.635700e-04
Inclusions_1 0.000000e+00
Short_Shot_2 2.415395e-02
Bubble_2 1.194426e-03
Exfoliation_2 1.778368e-02
Blow_Hole_2 2.136695e-02
Stain_2 0.000000e+00
Dent_2 5.308560e-04
Deformation_2 8.626410e-03
Contamination_2 1.061712e-03
Impurity_2 6.635700e-04
Crack_2 2.654280e-04
Scratch_2 0.000000e+00
Buring_Mark_2 0.000000e+00
Inclusions_2 1.327140e-04
dtype: float64
```

[코드 12] 정확성 품질지수 코드 (1)

```
dc_data.mean()
id 3.728663e+06
Product_Type 1.434923e+00
Shot 4.548727e+02
Velocity_1 1.481531e-01
Velocity_2 1.687936e-01
Velocity_3 1.910474e-01
High_Velocity 2.316849e+00
Cylinder_Pressure 2.393506e+02
Rapid_Rise_Time 9.574748e-03
Biscuit_Thickness 1.425856e+01
Clamping_Force 3.055502e+02
Cycle_Time 2.763490e+01
Pressure_Rise_Time 3.939772e-02
Casting_Pressure 8.601037e+02
Spray_Time 9.788368e+00
Spray_1_Time 1.401961e+00
Spray_2_Time 1.388744e+00
Melting_Furnace_Temp 6.808788e+02
Air_Pressure 6.100551e+00
Air_Pressure_Min 3.000000e+00
Air_Pressure_Max 9.000000e+00
Coolant_Temp 2.682192e+01
Coolant_Temp_Min 1.000000e+01
Coolant_Temp_Max 5.000000e+01
Coolant_Pressure 2.702136e+00
Factory_Temp 3.282968e+01
Factory_Temp_Min 1.800000e+01
Factory_Temp_Max 2.200000e+01
Factory_Humidity 6.167300e+01
Factory_Humidity_Min 1.800000e+01
Factory_Humidity_Max 2.200000e+01
Short_Shot_1 6.715917e-02
Bubble_1 9.670920e-03
Exfoliation_1 2.350571e-02
Blow_Hole_1 3.263936e-02
Stain_1 2.807253e-02
Dent_1 9.402283e-04
Deformation_1 1.464070e-02
Contamination_1 5.372733e-04
Impurity_1 2.686367e-04
Crack_1 1.343183e-04
Scratch_1 2.686367e-04
Buring_Mark_1 6.715917e-04
Inclusions_1 0.000000e+00
Short_Shot_2 2.444594e-02
Bubble_2 1.208865e-03
Exfoliation_2 1.799866e-02
Blow_Hole_2 2.055071e-02
Stain_2 0.000000e+00
Dent_2 5.372733e-04
Deformation_2 8.730692e-03
Contamination_2 1.074547e-03
Impurity_2 6.715917e-04
Crack_2 2.686367e-04
Scratch_2 0.000000e+00
Buring_Mark_2 0.000000e+00
Inclusions_2 1.343183e-04
dtype: float64
```

[코드 13] 정확성 품질지수 코드 (2)



② 다음 코드를 실행하여 원본 데이터와 분석 데이터의 차이로 정확도 값을 확인한다.

```
original = df_origin.mean()
train = df.mean()

print("정확성 지수: %i%%" % (((original.sum() - abs(((original - train).sum())))) / original.sum()) * 100))
```

지수: 98%

[코드 14] 분석 도구를 활용한 정확성 품질지수 구하기

▶ 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수중 } 100\% \text{가 아닌 지수 개수} / 3)) * 100$

: 본 데이터는 유일성, 유효성, 그리고 일관성 모두 100%이다. 따라서 3가지 지수 모두가 100%를 만족하므로 무결성 품질 지수는 100%이다.

▶ 가중치 지수 = 품질 지수 * 가중치

▶ 데이터 품질 지수 = $\sum$ 가중치 지수

구분	품질지수	가중치	가중치 지수	오류율
완정성 (누락)	100%	50%	50%	0%
유일성 (중복)	100%	10%	10%	0%
유효성 (유효)	100%	10%	10%	0%
일관성 (표현)	100%	10%	10%	0%
정확성 (실제)	98%	10%	9.8%	0.2%
무결성	100%	10%	10%	0%
품질지수		100%	99.8%	0.2%

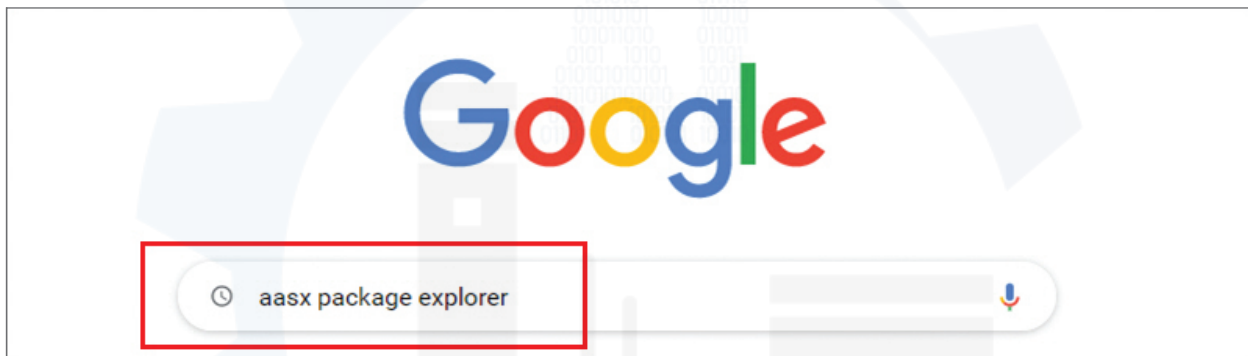
[표 8] 데이터 품질 지수

5 부록_AAS S/W 설치 가이드

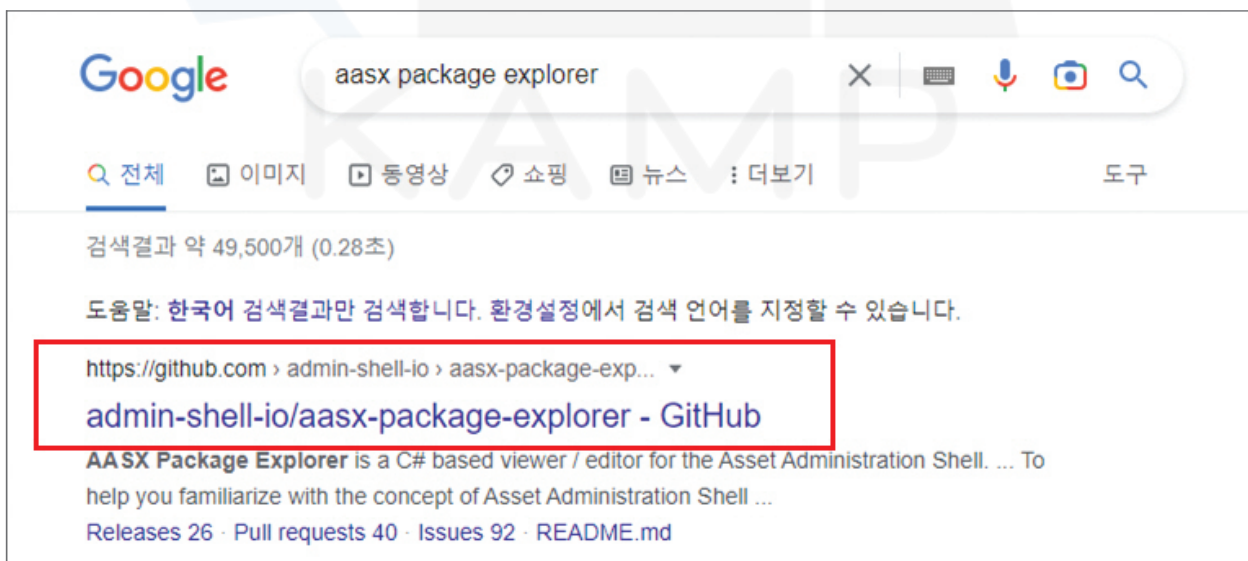
1. AASX Package Explorer 설치

AAS 모델은 독일 IDTA(Industrial Digital Twin Association)에서 제공하는 C# 기반 오픈 소스 소프트웨어 ‘AASX Package Explorer’를 사용하여 개발할 수 있다. 특히, AASX Package Explorer는 윈도우즈 10 및 윈도우즈 11 운영체제에서 실행 가능하고 GUI 또한 제공하기 때문에 처음 접하는 사용자도 어렵지 않게 AAS 모델을 개발 할 수 있다. 본 부록에서는 AAS 모델링에 필요한 소프트웨어 패키지 다운로드 방법과 AAS 파일 오픈 및 편집하기 위해 기본적인 메뉴 설명에 대해 안내한다.

① google 등의 검색 엔진에 ‘aasx package explorer’를 검색한다.



② 아래와 그림과 같이 ‘admin-shell-io/aasx-package-explorer GitHub’ 링크를 선택한다.



- ③ 페이지 하단의 Installation의 'the releases' 링크를 클릭한다.

Installation

We provide the binaries for Windows 10 in [the releases](#).

(Remark: In special cases you may like to use a current build. Please click on a green check mark and select "Check-release" details.)

- ④ aasx-package-explorer.날짜명.zip 이름의 파일을 다운로드 받는다. 단, 본 가이드북에서는 당시 최신 버전인 'aasx-package-explorer.2022-08-06.zip'를 사용

AASX Package Explorer 2022-08-06.alpha Latest

Add first version of signature ([https://github.com/admin-shell-io/aasx-package-explorer/pull/501\[\]](https://github.com/admin-shell-io/aasx-package-explorer/pull/501[]))
<https://github.com/admin-shell-io/aasx-package-explorer/commit/dc5984429699dc50a51f982a7ae273d54df73624>)
 Signing and validating of submodels or collections
 Prototype according actual work of Task Force Secure AAS
 Click on submodel or collection and choose "File / Security / Sign or Validate"
 Multiple signatures with different X509 certificates are possible

[Update test certificate](#)

▼ Assets 5

aasx-package-explorer-blazorui.2022-08-06.zip	3.63 MB	06 Aug 2022
aasx-package-explorer-small.2022-08-06.zip	36.9 MB	06 Aug 2022
aasx-package-explorer.2022-08-06.zip	115 MB	06 Aug 2022
Source code (zip)		05 Aug 2022
Source code (tar.gz)		05 Aug 2022

- ⑤ 다운로드한 'aasx-package-explorer.2022-08-06.zip' 파일의 압축을 해제한다.

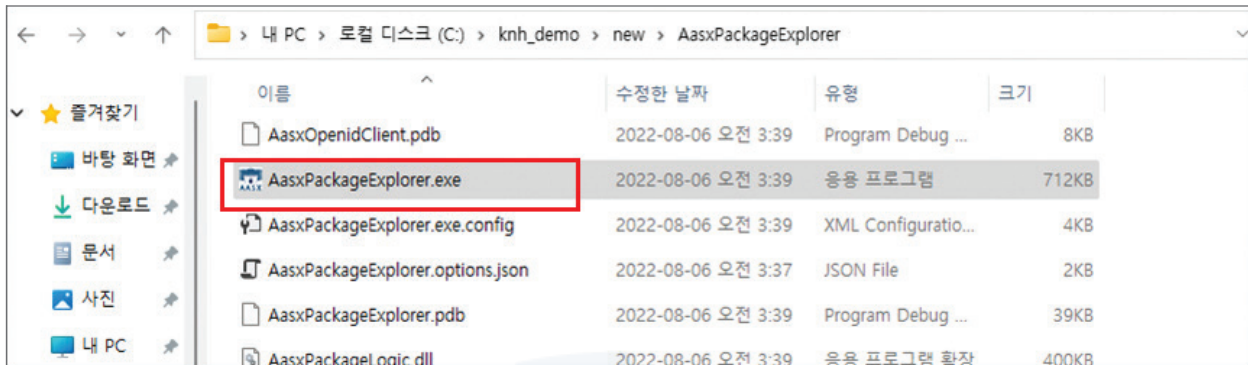
new

새로 만들기 | | | | | | | 정렬 | 보기 | ...

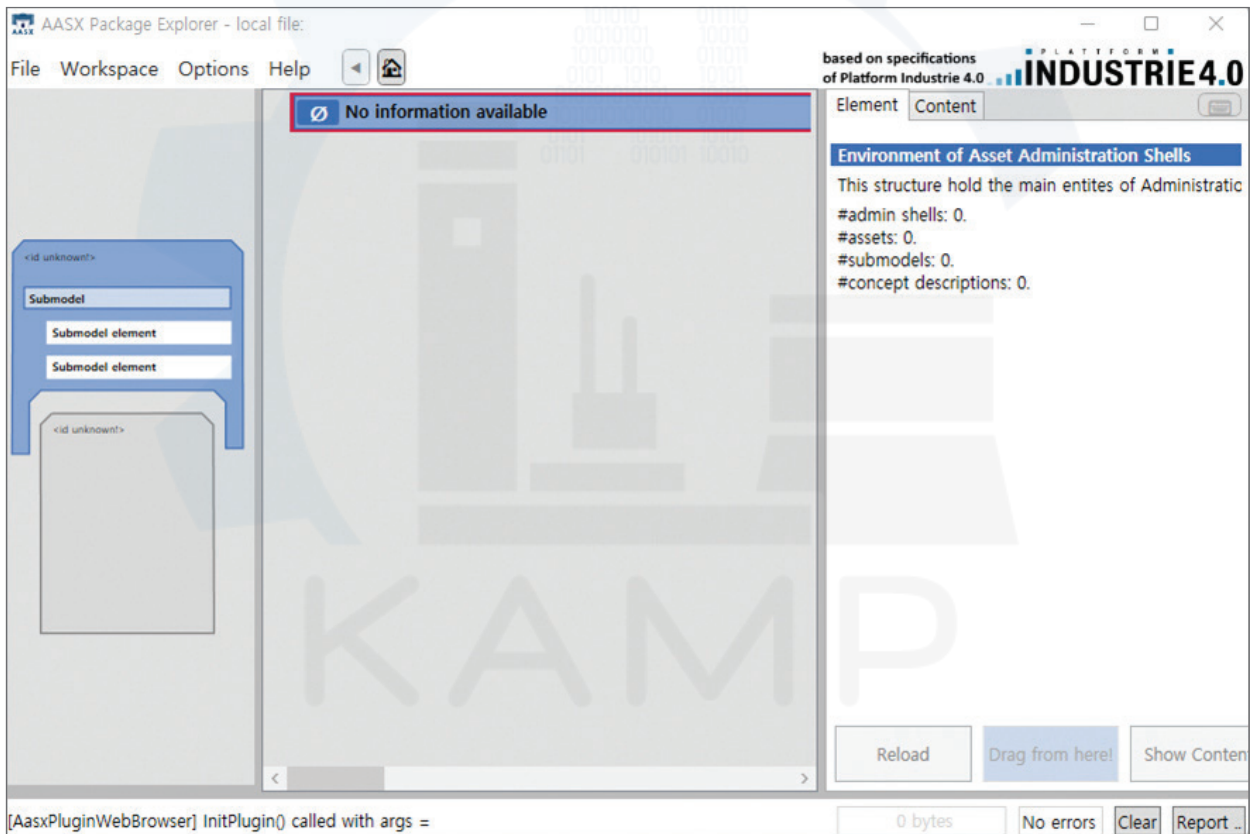
← → ^ ↑ > 내 PC > 로컬 디스크 (C:) > knh_demo > new

이름	수정한 날짜	유형	크기
AasxPackageExplorer	2022-10-27 오후 1:15	파일 폴더	
aasx-package-explorer.2022-08-06.zip	2022-10-27 오후 1:13	압축(ZIP) 파일	117,850KB

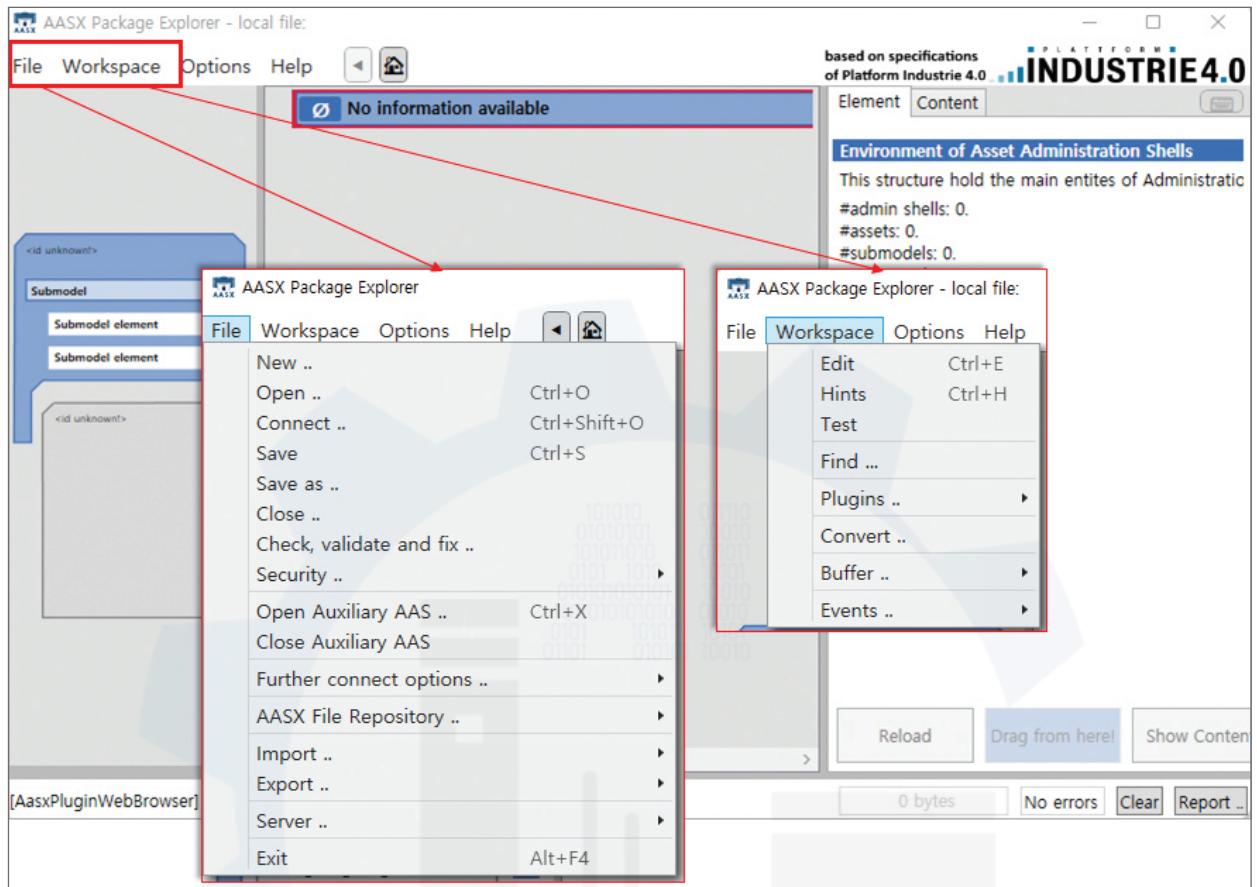
- ⑥ 디렉토리 'AasxPackageExplorer'을 열어서 실행파일인 'AasxPackageExplorer.exe'를 클릭하여 프로그램을 실행한다.



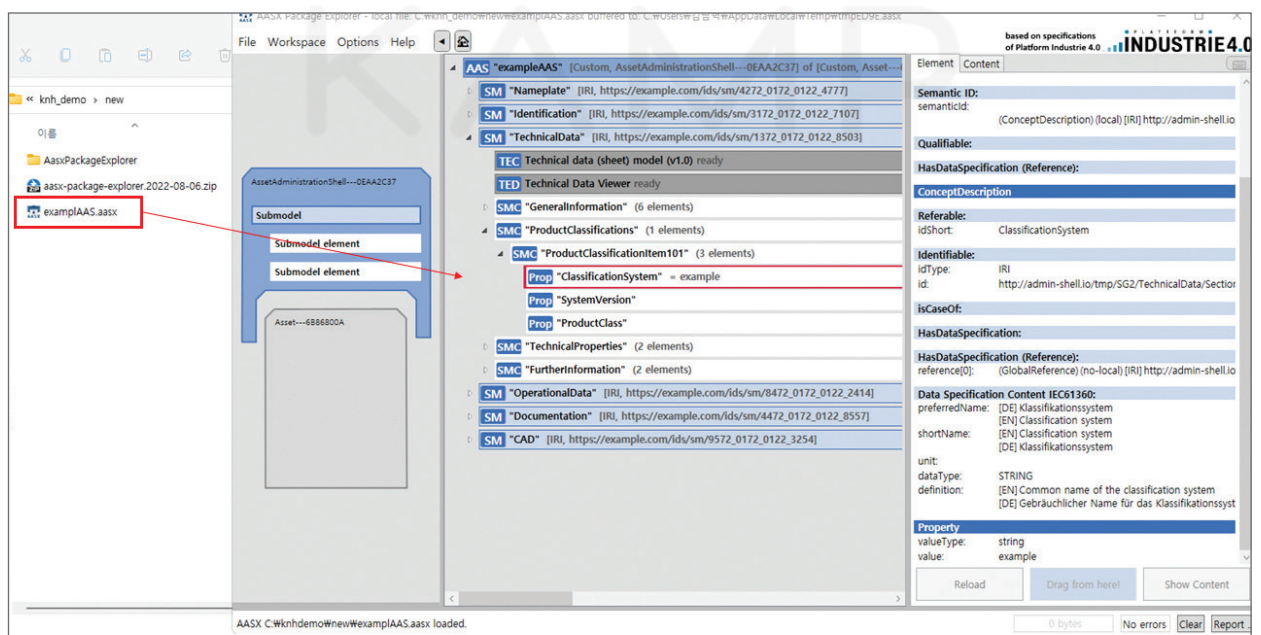
- ⑦ 아래와 같이 실행화면이 나타나면 AASX Package Explorer가 정상적으로 잘 동작하는 것이므로 AAS 개발 환경이 준비되었음을 의미한다.

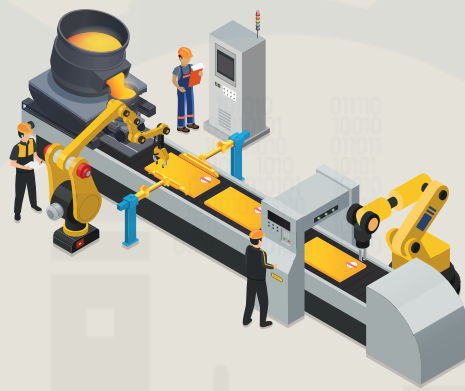


- ⑧ AASX Package Explorer에서 AAS 생성/편집/뷰 기능은 아래와 같이 'File'과 'Workspace' 메뉴를 통해 사용 가능하다. 기본 확장자는 'aasx'이며, 필요한 경우 'Save as' 또는 'Export' 기능을 통해 JSON이나 XML 확장자로 변환하여 사용 할 수 있다.



- ⑨ aasx 파일은 AASX Package Explorer의 File 메뉴에서 'Open'을 클릭하거나 화면에 aasx 파일을 드래그앤 드롭하여 작성된 AAS 모델을 뷰/편집 할 수 있다.





「주조 품질보증 AI 데이터셋」 분석실습 가이드북